



Linux
Professional
Institute

Success your DevOps Tools Engineer Exam

Mai 2020

Assakra Radhouen

Preface:

In 2013 , while browsing LinkedIn I stumbled across an article talking about something called “DevOps” . As I read it, I became increasingly excited as I realized DevOps’s revolutionary potential .

The company I’d worked at for over a decade was struggling to deliver software quickly enough. Provisioning environments was a costly, time-consuming, manual, and inelegant affair. Continuous integration was barely existent, and setting up development environments were an exercise in patience. As my job title included the words “DevOps Engineer,” I was peculiarly motivated to solve these problems!

In 2018, and after some years of DevOps practicing I decided to pass the LPI-701 DevOps tools engineer exam

I checked the LPI site , but unfortunately I can’t find a documentation about the exam. I used the official

documentation of each element included in the exam and of course I took notes , After a hard work I took the

certification on November 2018 and today I’m here to share all notes in a book and I hope that all people pass the exam.

About Me:

My name is Radhouen Assakra, DevOps , full stack engineer and Author . Certified DevOps LPI-701 engineer tools and I have a lot of badges about multiple technologies like Git , Kubernetes , Docker, and others .

Cofounder of devopstutorial.tech blogs to share knowledge and love with you.

For more details about me thanks to check :

- My LinkedIn: <https://www.linkedin.com/in/assakraradhouen/>
- My Twitter: [@assakraradhouen](https://twitter.com/assakraradhouen)
- My Github: <https://github.com/radhouen>

Plan

- **Module 1 : Modern software development .**
- **Module 2 : Component , platforms and cloud deployment .**
- **Module 3 : Source code management**
- **Module 4 : System image creation and VM Deployment .**
- **Module 5 : Container usage .**
- **Module 6 : Container infrastructure .**
- **Module 7 : Container deployment and orchestration .**
- **Module 8 : CI / CD .**
- **Module 9 : Ansible and configuration management tools.**
- **Module 10 : IT monitoring .**
- **Module 11 : Log management and analysis.**





Linux
Professional
Institute

Module 1

Modern Software Development

Plan

- **From agile to DevOps .**
- **Test-Driven Development.**
- **Service based applications.**
- **Micro-services architecture .**
- **Application security risks.**

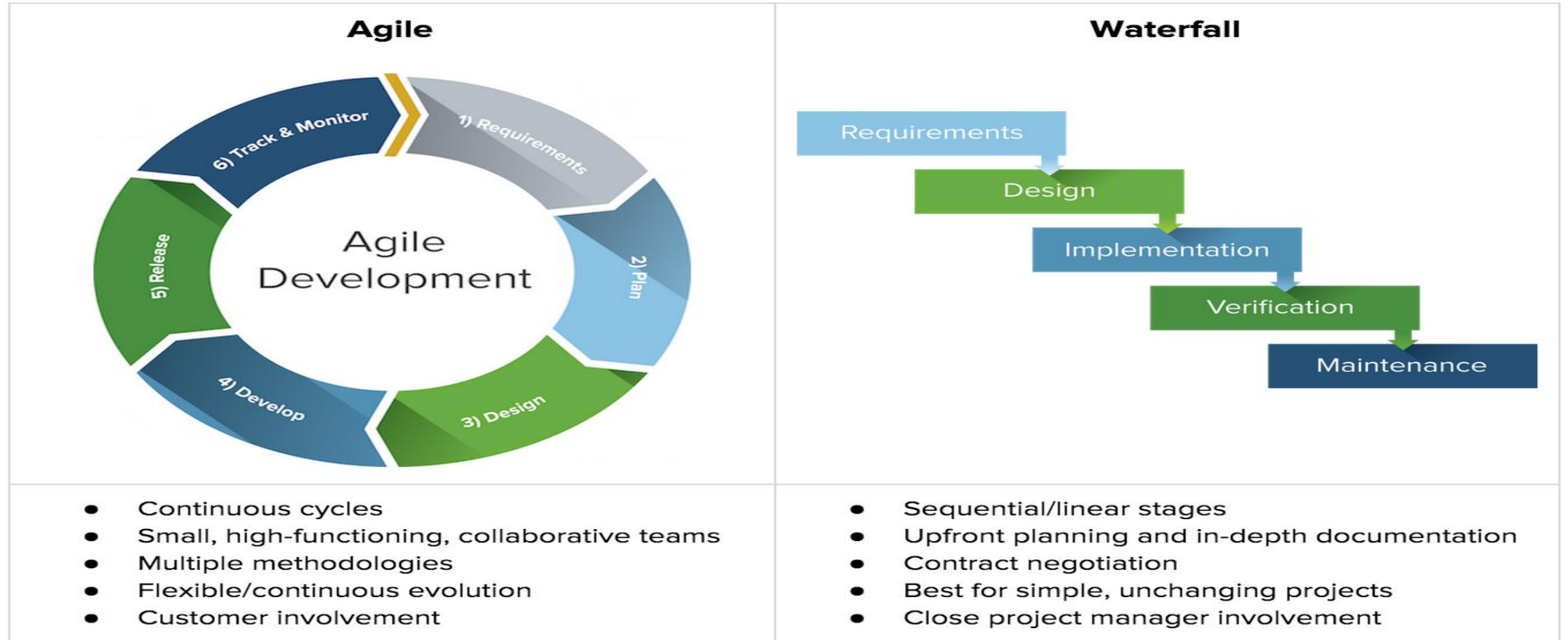


From Agile to DevOps : what is Agile

- An interactive approach which focuses on collaboration, customer feedback, and small, rapid releases .
- Helps to manage complex projects.
- Method can be implemented within a range of tactical frameworks like a sprint, safe and scrum.
- Agile development is managed in units of "sprints." This time is much less than a month for each sprint
- When the software is developed and released, the agile team will not care what happens to it.
- Scrum is most common methods of implementing Agile software development.
- Others agile methodologies :
 - Extreme Programming (XP)
 - Kanban
 - Feature-Driven Development (FDD)



From Agile to DevOps : Agile VS Waterfall



Advantages of the Agile Model

- It is focused client process. So, it makes sure that the client is continuously involved during every stage.
- Agile teams are extremely motivated and self-organized so it likely to provide a better result from the development projects.
- Agile software development method assures that quality of the development is maintained
- The process is completely based on the incremental progress. Therefore, the client and team know exactly what is complete and what is not. This reduces risk in the development process.

Limitation of the Agile Model

- It is not useful method for small development projects.
- It requires an expert to take important decisions in the meeting.
- Cost of implementing an agile method is little more compared to other development methodologies.
- The project can easily go off track if the project manager is not clear what outcome he/she wants.

Limitation of the waterfull Model

- It is not an ideal model for a large size project
- If the requirement is not clear at the beginning, it is a less effective method.
- Very difficult to move back to makes changes in the previous phases.
- The testing process starts once development is over. Hence, it has high chances of bugs to be found later in development where they are expensive to fix.

Conclusion

Agile and Waterfall are very different software development methodologies and are good in their respective way. However, there are certain major differences highlighted below Waterfall model is ideal for projects which have defined requirements, and no changes are expected. On the other hand, Agile is best suited where there is a higher chance of frequent requirement changes.

The waterfall is easy to manage, sequential, and rigid method.

Agile is very flexible and it possible to make changes in any phase.

In Agile process, requirements can change frequently. However, in a waterfall model, it is defined only once by the business analyst.

In Agile Description of project, details can be altered anytime during the system development life cycle “SDLC” process which is not possible in Waterfall method.

Transforming IT service delivery with DevOps by using Agile

When it comes to improving IT performance in order to give organizations competitive advantages, we need a new way of thinking, a new way of working that improve all the production and management processes and operations from the team or project level to the organizational level while encouraging collaboration between all the individuals involved for fast delivery of valuable products and services.

For this reason, a new culture, corporate philosophy and way of working is emerging. This way of working integrates agile methods, lean principles and practices, social psychological beliefs for motivating workers, systems thinking for building complex systems, continuous integration and continuous improvement of IT products and services for satisfying both customers and production and development teams. This new way of working is DevOps.

What's DevOps

Adam Jacobs in a presentation defined DevOps as “a cultural and professional movement, focused on how we build and operate high velocity organizations, born from the experiences of its practitioners”. This guru of DevOps also states that DevOps is reinventing the way we run our businesses. Moreover, he argue that DevOps is not the same but unique to the people who have practiced it (Jacobs, 2015).

Gartner analysts declare that DevOps “... is a culture shift designed to improve quality of solutions that are business-oriented and rapidly evolving and can be easily molded to today's needs” (Wurster, et al., 2013).

Thus, DevOps is a movement that integrates different ways of thinking and different ways of working for transforming organizations by improving IT services and products delivery.

How to successfully integrate DevOps culture in an organization

We cannot talk about DevOps in a corporate environment without integrating a set of principles and practices that make development and operations teams work together. For this reason, Gartner analysts support that DevOps takes into account several commonly agreed practices which form the fundamentals of DevOps practices. These practices are (Wurster, et al., 2013):

- Cross-functional teams and skills
- Continuous delivery :DevOps strives for deadlines and benchmarks with major releases. The ideal goal is to deliver code to production DAILY or every few hours.
- Continuous assessment :Feedback comes from the internal team
- Optimum utilization of tool-sets
- Automated deployment pipeline
- It's essential for the operational team to fully understand the software release and its hardware/network implications for adequately running the deployment process.

Here are the 6 Cs of DevOps

1. Continuous Business Planning

This starts with identifying the skills, outcomes, and resources needed.

2. Collaborative Development

This starts with development sketch plan and programming.

3. Continuous Testing

Unit and integration testing help increase the efficiency and speed of the development.

4. Continuous Release and Deployment

A nonstop CD pipeline will help you implement code reviews and developer check-ins easily.

5. Continuous Monitoring

This is needed to monitor changes and address errors and mistakes spontaneously whenever they happen.

6. Customer Feedback and Optimization

This allows for an immediate response from your customers for your product and its features and helps you modify accordingly.

Here are the 6 Cs of DevOps

Taking care of these six stages will make you a good DevOps organization. This is not a must-have model but it is one of the more sophisticated models. This will give you a fair idea on the tools to use at different stages to make this process more lucrative for a software-powered organization.

CD pipelines, CI tools, and containers make things easy. When you want to practice DevOps, having a microservices architecture makes more sense.

Conclusion

- **Agile**

- Software development method emphasis on iterative, incremental, and evolutionary development.
- Iterative approach which focuses on collaboration, customer feedback, and small, rapid releases.
- Priority to the working system over complete documentation

- **DevOps**

- Software development method focuses on communication, integration, and collaboration among IT professionals.
- Practice of bringing development and operations teams together.
- Process documentation is foremost : it will send the software to the operational team for deployment.
- DevOps is a culture, it's an agile's extension

What is TDD ?

Test-driven development is a software development process that relies on the repetition of a very short development cycle: requirements are turned into very specific test cases, then the software is improved so that the tests pass.

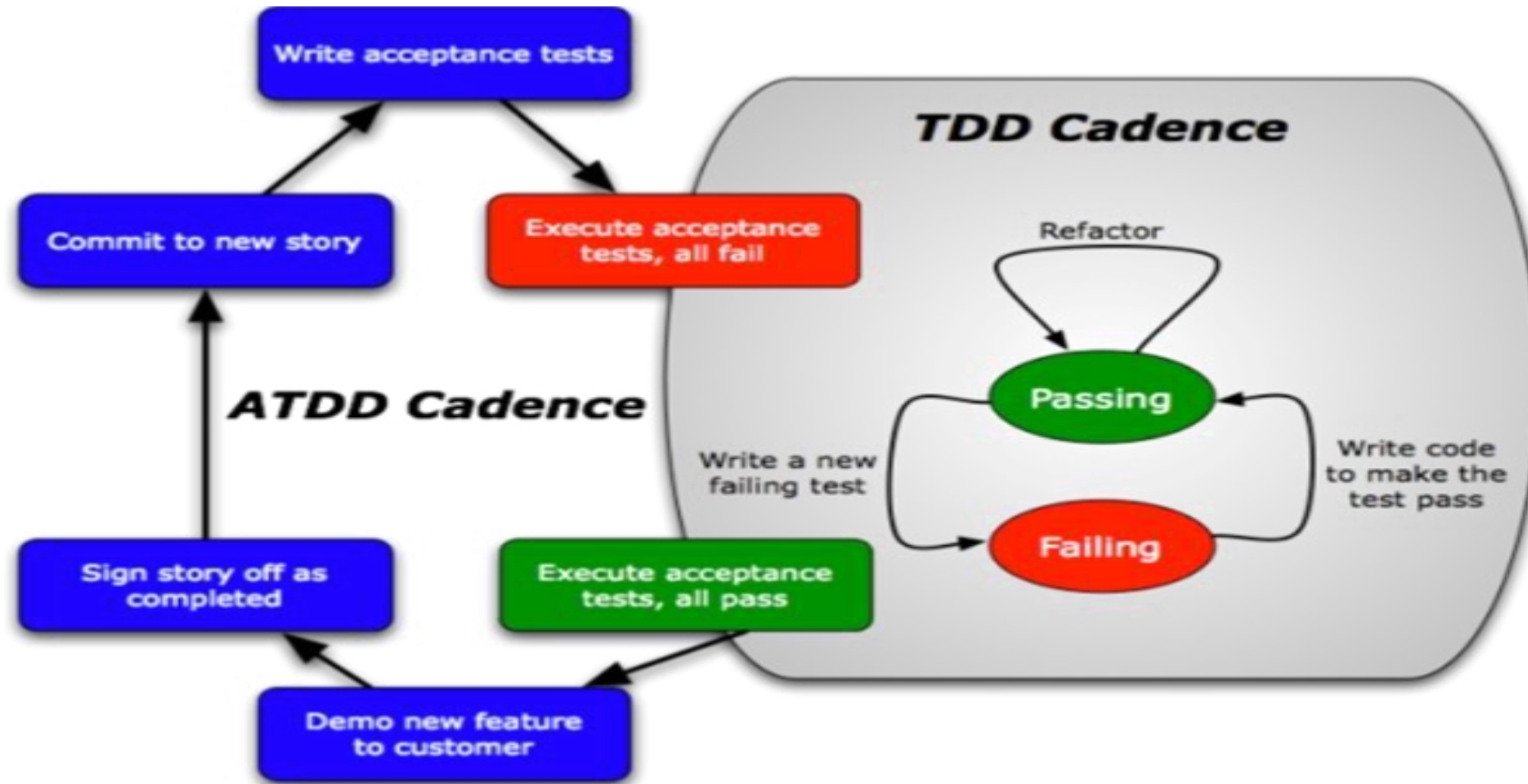
It refers to a style of programming in which three activities are nested:

- Coding.
- Testing (in the form of writing unit tests).
- Refactoring

TDD cycles

- Write a "single" unit test describing an aspect of the program.
- Run the test, which should fail because the program lacks that feature.
- Write "just enough" code, the simplest possible, to make the test pass.
- "refactor" the code until it conforms to the simplicity criteria.
- Repeat, "accumulating" unit tests over time

What is TDD ?



Service based applications

Application architecture

Why does application architecture matter?

- Build a product can scale.
- To distribute.
- Helps with speed to market
- Application architectures:
- Monolithic Architecture
- SOA Architecture
- Microservices Architecture

Service based applications: Monolithic architecture

- Synonymous with n-Tier applications.
- Separate concerns and decompose code base into functional components.
- Building a single web artifact and then trying to decompose the application into layers.
 - Presentation Layer
 - Business Logic Layer
 - Data Access Layer.
- Massive coupling issues :
 - Every time you have to build, test, or deploy.
 - Infrastructure costs : add resources for the entire application to single code scaling.
 - Bad performing part of your software architecture can bring the entire
 - structure down

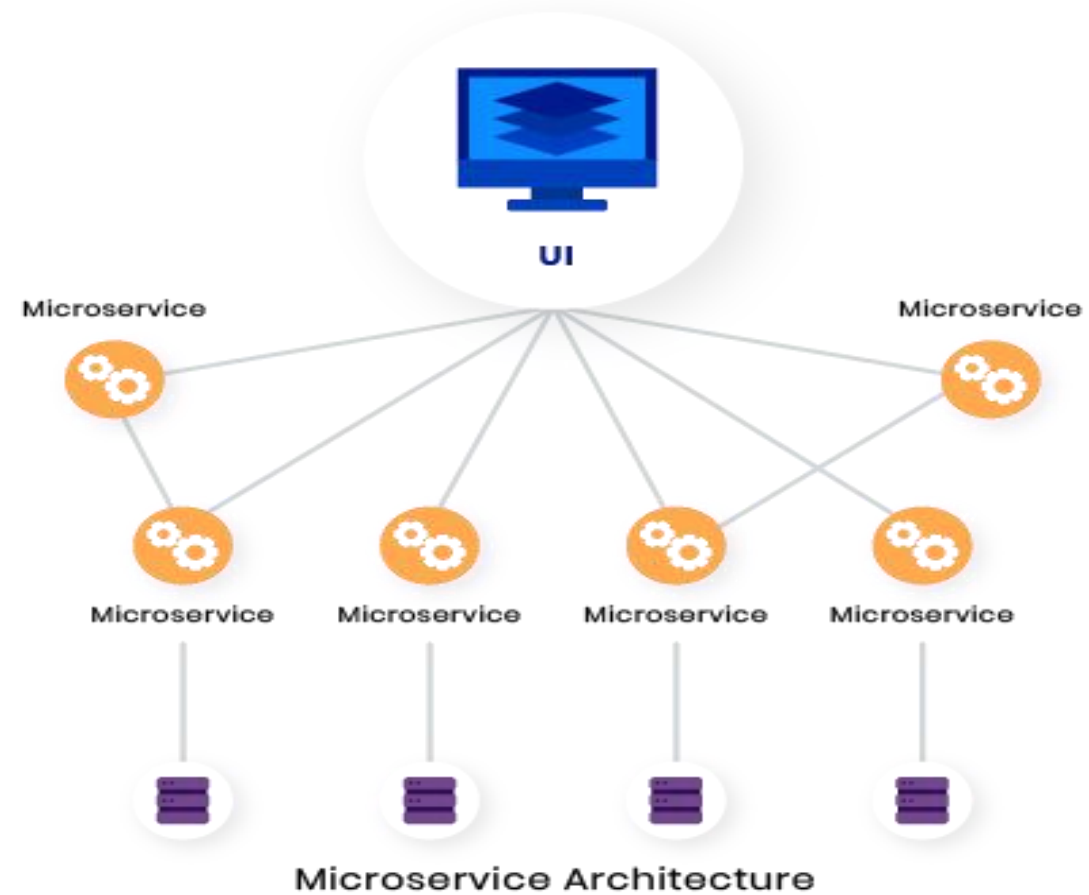
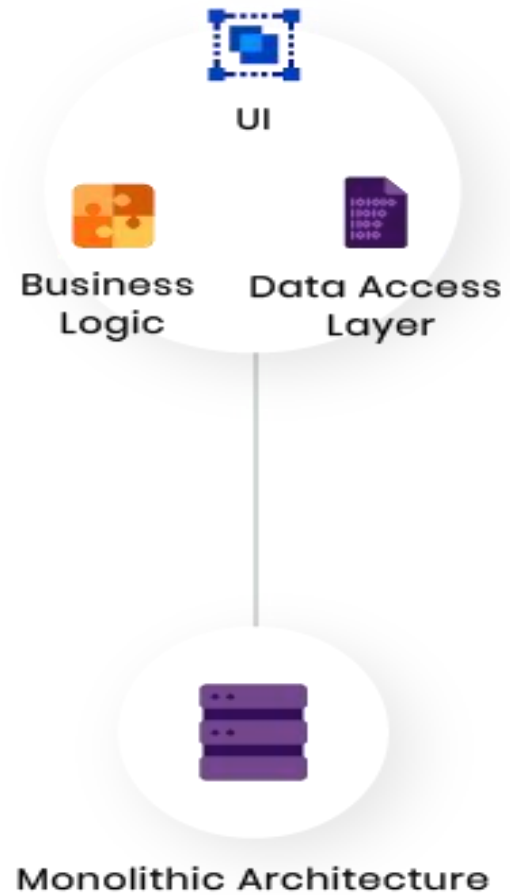
Service based applications: SOA architecture

- Service-based architecture
- Decouple your application in smaller modules.
- Good way of decoupling and communication.
- Separates the internal and external elements of the system.
- All the services would then work with an aggregation layer that can be termed as a bus.
- As SOA Bus got bigger and bigger with more and more.
- components added to the system $\Rightarrow$ issues of system coupling.

Service based applications:Micro-services architecture

- Evolution to the limitation of the SOA architecture.
- Decoupling or decomposition of the system into discrete work units.
- Use business cases, hierarchical, or domain separation to define each micro-service.
- Can use different languages or frameworks to work together.
- All the communication between the services in over REST over HTTP.
- Also renders itself well suited for the cloud-native deployment.
- <https://microservices.io/>
- <https://rubygarage.org/blog/monolith-soa-microservices-serverless>

Micro-services vs Monolith



Restful API

What is an API ?

- Application Program Interface
- APIs are everywhere
- Contract provided by one piece of software to another
- Structured request and response

Restful API

What is an REST ?

- Representational State Transfer.
- Architecture style for designing networked applications.
- Relies on a stateless, client-server protocol, almost always HTTP.
- Treats server objects as resources that can be created or destroyed.
- Can be used by virtually any programming language.

Restful API

REST Methods:

- <https://www.restapitutorial.com/lessons/httpmethods.html>
- **GET** : Retrieve data from a specified resource
- **POST** : Submit data to be processed to a specific resource.
- **PUT** : Update a specified resource
- **DELETE** : Delete a specified resource
- **HEAD** : Same as GET but does not return a body
- **OPTIONS** : Returns the supported HTTP methods
- **PATCH** : Update partial resources

Restful API

REST Endpoint :

The URI/URL where API/service can be accessed by a client application.

HTTP code status :

<https://www.restapitutorial.com/httpstatuscodes.html>

Authentication

Some API's require authentication to use their service. This could be free or paid.

Demo: REST API demo created by using GO.

Restful API : what's JSON

- JSON : JavaScript Object Notation
- A lightweight data-interchange format
- Easy for humans to read and write
- Easy for machines to parse and generate.
- Responses from the server should be always in
- JSON format and consistent.
- Always contain meta information and, optionally, data

Application security risks

Most security risks :

- SQL injection / LDAP injection
- Broken authentication
- Broken access control
- Cross-site scripting (XSS)
- Cross-site request forgery (CSRF)
- Unvalidated redirects and forwards
- Etc ...

Application security risks

How to prevent attacks ?

- Using special database features to separate commands from data.
- Authentication without passwords (cryptography private keys, bio-metrics, smart card, etc ...)
- Using the Cross-Origin Resource Sharing (CORS) headers to prevent Cross-site request forgery “CSRF”
- Avoid using redirects and forwards whenever possible. At least prevent users from affecting the destination.

Application security risks

CORS headers and CSRF tokens

- CSRF allows an attacker to make unauthorized requests on behalf of an authenticated user.
- Commands are sent from the user's browser to a web site or a web application.
- CORS handles this vulnerability well : disallows the retrieval and inspection of data from another Origin
(While allowing some cross-origin access)
- It prevent the third-party JavaScript from reading data out of the image, and will fail AJAX requests with a security error.

“XMLHttpRequest cannot load <https://app.mixmax.com/api/foo>. No 'Access-Control-Allow-Origin' header is present on the requested resource. Origin 'http://evil.example.com' is therefore not allowed access.”



Linux
Professional
Institute

Module 1

Components, platforms and cloud deployment

Plan

- **Data platforms and concepts**
- **Message brokers and queues**
- **Paas platforms**
- **OpenStack**
- **Cloud-init**
- **Content Delivery Networks**



Data platforms and concepts

Relational database:

- Based on the relational model of data.
- Relational database systems use SQL.
- Relational model organizes data into one or more tables.
- Each row in a table has its own unique key (primary key).
- Rows in a table can be linked to rows in other tables by adding a foreign keys.
- MySQL (MariaDB), Oracle, Postgres, IBM DB2 etc ...

Data platforms and concepts

NoSQL database:

- Mechanism for storage and retrieval of data other than the tabular relations used in relational databases.
- Increasingly used in big data and real-time web applications

Properties :

- Simplicity of design
- Simpler scaling to clusters of machines (problem for relational databases)
- Finer control over availability.
- Some operations faster (than relational DB)

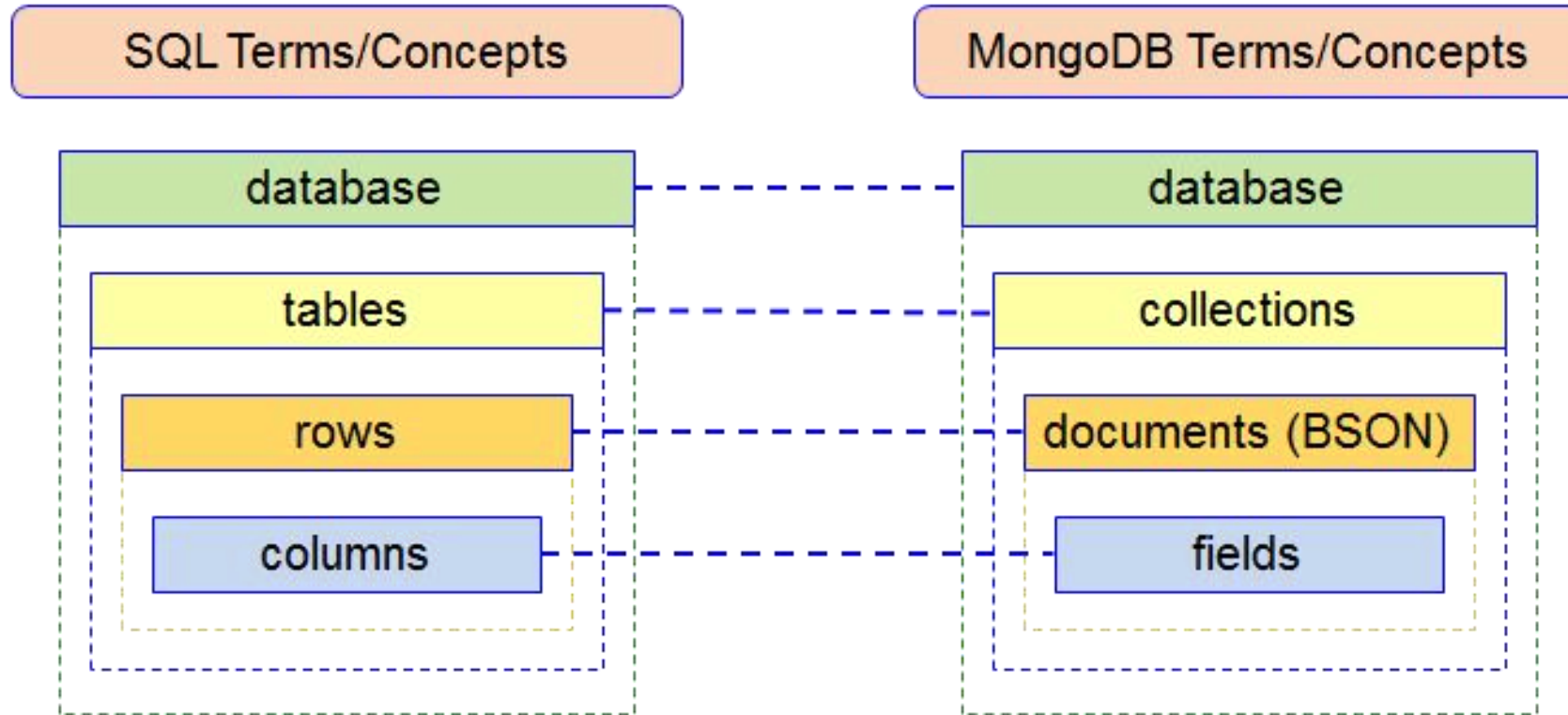
Various ways to classify NoSQL databases :

- Document Store : MongoDB, etc ...
- Key-Value Cache : Memcached, Redis, etc ...

Data platforms and concepts: SQL vs NoSQL

	NoSQL	SQL
Model	Non-relational	Relational
	Stores data in JSON documents, key/value pairs, wide column stores, or graphs	Stores data in a table
Data	Offers flexibility as not every record needs to store the same properties	Great for solutions where every record has the same properties
	New properties can be added on the fly	Adding a new property may require altering schemas or backfilling data
	Relationships are often captured by denormalizing data and presenting it in a single record	Relationships are often captured in a using joins to resolve references across tables
	Good for semi-structured data	Good for structured data
Schema	Dynamic or flexible schemas	Strict schema
	Database is schema-agnostic and the schema is dictated by the application. This allows for agility and highly iterative development	Schema must be maintained and kept in sync between application and database
Transactions	ACID transaction support varies per solution	Supports ACID transactions
Consistency	Consistency varies per solution, some solutions have tunable consistency	Strong consistency supported
Scale	Scales well horizontally	Scales well vertically

Data platforms and concepts: SQL vs NoSQL



Data platforms and concepts

Object storage:

Manages data as objects

Opposed to other storage architectures :

- File systems : manages data as a file hierarchy
- Block storage : manages data as blocks
- Watch Block storage vs file storage

Each object typically includes :

- The data itself,
- Metadata (additional information)
- A globally unique identifier.

can be implemented at multiple levels :

Device level (SCSI device, etc ...)

System level (used by some distributed file systems)

Cloud level (Openstack swift, AWS S3, Google Cloud Storage)

Data platforms and concepts

CAP theorem:

CAP : Consistency, Availability and Partition-tolerance.

It is impossible for a distributed data store to simultaneously provide more than two out of the three guarantees :

- **Consistency** : Receive the same information, regardless the node that process the order.
- **Availability** : the system provides answers for all requests it receives, even if one or more nodes are down.
- **Partition-tolerance** : the system still Works even though it has been divided by a network failure.

Data platforms and concepts

ACID properties :

ACID : Atomicity, Consistency, Isolation and Durability

Set of properties of database transactions intended to guarantee validity even in the event of errors, power failures, etc ...

- **Atomicity** : each transaction is treated as a single "unit", which either succeeds completely, or fails completely.
- **Consistency** (integrity): Ensures that a transaction can only bring the database from one valid state to another, maintaining database invariant (only starts what can be finished).
- **Isolation**: two or more transactions made at the same time must be independent and do not affect each other.
- **Durability**: If a transaction is successful, it will persist in the system (recorded in non-volatile memory)

Data platforms and concepts

Message brokers

A message broker acts as an intermediary platform when it comes to processing communication between two applications.

An architectural pattern for message validation, transformation, and routing.

Take incoming messages from applications and perform some action on them :

- **Divide** the publisher and consumer
- **Store** the messages
- **Route** messages
- **Check** and organize messages

Two fundamental architectures:

- Hub-and-spoke
- Message bus.

Examples of message broker software:

- AWS SQS, RabbitMQ, Apache Kafka, ActiveMQ, Openstack Zaqar, Jboss Messaging, ...

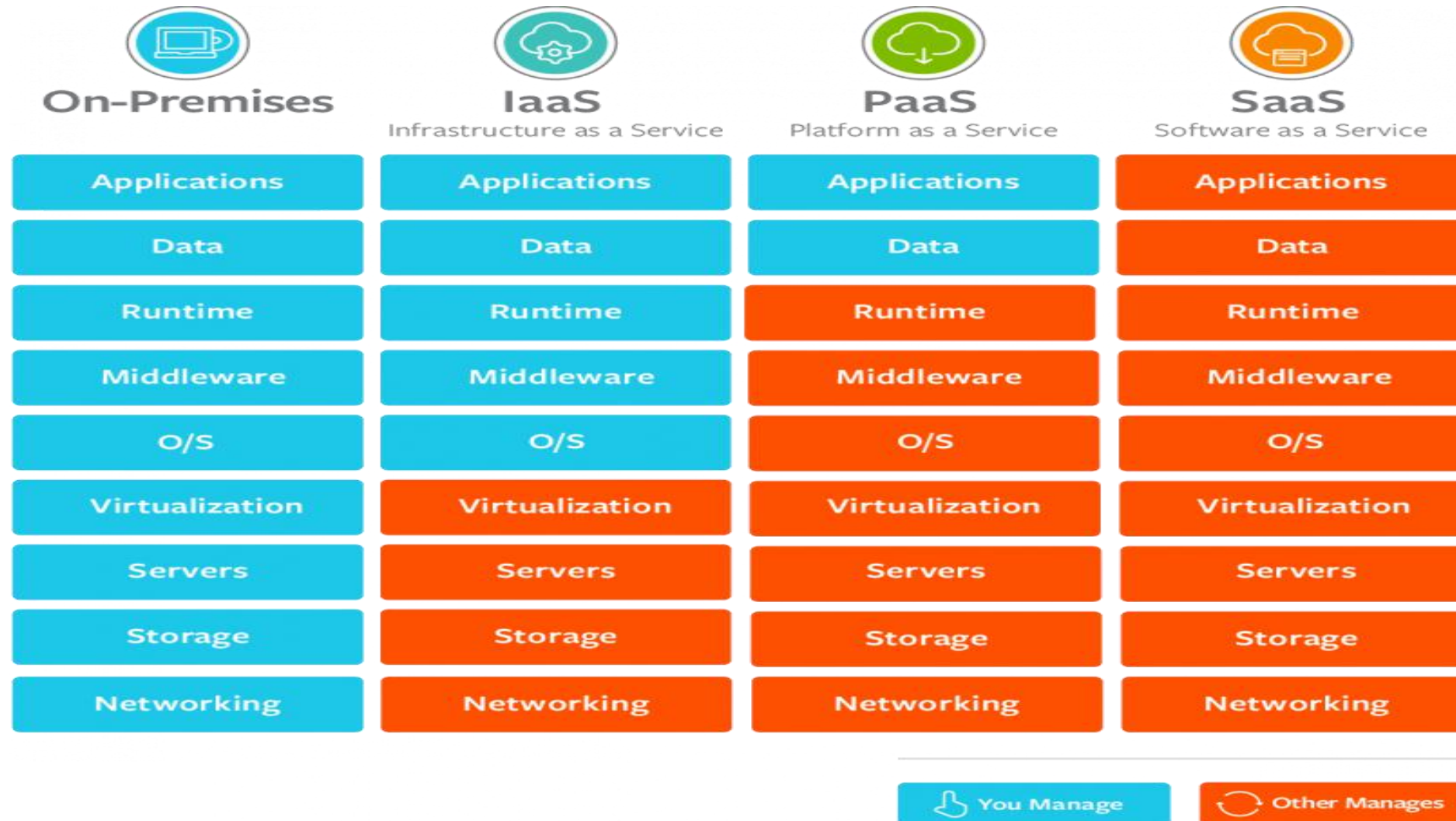
Message brokers and queues

Message brokers

Actions handled by broker :

- Manage a message queue for multiple receivers.
- Route messages to one or more destinations.
- Transform messages to an alternative representation.
- Perform message aggregation, decomposing messages into multiple messages and sending them to their destination, then recomposing the responses into one message to return to the user.
- Respond to events or errors.

PaaS Platforms



PaaS Platforms : Cloud PaaS software

- AWS Lambda
- Plesk
- Google Cloud Functions
- Azure Web Apps
- Oracle Cloud PaaS
- OpenShift
- Cloud Foundry
- Etc ...

PaaS Platforms : CloudFoundry

- Open source PaaS governed by the Cloud Foundry Foundation.
- Promoted for continuous delivery : supports the full application development life cycle (from initial development through all testing stages to deployment)
- Container-based architecture : runs apps in any programming language over a variety of cloud service providers.
- platform is available from either the Cloud Foundry Foundation as open-source software or from a variety of commercial providers as either a software product or delivered as a service.
- In a platform, all external dependencies (databases,messaging systems, files systems, etc ...) are considered services.

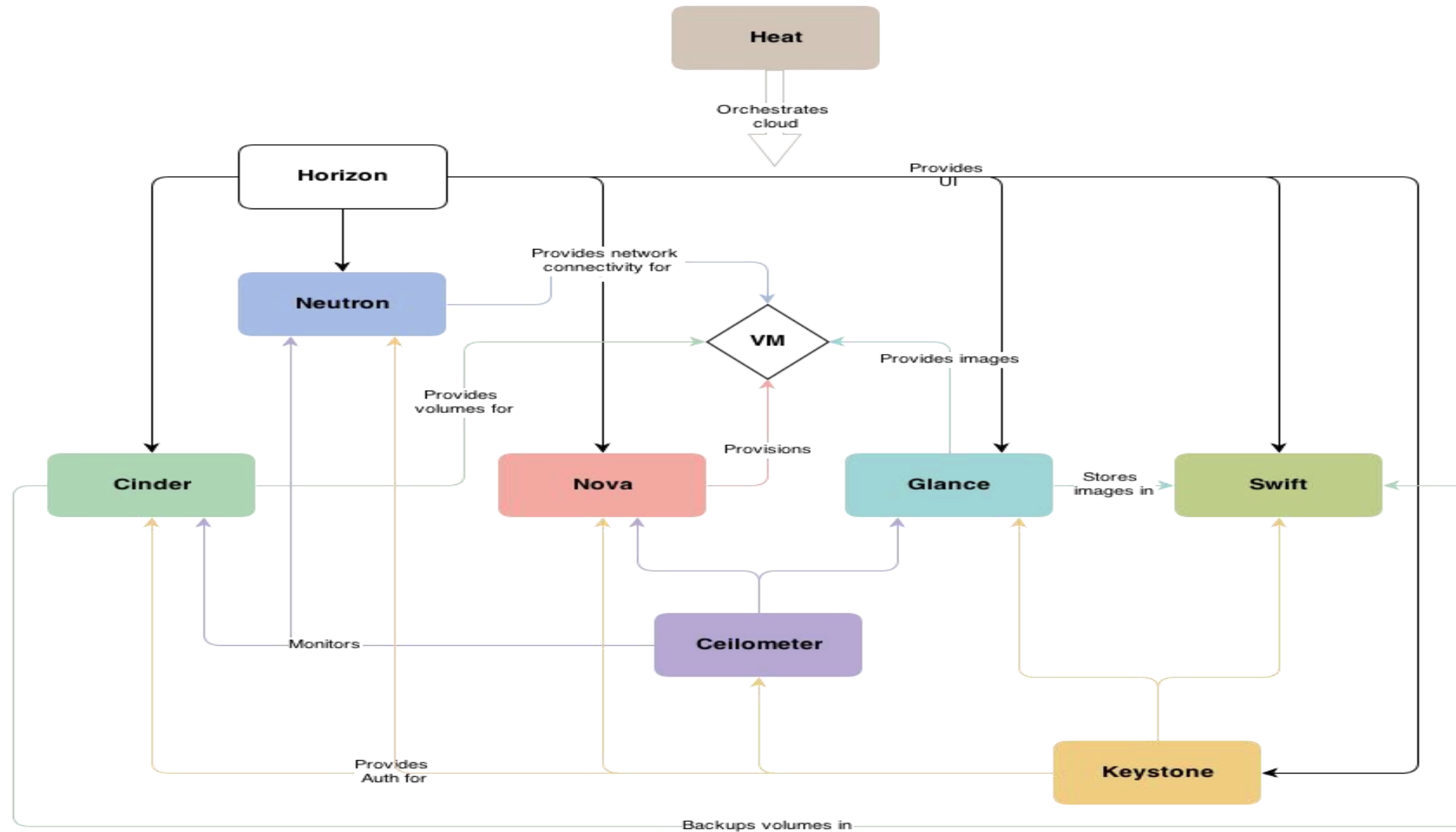
PaaS Platforms : OpenShift

- Open source cloud PaaS developed by Red Hat.
- Used to create, test, and run applications, and finally deploy them on cloud.
- Capable of managing applications written in different languages (Node.js, Ruby, Python, Perl, and Java).
- It is extensible : helps the users support the application written in other languages).
- It comes with various concepts of virtualization as its abstraction layer:
 - Uses an hyper-visor to abstract the layer from the underlying hardware.

PaaS Platforms : Openstack

- free and open-source software platform for cloud computing, mostly deployed as IaaS.
- virtual servers and other resources are made available to customers
- interrelated components that control diverse, multi-vendor hardware pools of processing, storage, and networking resources throughout a data center.
- Managed through a web-based dashboard, command-line tools, or RESTful API.
- Latest release : Stein / 10 April 2019; 3 months ago.
- OpenStack Component : Compute(Nova) , Image Service (Glance) , Object Storage(Swift)
- Block Storage(Cinder) ,Messaging Service (Zaqar), Dashboard(Horizon) , Networking(Neutron) ...
- OpenStack Components

PaaS Platforms : Openstack Architecture



OpenStack Components:

- **Heat:** Service to orchestrate multiple composite cloud using templates.
- **Horizon:** Web UI for openstack services
- **Cinder:** Block storage for openstack (be consumed by openstack compute project (NOVA)).
- **Neutron:** Openstack networking
- **Nova:** provides a way to provision compute instances , nova support :
 - virtual machine
 - Baremetals
 - has limited support for sustem containers
- **Ceilometer:** data collections services that collects event and metering data by monitoring notifications set from openstack services. It's publishes collected data to various target including data stores and message queues.
- **Swift:** distributed Object storage system
- **Keystone:** Provide Identity and authentication for all openstack services.

Cloud Init : what's cloud init ?

- Cloud-init allows you to customize a new server installation during its deployment using data supplied in YAML configuration files.
- Supported user data formats:
- Shell scripts (starts with `#!/`)
- Cloud config files (starts with `#cloud-config`)*
- Etc ...
- Modular and highly configurable.

Cloud Init : Modules

- cloud-init has modules for handling:
 - Disk configuration
 - Command execution
 - Creating users and groups
 - Package management
 - Writing content files
 - Bootstrapping Chef/Puppet/Ansible
 - Additional modules can be written in Python if desired.

Cloud Init : what can do with it ?

- Injects SSH keys.
- Grows root filesystems.
- Setting the hostname.
- Setting the root password.
- Setting locale and time zone.
- Running custom scripts.
- Etc ...

ACID properties

In computer science, ACID (**atomicity, consistency, isolation, durability**) is a set of properties of database transactions intended to guarantee data validity despite errors, power failures, and other mishaps.

- **Atomicity:** Transactions are often composed of multiple statements. Atomicity guarantees that each transaction is treated as a single "unit", which either succeeds completely, or fails completely: if any of the statements constituting a transaction fails to complete, the entire transaction fails and the database is left unchanged

for example, in an application that transfers funds from one account to another, the atomicity property ensures that, if a debit is made successfully from one account, the corresponding credit is made to the other account.

ACID properties

- **Consistency**

Consistency ensures that a transaction can only bring the database **from one valid state to another**, maintaining database invariants: any data written to the database must be valid according to all defined rules, including constraints, cascades, triggers, and any combination thereof.

For example, in an application that transfers funds from one account to another, the consistency property ensures that the total value of funds in both the accounts is the same at the start and end of each transaction.

ACID properties

- **Isolation:**

The intermediate state of a transaction is invisible to other transactions. As a result, transactions that run concurrently appear to be serialized.

For example, in an application that transfers funds from one account to another, the isolation property ensures that another transaction sees the transferred funds in one account or the other, but not in both, nor in neither.

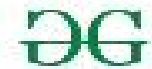
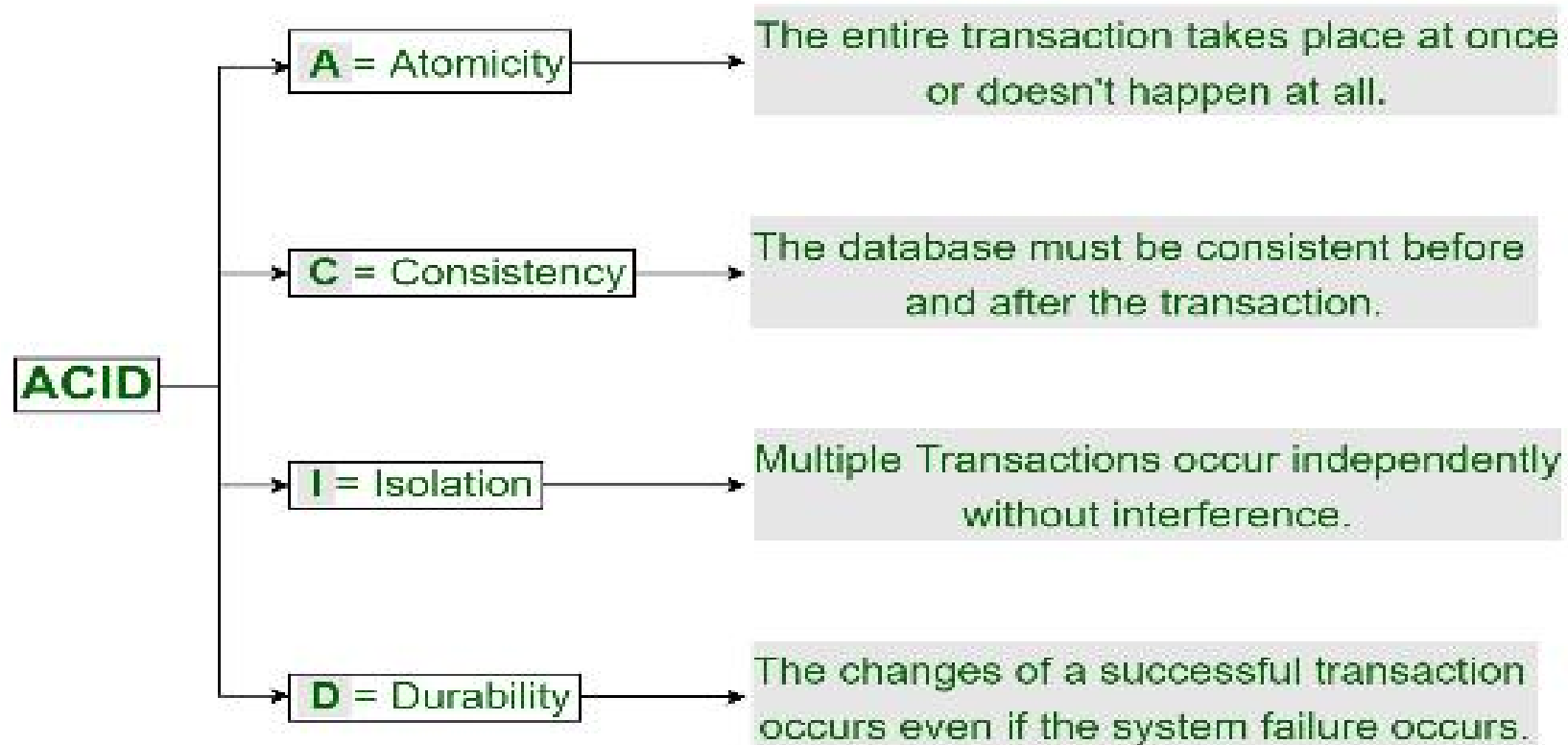
ACID properties

- **Durability:**

Durability guarantees that once a transaction has been committed, it will remain committed even in the case of a system failure (e.g., power outage or crash). This usually means that completed transactions (or their effects) are recorded in non-volatile memory.

For example, in an application that transfers funds from one account to another, the durability property ensures that the changes made to each account will not be reversed.

ACID Properties in DBMS





Linux
Professional
Institute

Module 3

Source code management

Plan

- **Whats is Version Controller**
- **Why we need Version Controller**
- **SCM Solutions , Centralized vs Distributed**
- **Configure git environment**
- **Git VS GitHub , git service hosting**
- **Git life cycle**
- **Git work flow**
- **Create GitHub repository**
- **Git Commands**
- **Git Branch**
- **Git Merge vs Rebase**
- **Git Flow**
- **Gitkraken**
- **Quiz**



- SCM – Source Code Management
- SCM involves tracking the modifications to code.
- Tracking modifications assists development and
- collaboration by :
 - Providing a running history of development
 - Helping to resolve conflicts when merging contributions from
 - multiple sources.
 - Software tools SCM are sometimes referred to as :
 - "Source Code Management Systems" (SCMS)
 - "Version Control Systems" (VCS)
 - "Revision Control Systems" (RCS)- or simply "code repositories"



- Version control, also known as revision control or source control.
- The management of changes to :
 - Documents
 - Computer programs
 - Large web sites
 - Other collections of information...
- Changes are usually identified by a number or letter code.
 - Example : revision1, revision2, ...
- Each revision is associated with a timestamp and the person making the change.
- Revisions can be compared, restored, and with some types of files, merged

Why use SCM ???



1. Version Control
2. Working in teams
3. Centralized storage of your code
4. Get involved / Open Source
5. Bettering your code
6. Show off

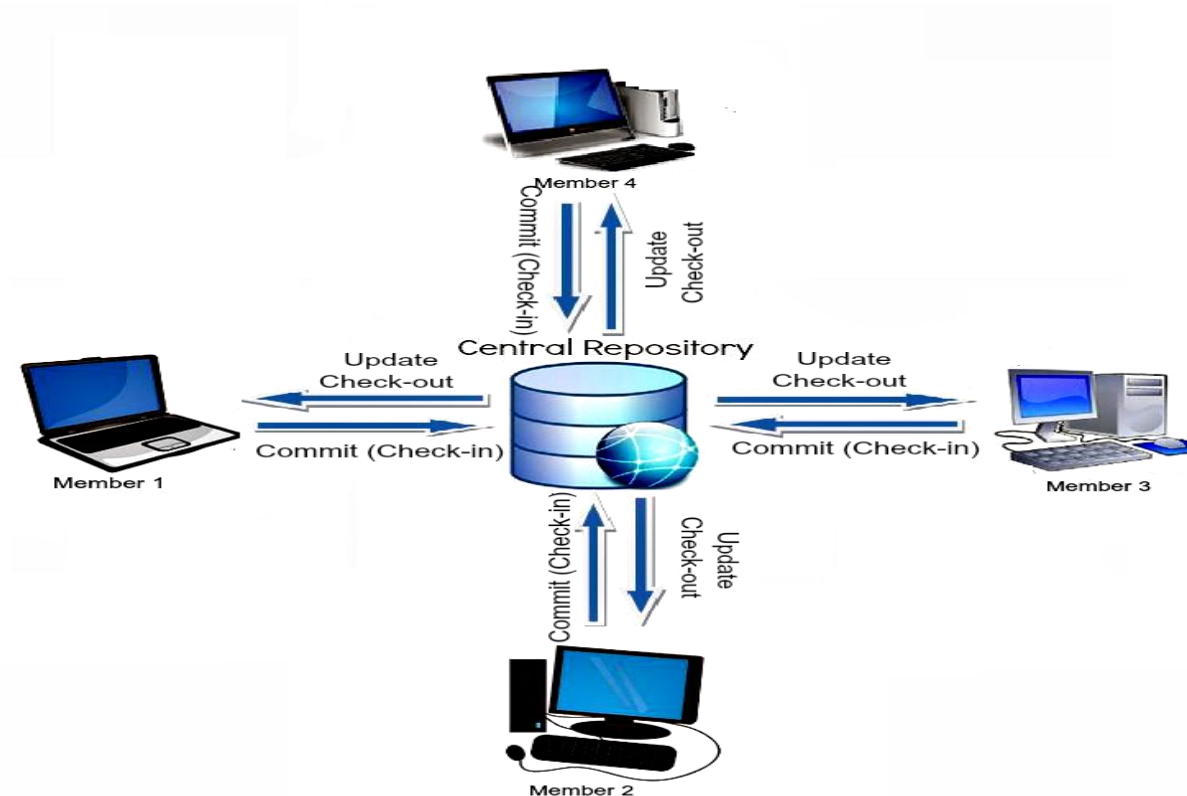


- Two types of version control: centralized and distributed.
- **Centralized version control :**
 - Have a single “central” copy of your project on a server.
 - Commit changes to this central copy
 - Never have a full copy of project locally
 - Solutions : CVS, SVN (Subversion)
- **Distributed version control :**
 - Version control is mirrored on every developer's computer.
 - Allows branching and merging to be managed automatically.
 - Ability to work offline (Allows users to work productively when not connected to a network)
 - Solutions : Git, Mercurial.

SCM solutions: Centralized vs Distributed Version Control Systems



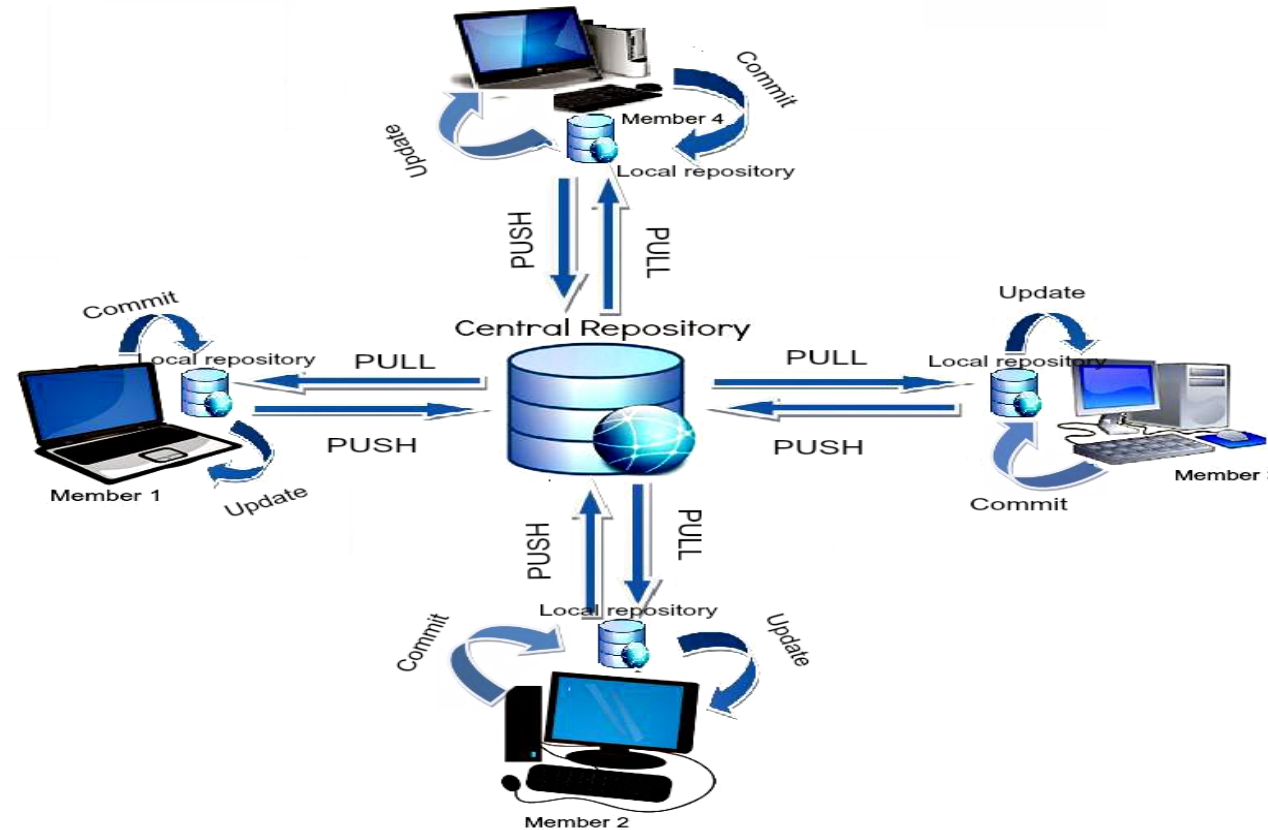
- The concept of a centralized system is that it works on a Client-Server relationship. The repository is located at one place and provides access to many clients."



SCM solutions: Centralized vs Distributed Version Control Systems



- Whereas, in a Distributed System, every user has a local copy of the repository in addition to the central repo on the server side.



Git concepts and repository structure



- Git is a distributed SCM system.
- Initially designed and developed by **Linus Torvalds** for Linux kernel development.
- A free software distributed under GNU General Public License version 2.
- Advantages :
 - Free and open source
 - Fast and small
 - Implicit backup
 - Secure : uses SHA1 to name and identify objects.
 - Easier branching : copy all the codes to new branch



SCM solutions: Configure your environment



- Git provides the `git config` tool, which allows you to set configuration variables. Git stores all global configurations in `.gitconfig` file, which is located in your home directory. To set these configuration values as global, add the `--global` option, and if you omit `--global` option, then your configurations are specific for the current Git repository.
- You can also set up system wide configuration. Git stores these values in the `/etc/gitconfig` file, which contains the configuration for every user and repository on the system. To set these values, you must have the root rights and use the `--system` option.
- Setting username: `$ git config --global user.name "Radhouen Assakra"`
- Setting email id: `$ git config --global user.email "askriradhouen@gmail.com"`
- Color highlighting:
 - `$ git config --global color.ui true`
 - `$ git config --global color.status auto`
 - `$ git config --global color.branch auto`
- Setting default editor: `$ git config --global core.editor vim`
- Setting default merge tool: `$ git config --global merge.tool vimdiff`
- Check Configuration: `$ git config --list`

Difference between Git and GitHub



- Git and Github are two different things. **Git is the version control system**, while GitHub **is a service for hosting Git repos** that helps people collaborate on writing software. However, they are often confounded because of their similar name, because of the fact that GitHub builds on top of Git, and because many websites and articles don't make the difference between them clear enough.





Bit Bucket



GitHub



GitLab



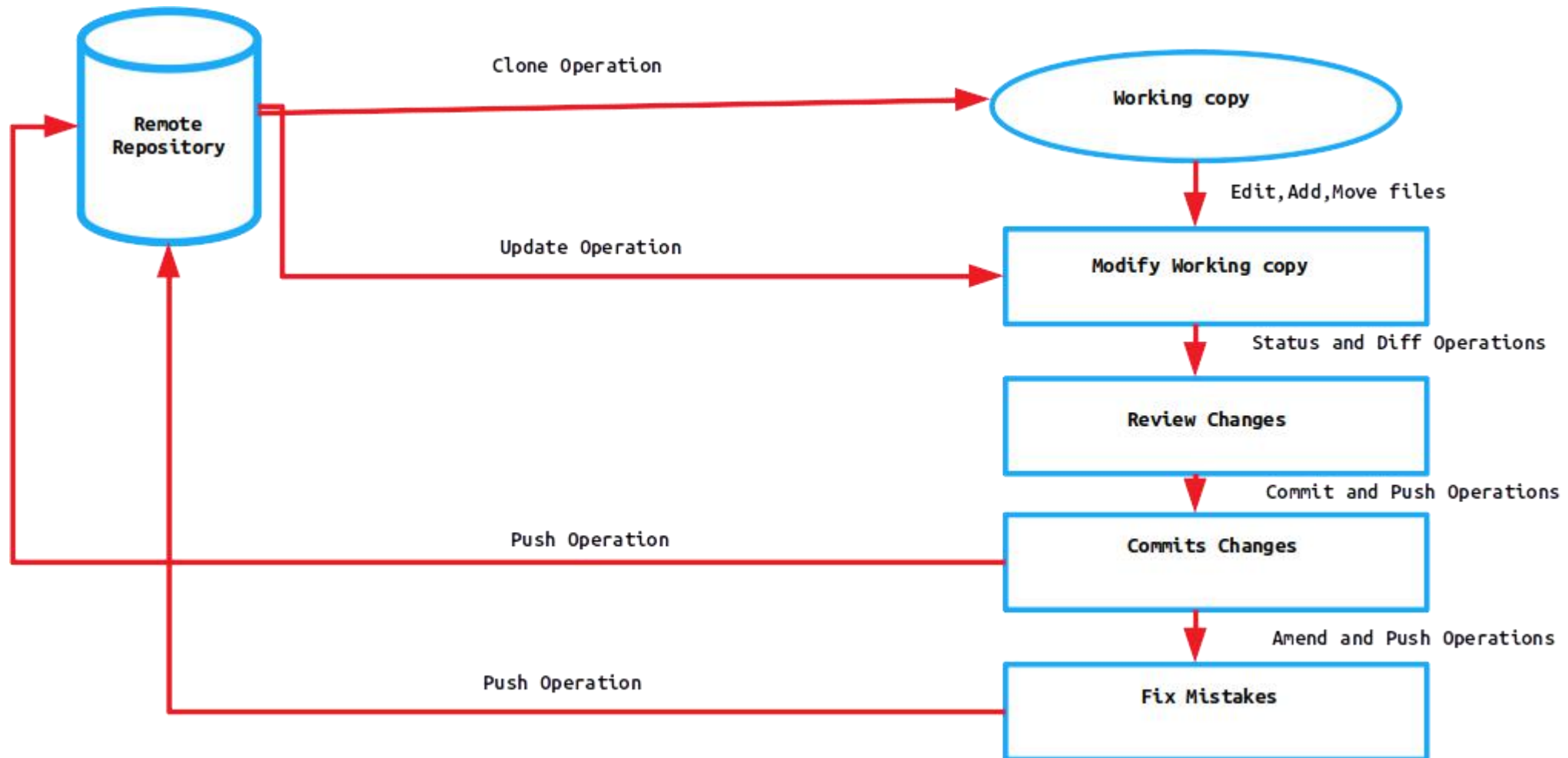
we will discuss the life cycle of Git. In later chapters, we will cover the Git commands for each operation.

General workflow is as follows :

- You clone the Git repository as a working copy.
- You modify the working copy by adding/editing files.
- If necessary, you also update the working copy by taking other developer's changes.
- You review the changes before commit.
- You commit changes. If everything is fine, then you push the changes to the repository.
- After committing, if you realize something is wrong, then you correct the last commit and push the changes to the repository.

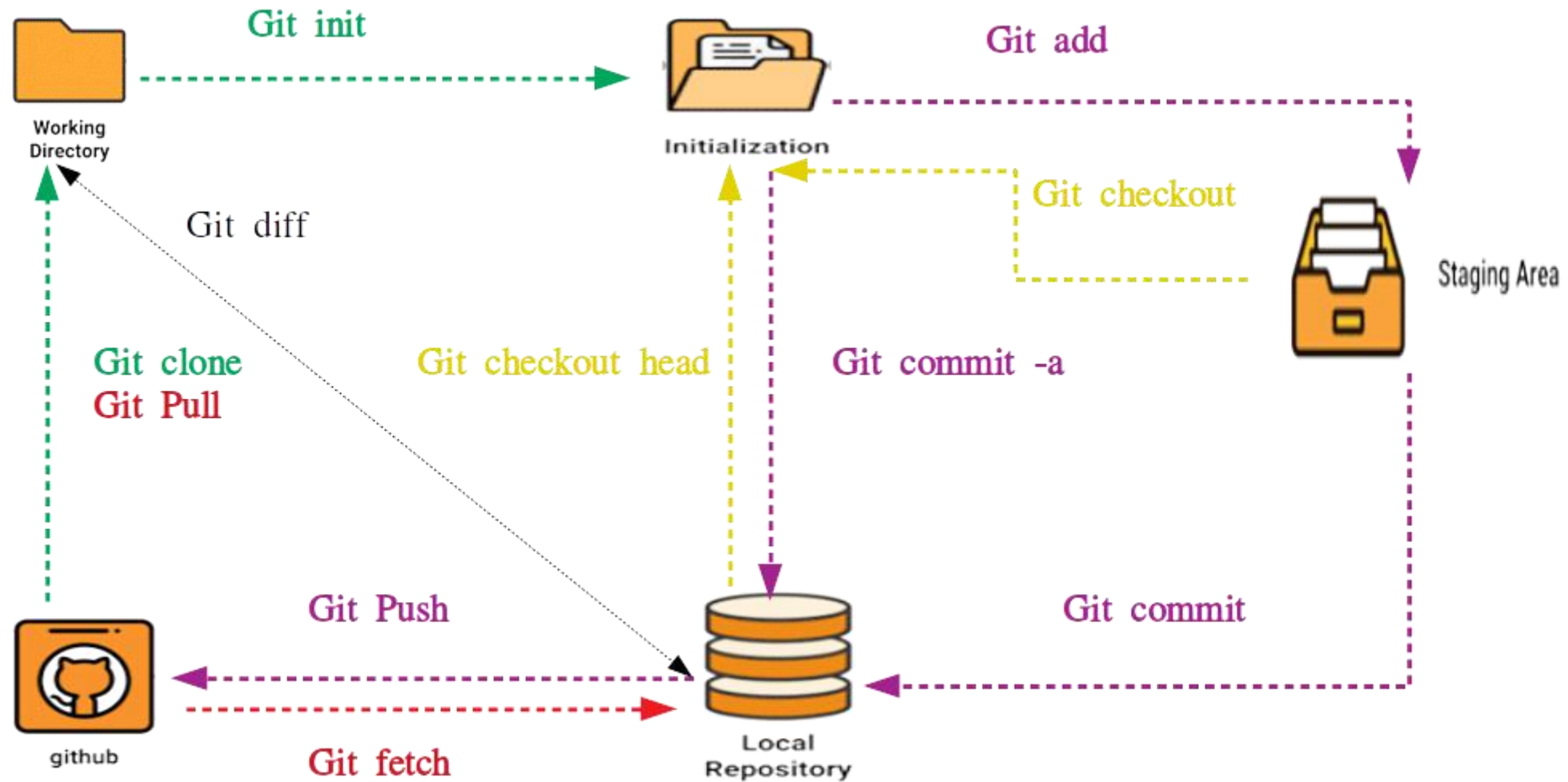
Shown below is the pictorial representation of the work-flow:

Git Life Cycle

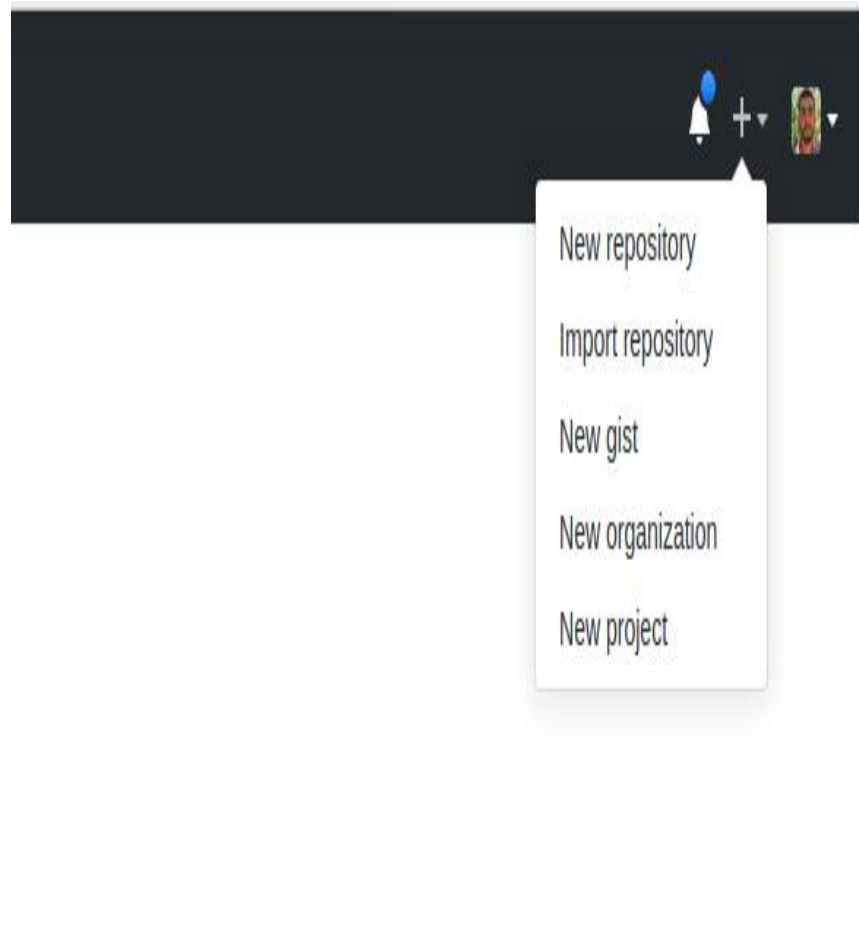


Git Life Cycle

-----Initialize
-----Update
-----Change
-----Revert



Git : Create a GitHub repository



Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Owner

 radhouen ▾

Repository name *

Great repository names are short and memorable. Need inspiration? How about [laughing-octo-sniffle?](#)

Description (optional)

☒ **Public**

Anyone can see this repository. You choose who can commit.

☐ **Private**

You choose who can see and commit to this repository.

Skip this step if you're importing an existing repository.

☐ **Initialize this repository with a README**

This will let you immediately clone the repository to your computer.

Add .gitignore: **None** ▾

Add a license: **None** ▾ ⓘ

Grant your Marketplace apps access to this repository

You are subscribed to 2 Marketplace apps

☒ 

GitHub Learning Lab

Your interactive guide to learning the skills you need without leaving GitHub

☐ 

Google Cloud Build

Build, test, and deploy in a fast, consistent, and secure manner

Create repository

Git : Create a GitHub repository



Quick setup — if you've done this kind of thing before

or ☐ HTTPS ☐ SSH



Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# test" >> README.md
git init
git add README.md
git commit -m "first commit"
git remote add origin https://github.com/radhouden/test.git
git push -u origin master
```



...or push an existing repository from the command line

```
git remote add origin https://github.com/radhouden/test.git
git push -u origin master
```



...or import code from another repository

You can initialize this repository with code from a Subversion, Mercurial, or TFS project.

[Import code](#)

Git :Staging and Commits



- Git Init : initialize git repository .git directory will be created
- Git Add : git add <fileName> , git add -A or git add . , git add -u(Modified and deleted files), git add *.go
- Git rm : Remove file from the staging Area
- Git Commit : git commit -a , git commit -m , git commit -am , git commit -amend
- Git Clone
- Git Stash
- Git Ignore
- Git Fork: A fork is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project.
- Git Repository
- Git Index : When you create a commit, what is committed is what is currently in the index, not what is in your working directory.
- Git Head : HEAD is a reference to the last commit in the currently check-out branch.
- Git Origin Master
- Git Remote
- Git Tags : Git has the ability to tag specific points in a repository's history as being important. Typically, people use this functionality to mark release points (v1.0, v2.0 and so on).



- **Git Checkout** : is the act of switching between different versions of a target entity. The git checkout command operates upon three distinct entities: files, commits, and branches. ... In the Undoing Changes topic, we saw how git checkout can be used to view old commits.
- **Git Revert** : `git revert [saved hash]` , get hash from `git log` , will revert to specific commit .
- **Git Reset** : `git reset HEAD~ --hard` : remove the last commit from the master branch .
- **Git Rm** : Remove files from the index, or from the working tree and the index
- **Git Cherry-pick** : Sometimes you don't want to merge a whole branch into another, and only need to pick one or two specific commits. This process is called 'cherry picking'.

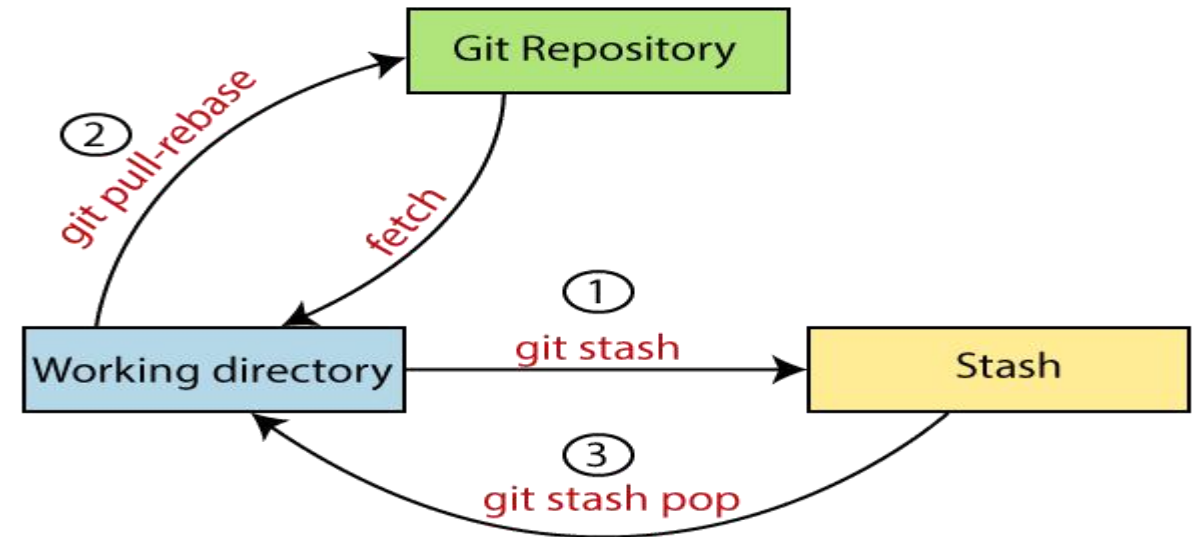


- **Git Log** : Show commit logs
- **Git Diff** : Show changes between commits, commit and working tree
- **Git Status** : displays the state of the working directory and the staging area
- **Git Blame** : Show what revision and author last modified each line of a file

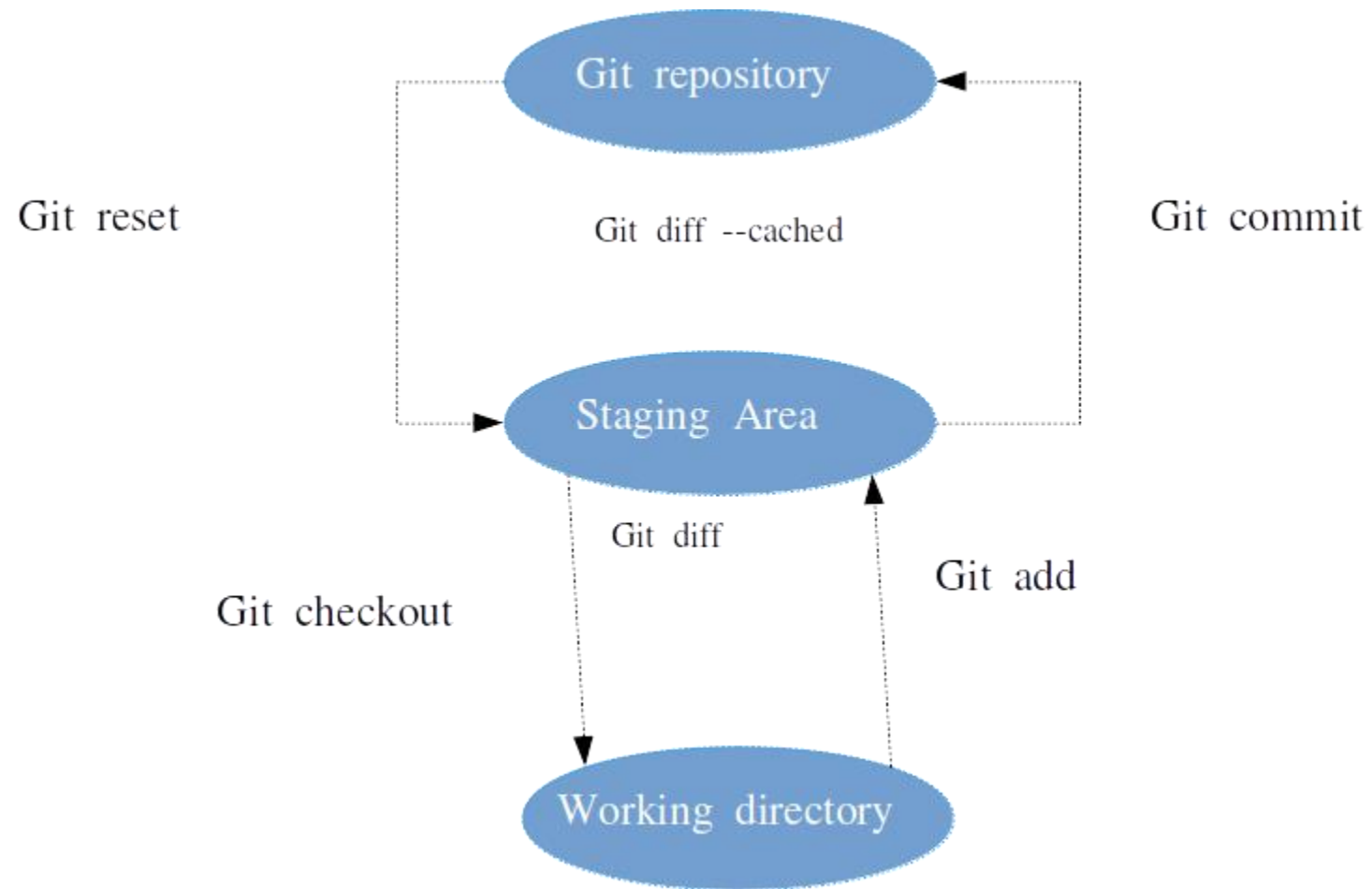
Git Stashing Changes



- **Git stash:** takes your uncommitted changes (both staged and unstaged), saves them away for later use, and then reverts them from your working copy
- **Git stash apply:** eapply the changes to your working copy and keep them in your stash
- **Git stash pop:** eapply the changes to your working copy and delete them from your stash



Git work flow :Recovering from mistakes in Git





- Git Branch
- Merge & Merge Conflict
- Git Rebase
- **Git Squash:** With git it's possible to squash previous commits into one. This is a great way to group certain changes together before sharing them with others



- **Git Fetch:** Is a primary command used to download contents from a remote repository.
- **Git Pull:** Fetch from and integrate with another repository or a local branch.
- **Git Push:** Command is used to upload local repository content to a remote repository. Pushing is how you transfer commits from your local repository to a remote repo.

Git : Gitignore Explained



A local **.gitignore** file is usually placed in the root directory of a project. You can also create a global **.gitignore** file and any entries in that file will be ignored in all of your Git repositories.

This is an example of what the **.gitignore** file could look like:

```
# Ignore Mac system files
.DS_store

# Ignore node_modules folder
node_modules

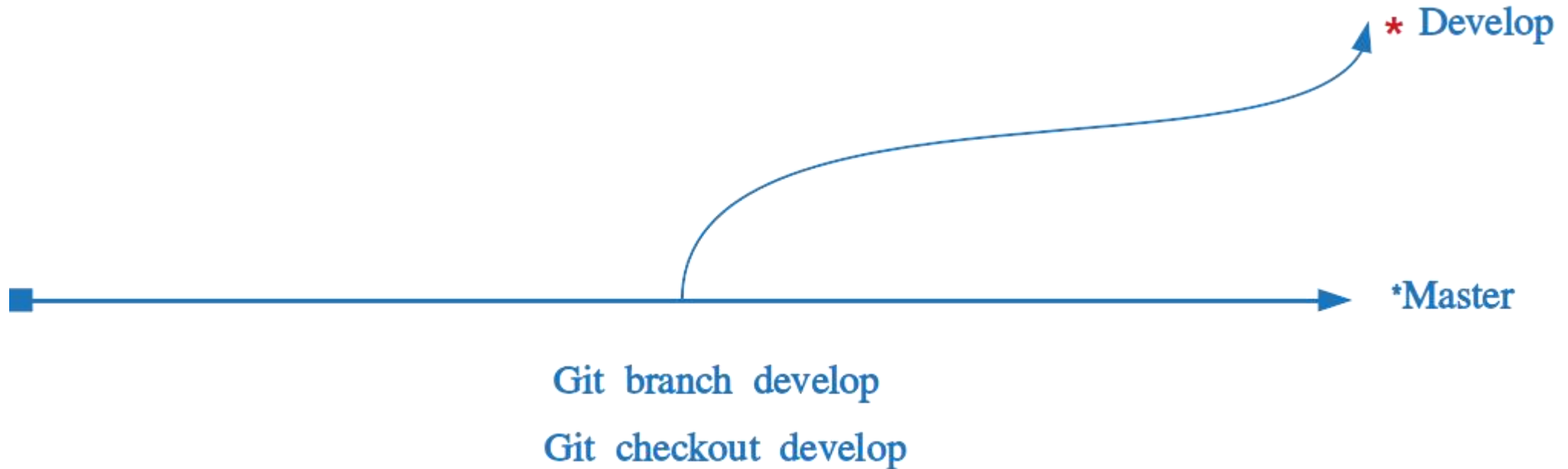
# Ignore all text files
*.txt

# Ignore files related to API keys
.env

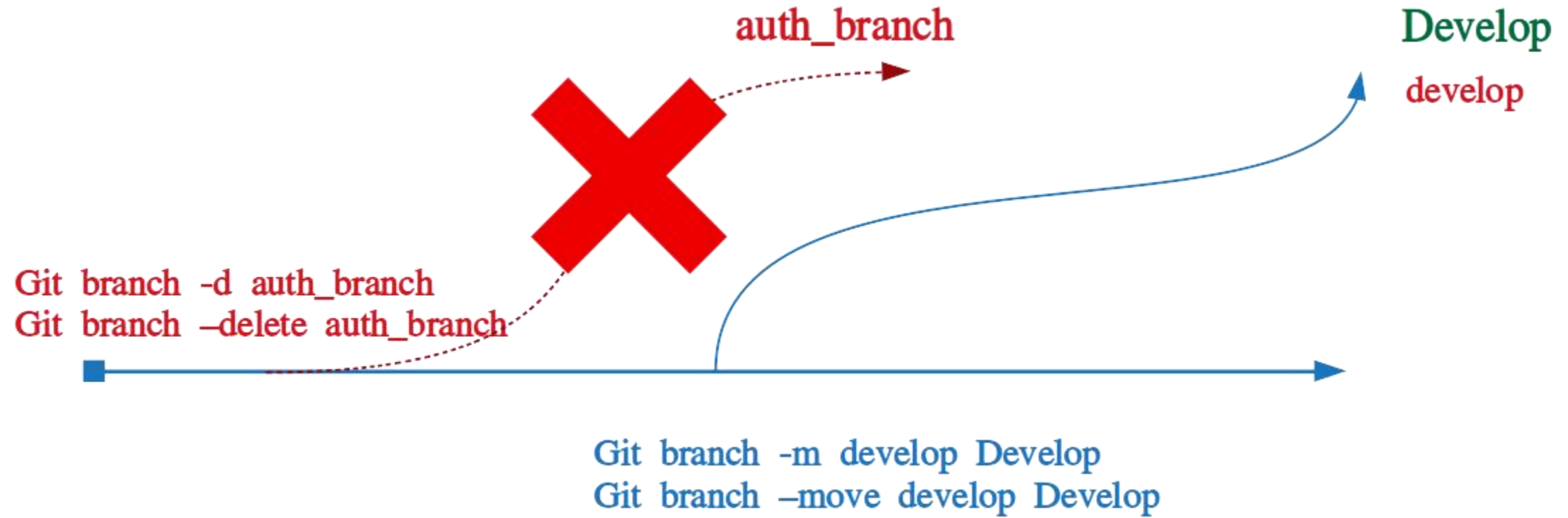
# Ignore SASS config files
.sass-cache
```



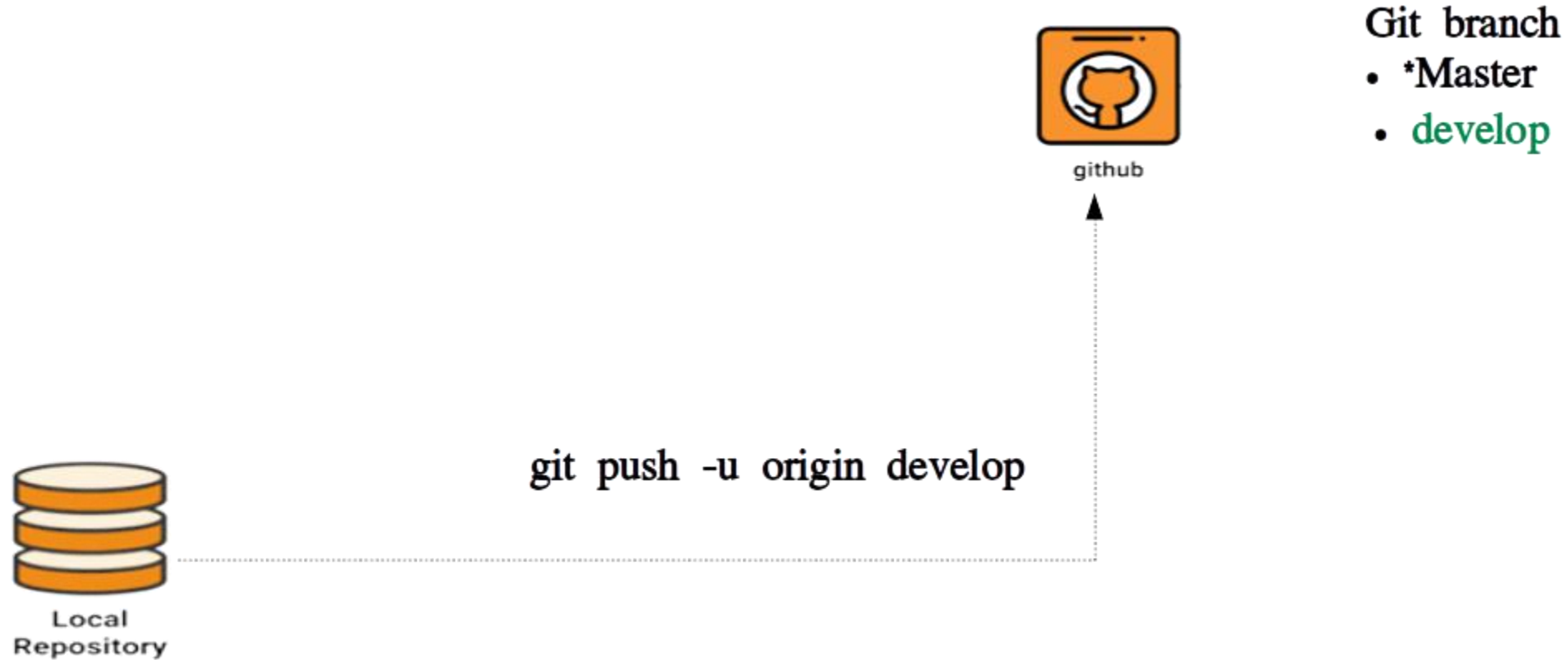
Branches are used to develop features isolated from each other. The master branch is the "default" branch when you create a repository. Use other branches for development and merge them back to the master branch upon completion.



Git Branching: Rename and Delete



Git Branching: Push branch to remote repository



Git work flow : Rebase vs Merge



- Rebasing and merging are both designed to integrate changes from one branch into another branch but in different ways.
- For ex. let's say we have commits like below, the merge will result as a combination of commits, whereas rebase will add all the changes in feature branch starting from the last commit of the master branch:
- When you do rebase a feature branch onto master, you move the base of the feature branch to master branch's ending point.
- Merging takes the contents of the feature branch and integrates it with the master branch. As a result, only the master branch is changed. The feature branch history remains same.
- Merging adds a new commit to your history.

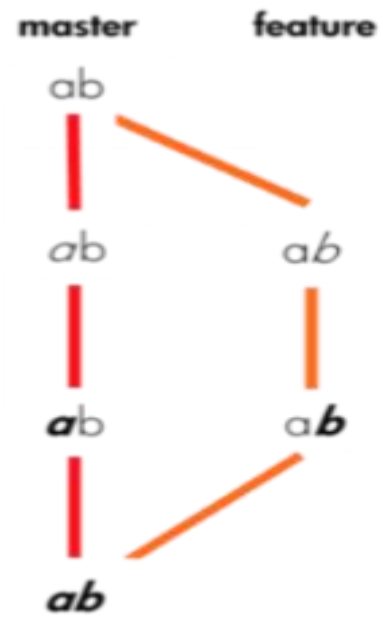
Git work flow : Rebase vs Merge



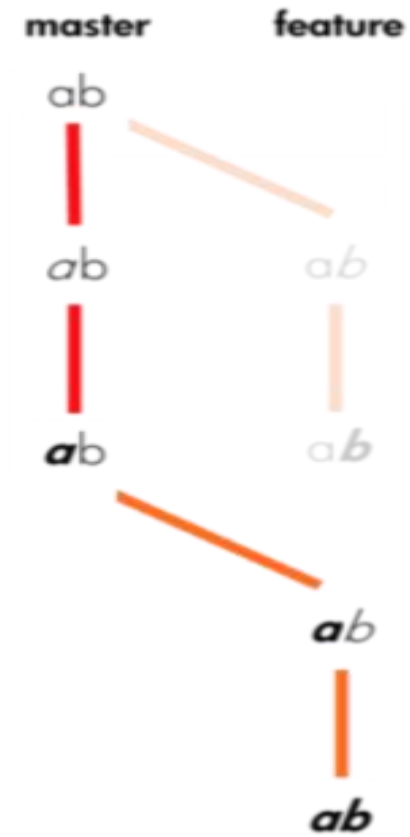
Commits



Merge



Rebase



Git flow manifest



- **Master** is for releases only
- **Develop** Not ready for public consumption but compiles and passes all tests
- **Feature branches :**
 - Where most development happens
 - Branch off of develop
 - Merge into develop
- **Release branches**
 - Branch off of develop
 - Merge into master and develop
- **Hotfix**
 - Branch off of master
 - Merge into master and develop
- **Bugfix**
 - Branch off of develop
 - Merge into develop

Git cycle of a feature branch



1) Enable git flow for the repo

- `git flow init -d`

2) Start the feature branch

- `git flow feature start newstuff`
- Creates a new branch called `feature/newstuff` that branches off of `develop`

3) Push it to GitHub for the first time

- Make changes and commit them locally
- `git flow feature publish newstuff`

4) Additional (normal) commits and pushes as needed

- `git commit -a`
- `git push`

5) Bring it up to date with develop (to minimize big changes on the ensuing pull request)

- `git checkout develop`
- `git pull origin develop`
- `git checkout feature/newstuff`
- `git merge develop`

6) Finish the feature branch (don't use `git flow feature finish`)

- Do a pull request on GitHub from `feature/newstuff` to `develop`
- When successfully merged the remote branch will be deleted
- `git remote update -p`
- `git branch -d feature/newstuff`
- Source: <https://danielkummer.github.io/git-flow-cheatsheet/>



Linux
Professional
Institute

Module 4

System image creation and VM

Deployment

Plan

- **Vagrant**
- **Vagrantfile**
- **Vagrantbox**
- **Packer**



- Create and configure lightweight, reproducible, and portable development environments.
- A higher-level wrapper around virtualization
- software such as VirtualBox, VMware, KVM.
- Wrapper around configuration management software such as Ansible, Chef, Salt, and Puppet.
- Public clouds e.g. AWS, DigitalOcean can be providers too.

- Same steps irrespective of OS and providers :
 - `$ mkdir centos`
 - `$ cd centos`
 - `$ vagrant init centos/7`
 - `$ vagrant up`
- OR :
 - `$ vagrant up --provider <PROVIDER>`
 - `$vagrant ssh`

- **Creating a VM :**
- *vagrant init* -- Initialize Vagrant with a Vagrantfile and `./vagrant` directory, using no specified base image.
Before you can do `vagrant up`, you'll need to specify a base image in the Vagrantfile.
 - *Vagrant init -m* : Create a minimal Vagrantfile (no comments or helpers):
 - *Vagrant init -f* : Create a new Vagrantfile, overwriting the one at the current path
 - *Vagrant init -box-version* : Create a Vagrantfile, locking the box to a version constraint:
- *vagrant init <boxpath>* -- Initialize Vagrant with a specific box. To find a box, go to the public Vagrant box catalog. When you find one you like, just replace it's name with boxpath. For example,
 - *vagrant init ubuntu/trusty64.*

- Starting a VM :
 - *vagrant up* -- starts vagrant environment (also provisions only on the FIRST vagrant up)
 - *vagrant resume* -- resume a suspended machine (vagrant up works just fine for this as well)
 - *vagrant provision* -- forces re-provisioning of the vagrant machine
 - *vagrant reload* -- restarts vagrant machine, loads new Vagrantfile configuration
 - *vagrant reload --provision* -- restart the virtual machine and force provisioning
- Getting into a VM:
 - *vagrant ssh* -- connects to machine via SSH
 - *vagrant ssh <boxname>* -- If you give your box a name in your Vagrantfile, you can ssh into it with boxname. Works from any directory.
- Stopping a VM:
 - *vagrant halt* -- stops the vagrant machine
 - *vagrant suspend* -- suspends a virtual machine (remembers state)

- Saving Progress:

- *vagrant snapshot save [options] [vm-name] <name>* -- vm-name is often default. Allows us to save so that we can rollback at a later time .

- Tips:

- *vagrant -v* -- get the vagrant version
- *vagrant status* -- outputs status of the vagrant machine
- *vagrant global-status* -- outputs status of all vagrant machines
- *vagrant global-status --prune* -- same as above, but prunes invalid entries
- *vagrant provision --debug* -- use the debug flag to increase the verbosity of the output
- *vagrant push -- yes*, vagrant can be configured to deploy code!
- *vagrant up --provision | tee provision.log* -- Runs vagrant up, forces provisioning and logs all output to a file

- Alright, so we have a virtual machine running a basic copy of Ubuntu and we can edit files from our machine and have them synced into the virtual machine. Let us now serve those files using a webserver.
- We could just SSH in and install a webserver and be on our way, but then every person who used Vagrant would have to do the same thing. Instead, Vagrant has built-in support for automated provisioning. Using this feature, Vagrant will automatically install software when you vagrant up so that the guest machine can be repeatably created and ready-to-use.

- Example 1 provisioning with Shell :

<https://www.vagrantup.com/intro/getting-started/provisioning.html>

- Example 2 provisioning with Ansible:

https://docs.ansible.com/ansible/latest/scenario_guides/guide_vagrant.html

- A *Vagrantbox* is a tarred, gzip file containing the following:
 - *Vagrantfile* : The information from this will be merged into your Vagrantfile that is created when you run `vagrant init boxname` in a folder.
 - *box-disk.vmdk* (For Virtualbox) : the virtual machine image.
 - *box.ovf* : defines the virtual hardware for the box.
 - *metadata.json* :tells vagrant what provider the box works with.

- Boxes commands :
 - *vagrant box list* -- see a list of all installed boxes on your computer
 - *vagrant box add <name> <url>* -- download a box image to your computer
 - *vagrant box outdated* -- check for updates vagrant box update
 - *vagrant boxes remove <name>* -- deletes a box from the machine
 - *vagrant package* -- packages a running virtualbox env in a reusable box

Packer ,what' s Packer ?



- Open source tool for creating identical machine images :
 - for multiple platforms
 - from a single source configuration.
- Advantages of using Packer :
 - Fast infrastructure deployment
 - Multi-provider portability
 - Stability
 - Identity

- **Continuous Delivery:**

Generate new machine images for multiple platforms on every change to Ansible, Puppet or Chef repositories

- **Environment Parity:**

Keep all dev/test/prod environments as similar as possible.

- **Auto-Scaling acceleration:**

Launch completely provisioned and configured instances in seconds, rather than minutes or even hours.

Packer Terminology:

- The *JSON configuration files* used to define/describe images.
- Templates are divided into core sections:
 - *variables (optional)* : Variables allow you to set API keys and other variable settings without changing the configuration file
 - *builders (required)* : Platform specific building configuration
 - *provisioners (optional)* : Tools that install software after the initial OSinstall
 - *post-processors (optional)* :Actions to happen after the image has beenbuilt

- This varies depending on which builder you use. The following is an example for the QEMU builder:
 1. Download ISO image
 2. Create virtual machine
 3. Boot virtual machine from the CD
 4. Using VNC, type in commands in the installer to start an automated install via kickstart/preseed/etc
 5. Packer automatically serves kick-start/preseed file with built-in HTTP server
 6. Packer waits for ssh to become available
 7. OS installer runs and then reboots
 8. Packer connects via ssh to VM and runs provisioner (if set)
 9. Packer Shuts down VM and then runs the post processor (if set)
 10. PROFIT!



Commands

- Usage: packer [--version] [--help] <command> [<args>]
- Available commands are:
 - *build*: build image(s) from template
 - *console*: creates a console for testing variable interpolation
 - *fix* : fixes templates from old versions of packer
 - *inspect* : see components of a template
 - *validate*: check that a template is valid
 - *version*: Prints the Packer version



Linux
Professional
Institute

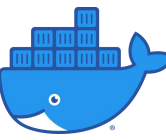
Module 5

Container usage

Plan

- **What is a Container and Why?**
- **Docker and containers**
- **Docker command line**
- **Connect container to Docker networks**
- **Manage container storage with volumes**
- **Create Dockerfiles and build images**





Virtualization

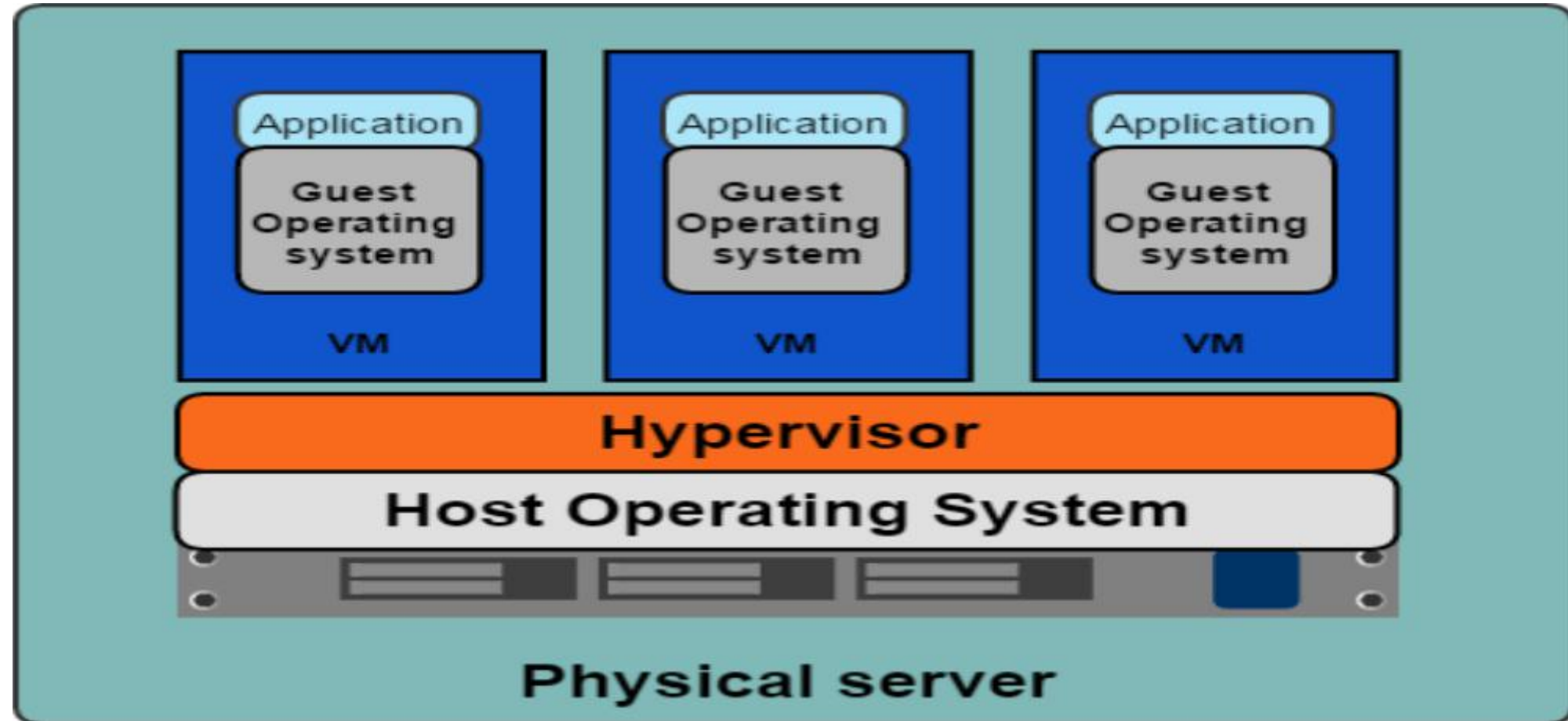
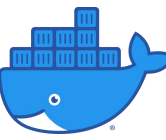


VS

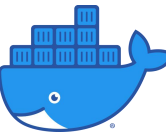
Containerization



What is a Container and Why?

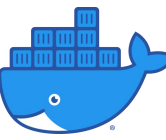


What is a Container and Why?



- **Advantages of Virtualization:**
 - Minimize hardware costs.
 - Multiple virtual servers on one physical hardware.
 - Easily move VMs to other data centers.
 - Conserve power
 - Free up unused physical resources.
 - Easier automation.
 - Simplified provisioning/administration of hardware and software.
 - Scalability and Flexibility: Multiple operating systems

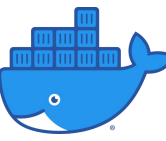
What is a Container and Why?



- Problems of Virtualization:
 - Each VM requires an operating system (OS)
 - Each OS requires a license(windows case).
 - Each OS has its own compute and storage overhead
 - Needs maintenance, updates

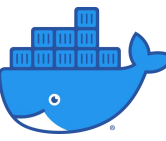


What is a Container and Why?



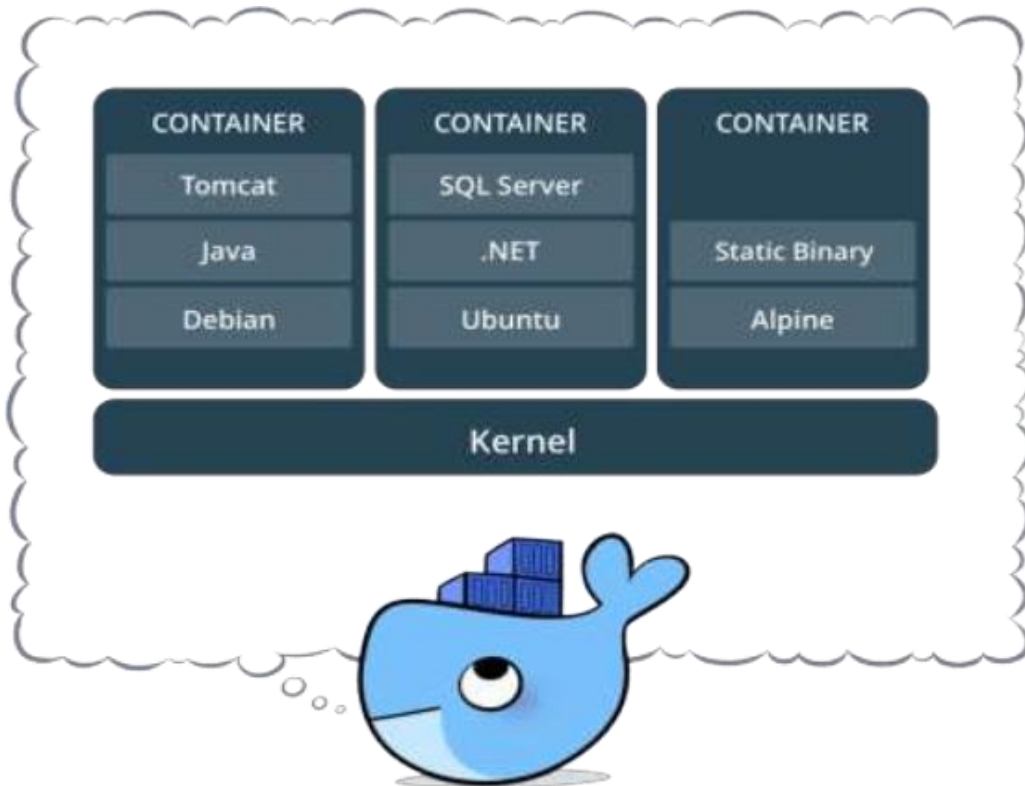
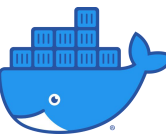
- Containers Solution :
 - Containers provide a standard way to **package** your **application's code**, **configurations**, and **dependencies** into a **single object**.
 - Containers share an operating system installed on the server and run as resource-isolated processes, ensuring quick, reliable, and consistent deployments, regardless of environment.
- Linux Containers :
 - **cgroup**: Control Groups provide a mechanism for aggregating/partitioning sets of tasks, and all their future children, into hierarchical groups with specialized behaviour.
 - **namespace**: wraps a global system resource in an abstraction that makes it appear to the processes within the namespace that they have their own isolated instance of the global resource.
- In short:
 - **Cgroups** = limits how much you can use;
 - **namespaces** = limits what you can see (and therefore use)

What is a Container and Why?

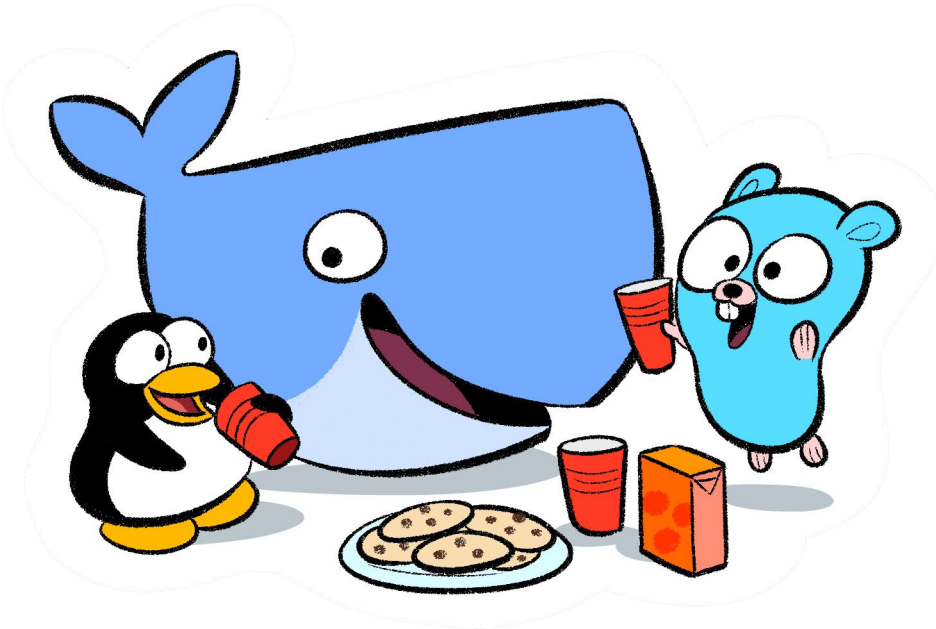
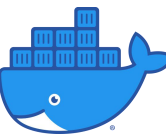


- Cgroups involve resource metering and limiting:
 - Memory
 - CPU
 - Block I/O
 - Network
- Namespaces provide processes with their own view of the system:
 - Multiple namespaces:
 - Mount - isolate filesystem mount points
 - UTS - isolate hostname and domainname
 - IPC - isolate interprocess communication (IPC) resources
 - PID - isolate the PID number space
 - Network - isolate network interfaces
 - User - isolate UID/GID number spaces
 - Cgroup - isolate cgroup root directory

What is a Container and Why?

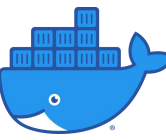


- Standardized packaging for software and dependencies
- Isolate apps from each other
- Share the same OS kernel
- Works with all major Linux and Windows Server



- **License:** Binaries: Freemium software as a service; Source code: Apache License 2.0
- **Original author(s):** Solomon Hykes
- **Platform:** x86-64, ARM, s390x, ppc64le
- **Developed by:** Docker, Inc.
- **Written in:** Go
- **Initial release date:** March 20, 2013

The Docker Family Tree



Open source framework for assembling core components that make a container platform

Intended for:

Open source contributors + ecosystem developers



Subscription-based, commercially supported products for delivering a secure software supply chain

Intended for:

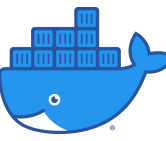
Production deployments + Enterprise customers



Free, community-supported product for delivering a container solution

Intended for: Software dev & test

Key Benefits of Docker Containers



- **Speed :**

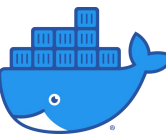
- No OS to boot = applications online in seconds

- **Portability :**

- Less dependencies between process layers = ability to move between infrastructure

- **Efficiency:**

- Less OS overhead
- Improved VM density



Image

The basis of a Docker container. The content at rest.



Container

The image when it is 'running.' The standard unit for app service



Engine

The software that executes commands for containers. Networking and volumes are part of Engine.

Can be clustered together.



Registry

Stores, distributes and manages Docker images



Control Plane

Management plane for container and cluster orchestration

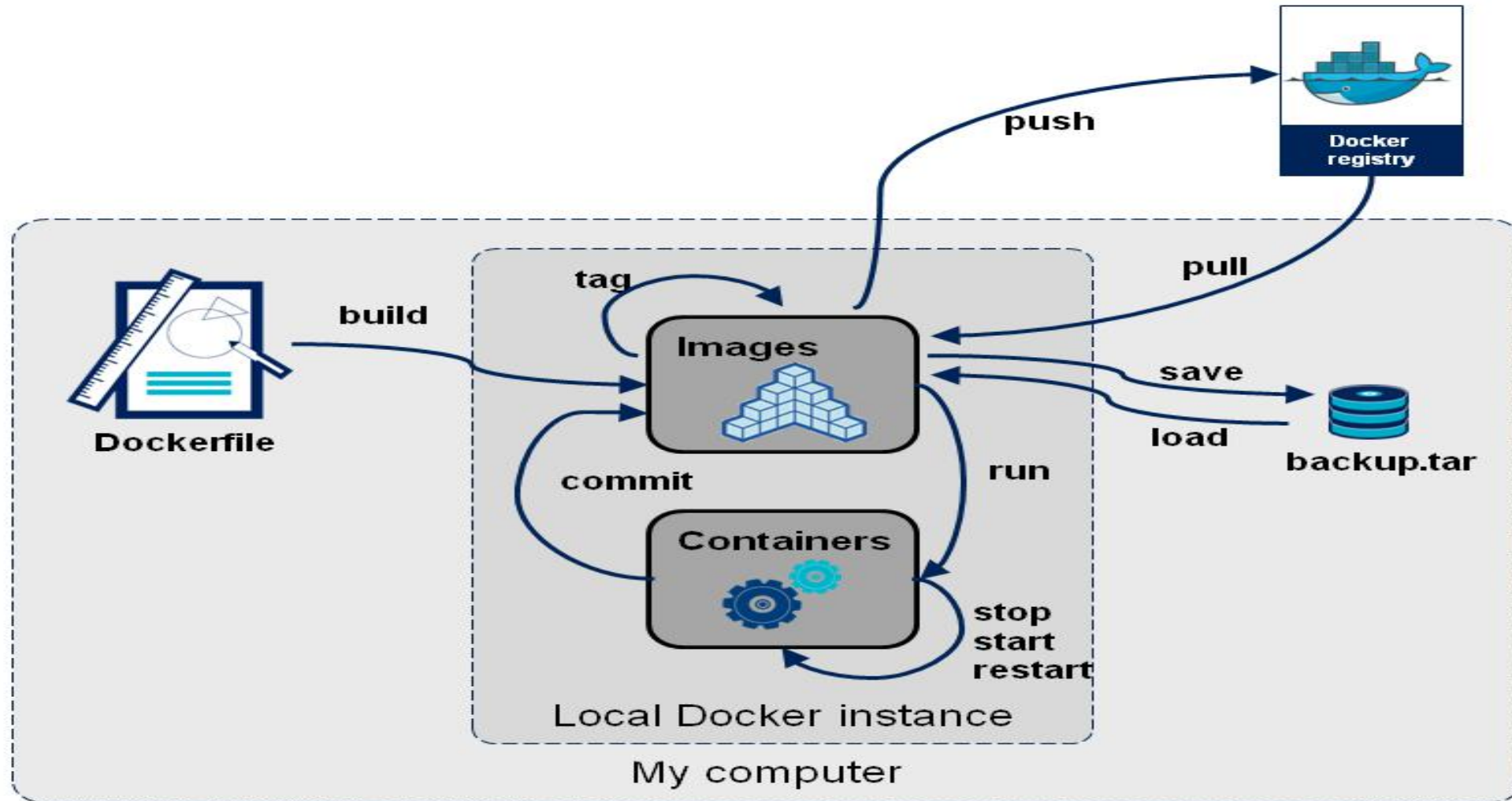
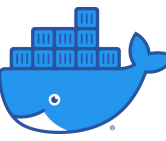


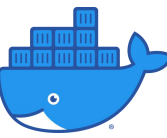
Dockerfile

Defines what goes on in the environment inside your container

DOCKERFILE

Docker workflow





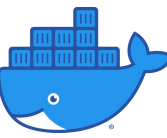
Commands

- **Containers**

Your basic isolated Docker process. Containers are to Virtual Machines as threads are to processes. Or you can think of them as chroots on steroids.

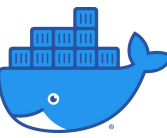
- **Lifecycle**

- *docker create* creates a container but does not start it.
- *docker rename* allows the container to be renamed.
- *docker run* creates and starts a container in one operation.
- *docker rm* deletes a container.
- *docker update* updates a container's resource limits.



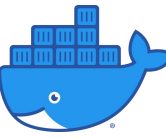
Commands

- **Starting and Stopping**
 - *docker start* starts a container so it is running.
 - *docker stop* stops a running container.
 - *docker restart* stops and starts a container.
 - *docker pause* pauses a running container, "freezing" it in place.
 - *docker unpause* will unpause a running container.
 - *docker wait* blocks until running container stops.
 - *docker kill* sends a SIGKILL to a running container.
 - *docker attach* will connect to a running container.

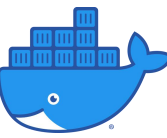


Commands

- **Images** : Images are just templates for docker containers.
- **Life cycle** :
 - *docker images* shows all images.
 - *docker import* creates an image from a tarball.
 - *docker build* creates image from Dockerfile.
 - *docker commit* creates image from a container, pausing it temporarily if it is running.
 - *docker rmi* removes an image.
 - *docker load* loads an image from a tar archive as STDIN, including images and tags (as of 0.7).
 - *docker save* saves an image to a tar archive stream to STDOUT with all parent layers, tags & versions (as of 0.7).
- **Info** :
 - *docker history* shows history of image.
 - *docker tag* tags an image to a name (local or registry).



- Network drivers : Docker's networking subsystem is pluggable, using drivers.
- Several drivers exist by default, and provide core networking functionality:
 - **bridge**: The default network driver
 - **host**: For standalone containers, remove network isolation between the container and the Docker host, and use the host's networking directly.
 - **overlay**: Connect multiple Docker daemons together and enable swarm services to communicate with each other.
 - **macvlan** : Allow to assign a MAC address to a container, making it appear as a physical device on network
 - **none**: Disable all networking. Usually used in conjunction with a custom network driver.
- provide better isolation and interoperability between containerized applications:
 - automatically expose all ports to each other
 - no ports exposed to the outside world
- provide automatic DNS resolution between containers.
- Containers can be attached and detached from user-defined networks on the fly.



Commands

- **Network:**

- *docker network create my-net* : create new network my-net
- *docker network ls* list all docker network
- *docker network rm my-net* : delete network
- *docker create --name my-nginx --network my-net --publish 8080:80 \nginx:latest* : create new container and connect to network my-net
- *docker network connect my-net my-nginx* : connect to network my-net
- *docker network disconnect my-net my-nginx* : disconnect from network my-net



Linux
Professional
Institute

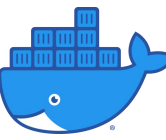
Module 6

Container Infrastructure

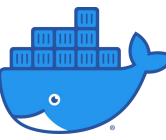
Plan

- **what's Docker Machine ?**
- **Docker Machine drivers**
- **Docker machine commands**
- **Flocker**

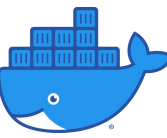




- Docker Machine create hosts with Docker Engine installed on them.
- Machine can create Docker hosts on :
 - local Mac
 - Windows box
 - Company network
 - Data center
 - Cloud providers like Azure, AWS, or Digital Ocean.
- docker-machine commands can:
 - Start, inspect, stop, and restart a managed host,
 - Upgrade the Docker client and daemon,
 - Configure a Docker client to talk to host
 - Create a machine. Requires the --driver flag to indicate which provider (VirtualBox, DigitalOcean, AWS, etc.)



- Here is an example of using the `--virtualbox` driver to create a machine called dev.
- `$ docker-machine create --driver virtualbox dev`
- Machine drivers:
 - Amazon Web Services
 - Microsoft Azure
 - Digital Ocean
 - Google Compute Engine
 - Microsoft Hyper-V
 - OpenStack
 - Rackspace
 - IBM Softlayer
 - Oracle VirtualBox
 - VMware vCloud Air
 - VMware Fusion



Commands

- **Docker machine :**

- *active*
- *config*
- *create*
- *env*
- *inspect*
- *ip*
- *kill*
- *ls*
- *provision*
- *regenerate-certs*
- *restart*
- *rm*
- *ssh*
- *scp*
- *mount*
- *start*
- *status*
- *stop*
- *upgrade*
- *url*
- *version*
- *help*

Print which machine is active

Print the connection config for machine

Create a machine

Display the commands to set up the env for the Docker client

Inspect information about a machine

Get the IP address of a machine

Kill a machine

List machines

Re-provision existing machines

Regenerate TLS Certificates for a machine

Restart a machine

Remove a machine

Log into or run a command on a machine with SSH.

Copy files between machines

Mount or unmount a directory from a machine with SSHFS.

Start a machine

Get the status of a machine

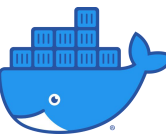
Stop a machine

Upgrade a machine to the latest version of Docker

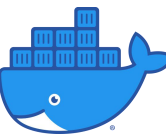
Get the URL of a machine

Show the Docker Machine version or a machine docker version

Shows a list of commands or help for one command



- Flocker is an open-source Container Data Volume Manager for your Dockerized applications.
- By providing tools for data migrations, Flocker gives ops teams the tools they need to run containerized stateful services like databases in production.
- Unlike a Docker data volume which is tied to a single server, a Flocker data volume, called a dataset, is portable and can be used with any container, no matter where that container is running.
- Flocker manages Docker containers and data volumes together. When you use Flocker to manage your stateful microservice, your volumes will follow your containers when they move between different hosts in your cluster.
- You can also use Flocker to manage only your volumes, while continuing to manage your containers however you choose.



- Flocker is an open-source Container Data Volume Manager for your Dockerized applications.
- By providing tools for data migrations, Flocker gives ops teams the tools they need to run containerized stateful services like databases in production.
- Unlike a Docker data volume which is tied to a single server, a Flocker data volume, called a dataset, is portable and can be used with any container, no matter where that container is running.
- Flocker manages Docker containers and data volumes together. When you use Flocker to manage your stateful microservice, your volumes will follow your containers when they move between different hosts in your cluster.
- You can also use Flocker to manage only your volumes, while continuing to manage your containers however you choose.



Linux
Professional
Institute

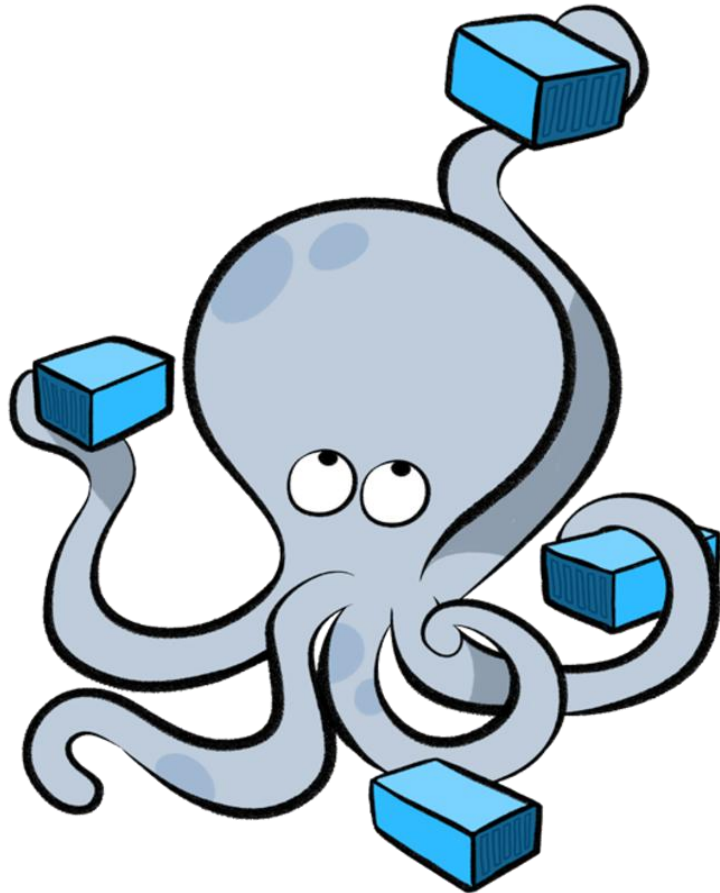
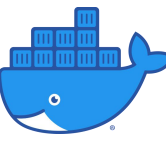
Module 6

Container Infrastructure

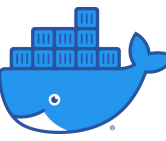
Plan

- **Docker-compose**
- **Docker Swarm**
- **Kubernetes**





- Compose is a tool for defining and running multi-container Docker applications.
- With Compose, you use a YAML file to configure your application's services.
- Then, with a single command, you create and start all the services from your configuration.
- Compose works in all environments:
 - Production :
 - Staging,
 - Development : Create and start one or more containers for each dependency (databases, queues, caches, web service APIs, etc) with a single command.
 - Testing,
 - As well as CI workflows.



Docker-compose use cases :

- Compose can be used in many different ways

1- Development environments :

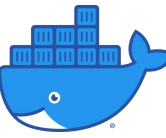
- Create and start one or more containers for each dependency (databases, queues, caches, web service APIs, etc) with a single command.

2- Automated testing environments :

- Create and destroy isolated testing environments in just a few commands.

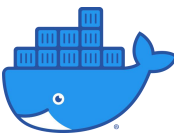
3- Cluster deployments :

- Compose can deploy to a remote single docker Engine.
- The Docker Engine may be a single instance provisioned with Docker Machine or an entire Docker Swarm cluster



Create service with docker-compose ?

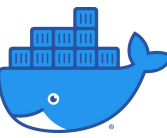
- Using Compose is basically a three-step process:
 - Define your app's environment with a **Dockerfile** so it can be reproduced anywhere.
 - Define the services that make up your app in **docker-compose.yml** so they can be run together in an isolated environment.
 - Lastly, run **docker-compose up** and Compose will start and run your entire app.



Commands

- **Docker composeS:**

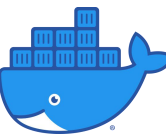
- *build* Build or rebuild services
- *bundle* Generate a Docker bundle from the Compose file
- *config* Validate and view the Compose file
- *create* Create services
- *down* Stop and remove containers, networks, images, and volumes
- *events* Receive real time events from containers
- *exec* Execute a command in a running container
- *help* Get help on a command
- *images* List images
- *kill* Kill containers
- *logs* View output from containers
- *pause* Pause services
- *port* Print the public port for a port binding
- *ps* List containers
- *pull* Pull service images
- *push* Push service images
- *restart* Restart services
- *rm* Remove stopped containers
- *run* Run a one-off command
- *scale* Set number of containers for a service
- *start* Start services
- *stop* Stop services
- *top* Display the running processes
- *unpause* Unpause services
- *up* Create and start containers
- *version* Show the Docker-Compose version information



Containers have become popular thanks to their focus on consistency across platforms from development to production. The rise in interest to containers has in turn brought in higher demands for their deployment and management.

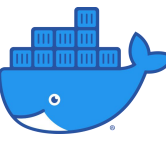
The need for better control attracted a number of software options as solutions *for container orchestration*, which allows for abstraction of individual containers to services with a number of instances or replicas.

Two of the major players developing container orchestration are *Kubernetes* and *Docker Swarm*. In this post, we will take a look at Kubernetes vs Docker comparison.

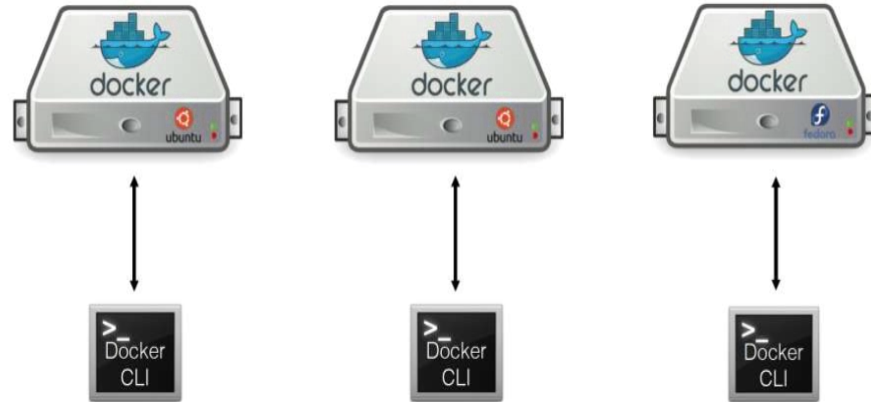


What is Docker Swarm ?

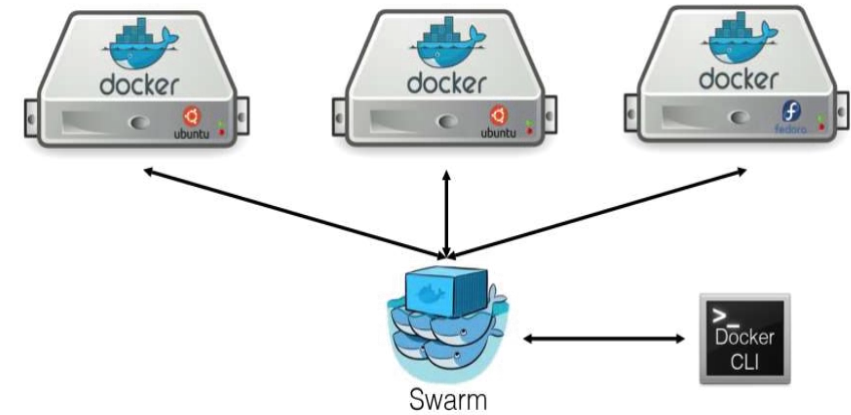
- Clustering and scheduling tool for docker container, feature embedded in Docker engine.
- Containers added or removed as a demands changes .
- Swarm turns multiple Docker hosts into single virtual docker host .

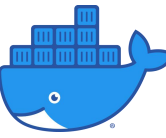


Traditional Model



Swarm





features highlights

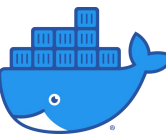
1- Cluster management integrated with Docker Engine: Use the Docker Engine CLI to create a swarm of Docker Engines where you can deploy application services. You don't need additional orchestration software to create or manage a swarm.

2- Decentralized design: Instead of handling differentiation between node roles at deployment time, the Docker Engine handles any specialization at runtime. You can deploy both kinds of nodes, managers and workers, using the Docker Engine. This means you can build an entire swarm from a single disk image.

3- Declarative service model: Docker Engine uses a declarative approach to let you define the desired state of the various services in your application stack. For example, you might describe an application comprised of a web front end service with message queueing services and a database backend.

4- Scaling: For each service, you can declare the number of tasks you want to run. When you scale up or down, the swarm manager automatically adapts by adding or removing tasks to maintain the desired state.

5- Desired state reconciliation: The swarm manager node constantly monitors the cluster state and reconciles any differences between the actual state and your expressed desired state. For example, if you set up a service to run 10 replicas of a container, and a worker machine hosting two of those replicas crashes, the manager creates two new replicas to replace the replicas that crashed. The swarm manager assigns the new replicas to workers that are running and available.



features highlights

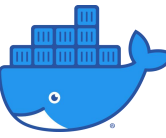
6- Multi-host networking: You can specify an overlay network for your services. The swarm manager automatically assigns addresses to the containers on the overlay network when it initializes or updates the application.

7- Service discovery: Swarm manager nodes assign each service in the swarm a unique DNS name and load balances running containers. You can query every container running in the swarm through a DNS server embedded in the swarm.

8- Load balancing: You can expose the ports for services to an external load balancer. Internally, the swarm lets you specify how to distribute service containers between nodes.

9- Secure by default: Each node in the swarm enforces TLS mutual authentication and encryption to secure communications between itself and all other nodes. You have the option to use self-signed root certificates or certificates from a custom root CA.

10- Rolling updates: At roll out time you can apply service updates to nodes incrementally. The swarm manager lets you control the delay between service deployment to different sets of nodes. If anything goes wrong, you can roll-back a task to a previous version of the service.



Node :

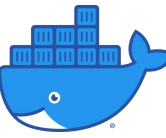
- A node is an instance of the Docker engine participating in the swarm. You can also think of this as a Docker node. You can run one or more nodes on a single physical computer or cloud server, but production swarm deployments typically include Docker nodes distributed across multiple physical and cloud machines.
- To deploy your application to a swarm, you submit a *service definition* to a *manager node*. The manager node *dispatches* units of work called *tasks* to *worker nodes*.

Manager nodes also perform the orchestration and cluster management functions required to maintain the *desired state* of the swarm. Manager nodes elect a single leader to conduct orchestration tasks.

Manager nodes handle cluster management tasks:

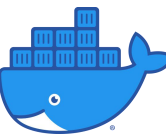
- maintaining cluster state
- scheduling services
- serving swarm mode HTTP API endpoints

Worker nodes *receive and execute tasks dispatched from manager nodes*. By default manager nodes also run services as worker nodes, but you can configure them to run manager tasks exclusively and be manager-only nodes. An agent runs on each worker node and reports on the tasks assigned to it. The worker node notifies the manager node of the current state of its assigned tasks so that the manager can maintain the desired state of each worker.



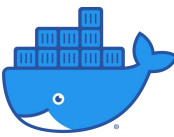
Services and Tasks:

- A service is the definition of the tasks to execute on the manager or worker nodes. It is the central structure of the swarm system and the primary root of user interaction with the swarm.
- When you create a service, you specify which *container image* to use and which commands to execute inside running containers.
- In the replicated services model, the swarm manager distributes a specific *number of replica* tasks among the nodes based upon the scale you set in the *desired state*.
- For global services, the swarm runs one task for the service on *every available node in the cluster*.
- A task carries a Docker container and the commands to run inside the container. It is the atomic scheduling unit of swarm. Manager nodes assign tasks to worker nodes according to the number of replicas set in the service scale. Once a task is assigned to a node, it cannot move to another node. It can only run on the assigned node or fail.



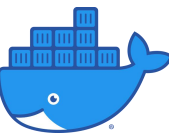
Load Balancing:

- The swarm manager uses ingress load balancing to expose the services you want to make available externally to the swarm. The swarm manager can automatically assign the service a PublishedPort or you can configure a PublishedPort for the service. You can specify any unused port. If you do not specify a port, the swarm manager assigns the service a port in the 30000-32767 range.
- External components, such as cloud load balancers, can access the service on the PublishedPort of any node in the cluster whether or not the node is currently running the task for the service. All nodes in the swarm route ingress connections to a running task instance.
- Swarm mode has an internal DNS component that automatically assigns each service in the swarm a DNS entry. The swarm manager uses internal load balancing to distribute requests among services within the cluster based upon the DNS name of the service.



Commands

- **Docker Swarm:**
 - *ca* Display and rotate the root CA
 - *init* Initialize a swarm
 - *join* Join a swarm as a node and/or manager
 - *join-token* Manage join tokens
 - *leave* Leave the swarm
 - *unlock* Unlock swarm
 - *unlock-key* Manage the unlock key
 - *update* Update the swarm



Uses Case

- **Initialize A Swarm:**

1. Make sure the Docker Engine daemon is started on the host machines.
2. On the manager node : `docker swarm init --advertise-addr <MANAGER-IP>`
3. On each worker node : `docker swarm join --token \ <token_generated_by_manager> <MANAGER-IP>`
4. On manager node, view information about nodes: `docker node ls`



kubernetes

What's kubernetes ?

- A highly collaborative open source project originally conceived by Google

Sometimes called:

- *Kube*
- *K8s (that's 'k' + 8 letters + 's')*

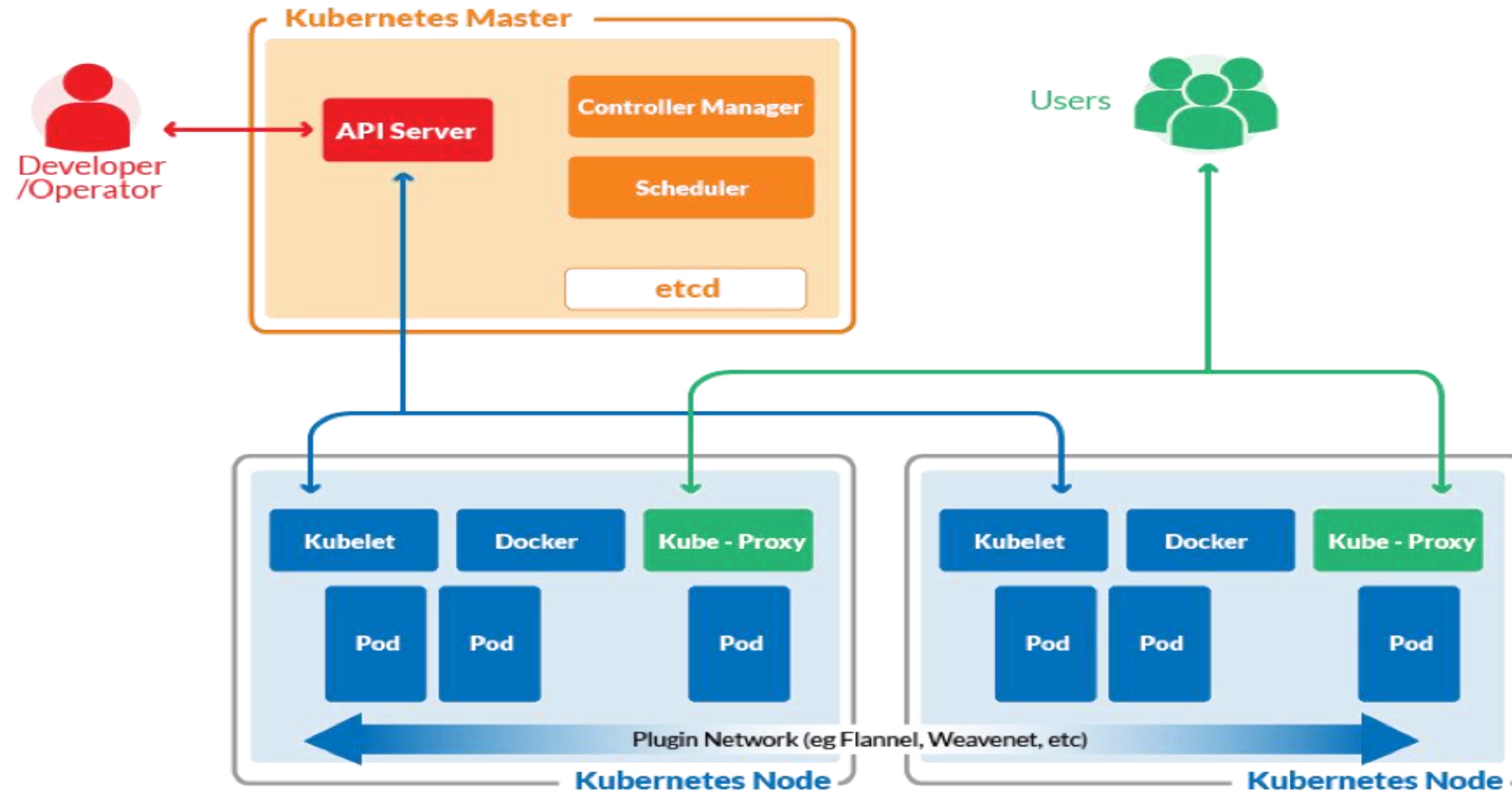
1. Start, stop, update, and manage a cluster of machines running containers in a consistent and maintainable way.
2. Particularly suited for horizontally scalable, stateless, or 'micro-services' application architectures
3. K8s > (docker swarm + docker-compose)
4. Kubernetes does NOT and will not expose all of the 'features' of the docker command line.
5. Run kubernetes locally using:
 - *Minikube* : a tool that makes it easy to run Kubernetes locally.
 - *Microk8s* : ligh tools to run kubernetes locally

Kubernetes Primitives and key words:

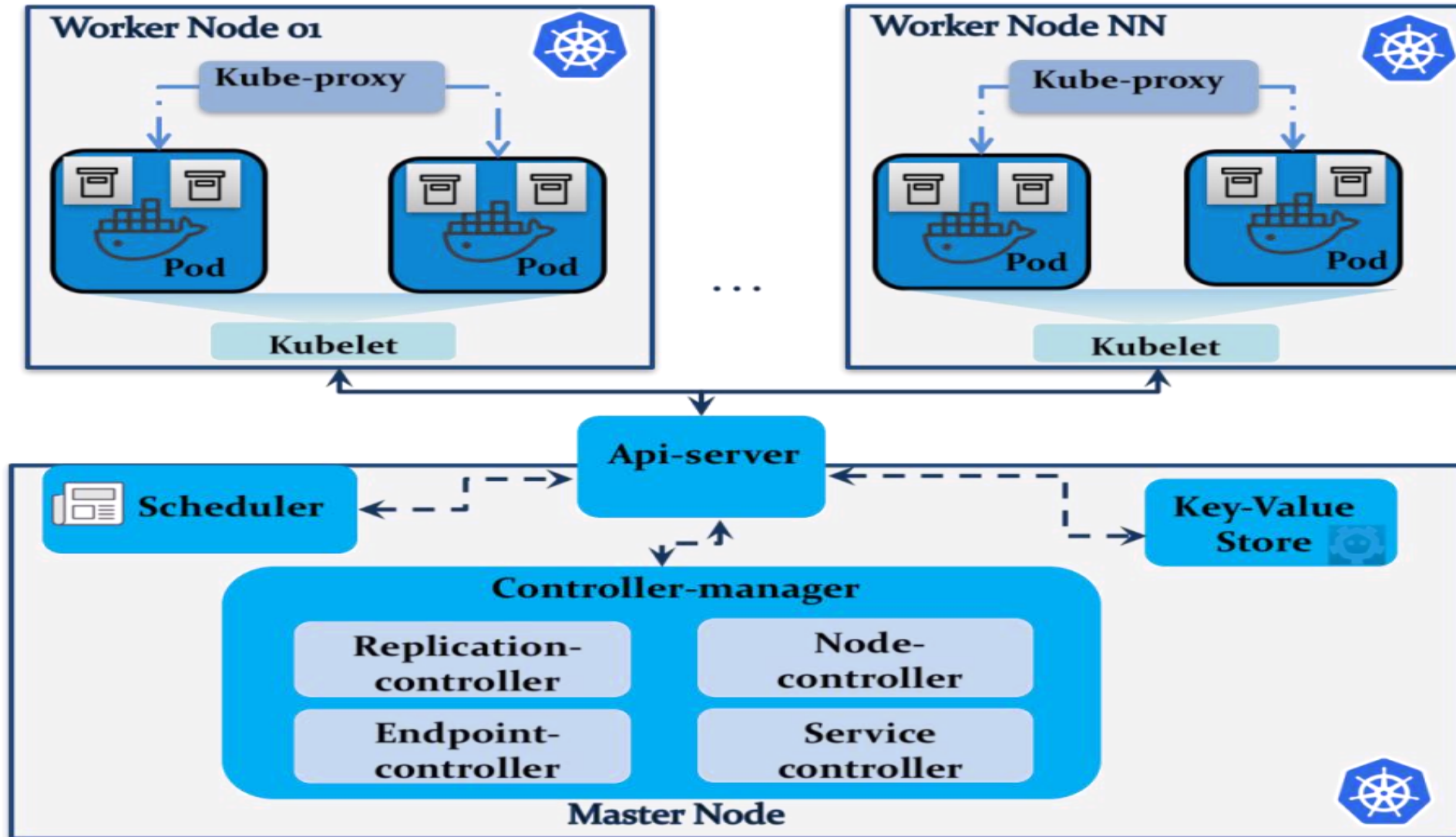
- Master
- Minion/Node
- Pod
- Replication Controller (old version) , ReplicaSet
- Deployment
- Service
- Label
- Namespace



Kubernetes Overview



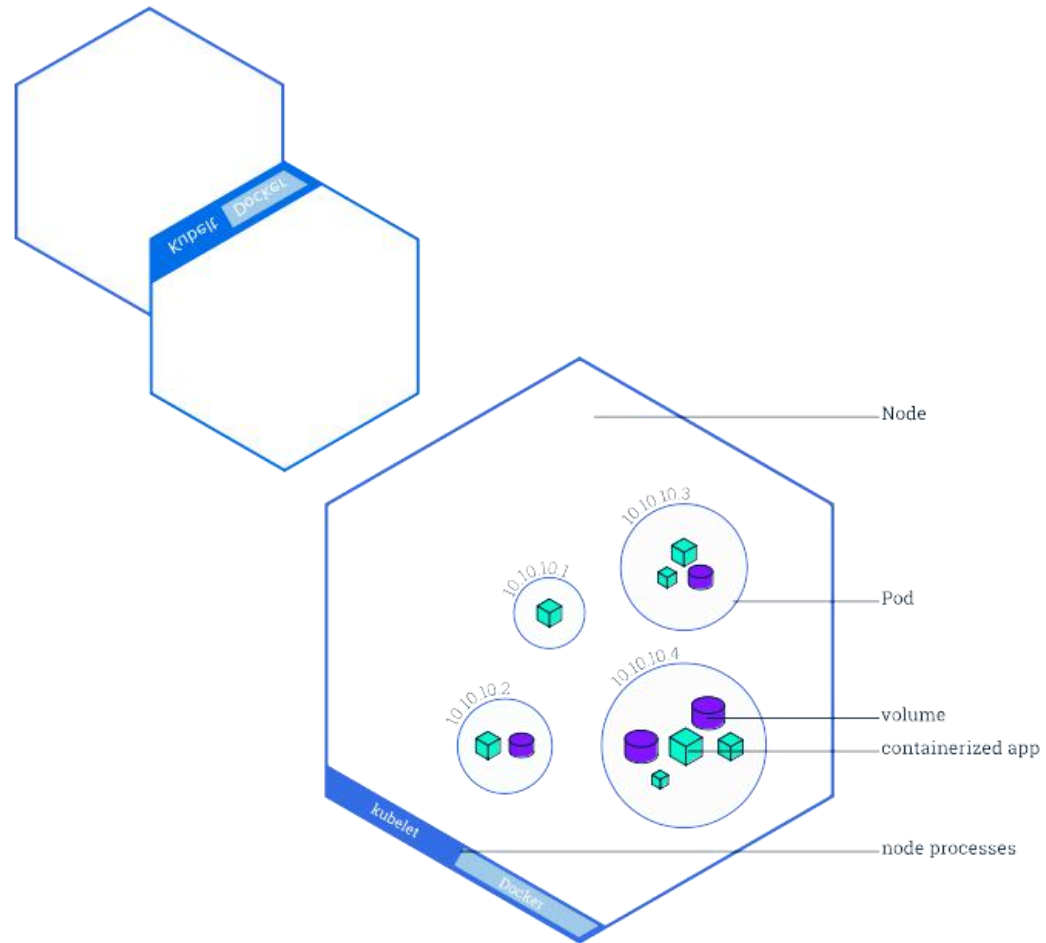
Kubernetes Primitives and key words:



- Typically consists of:
 - kube-apiserver
 - kube-scheduler
 - kube-controller-manager
 - Etcd
- Might contain:
 - kube-proxy
 - a network management utility

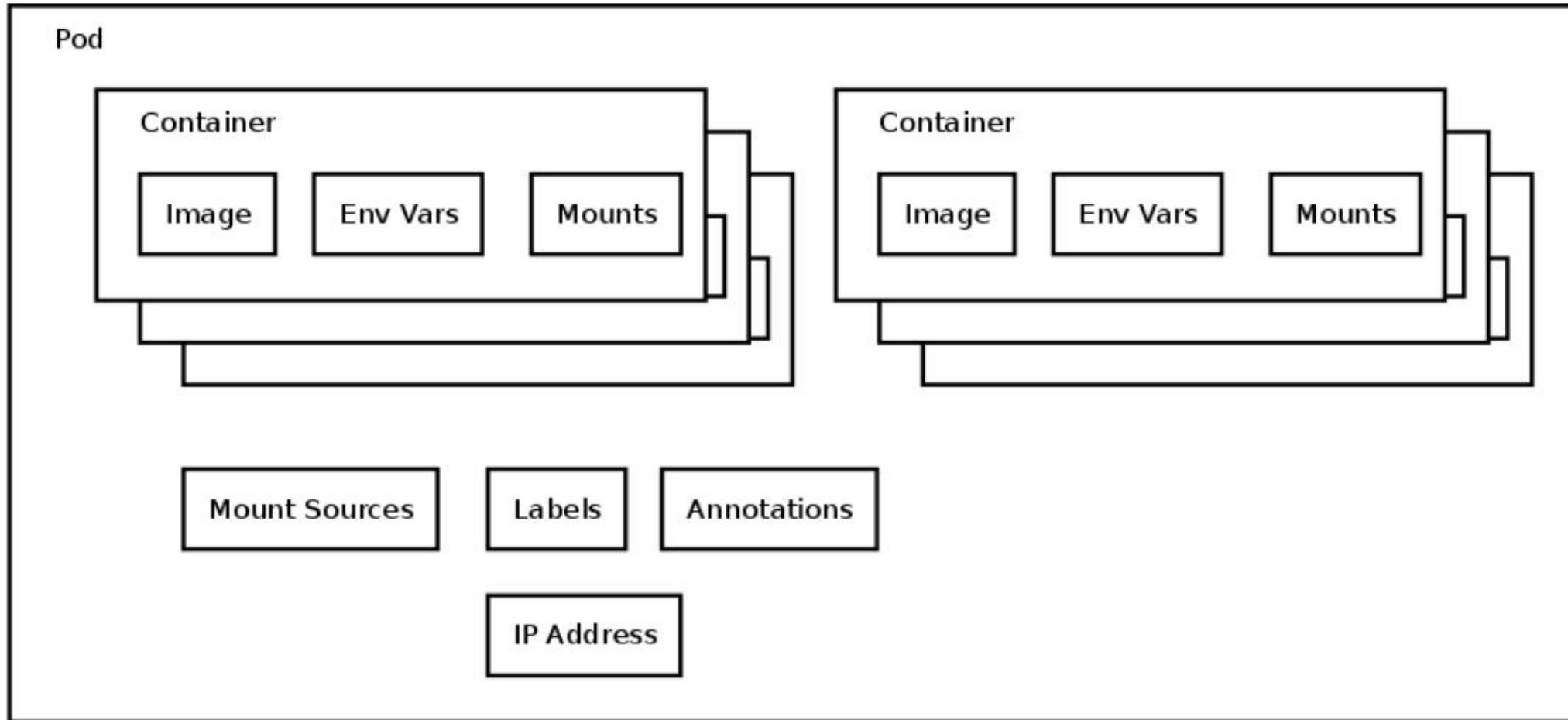
Typically consists of:

- **Kube-apiserver** : Component on the master that exposes the Kubernetes API. It is the front-end for the Kubernetes control plane. It is designed to scale horizontally – that is, it scales by deploying more instances.
- **Etcd** : Consistent and highly-available key value store used as Kubernetes' backing store for all cluster data.
- **Scheduler** : Component on the master that watches newly created pods that have no node assigned, and selects a node for them to run on.
- **Controller-manager**: Component on the master that runs controllers . Logically, each controller is a separate process, but to reduce complexity, they are all compiled into a single binary and run in a single process:
 - **Node Controller** : For checking the cloud provider to determine if a node has been deleted in the cloud after it stops responding
 - **Route Controller** : For setting up routes in the underlying cloud infrastructure
 - **Service Controller** : For creating, updating and deleting cloud provider load balancers
 - **Volume Controller** : For creating, attaching, and mounting volumes, and interacting with the cloud provider to orchestrate volume



- Typically consists of:
 - kubelet
 - kube-proxy
 - cAdvisor
- Might contain:
 - a network management utility

- A Pod is the basic execution unit of a Kubernetes application—the smallest and simplest unit in the Kubernetes object model that you create or deploy. A Pod represents processes running on your Cluster .
- Single schedulable unit of work:
 - Can not move between machines.
 - Can not span machines.
 - One or more containers
 - Shared network name-space
- Metadata about the container(s)
- Env vars – configuration for the container
- Every pod gets an unique IP:
 - Assigned by the container engine, not kube



apiVersion: v1

kind: Pod

metadata:

name: myapp-pod

labels:

app: myapp

spec:

containers:

- name: myapp-container

image: busybox

command: ['sh', '-c', 'echo Hello Kubernetes! && sleep 3600']

A ReplicationController ensures that a *specified number of pod replicas are running at any one time*. In other words, a ReplicationController makes sure that a pod or a homogeneous set of pods is always up and available.

Consists of :

- Pod template
 - Count
 - Label Selector
- Kube will try to keep *\$count copies* of pods *matching* the *label* selector *running*.
 - If too few copies are running the replication controller will start a new pod somewhere in the cluster

A ReplicaSet's purpose is to maintain a stable set of replica Pods running at any given time. As such, it is often used to guarantee the availability of *a specified number of identical Pods*.

Replica Sets are declared in essentially the same way as Replication Controllers, except that they have more options for the selector.

Replication controller

Pod template



Labek selector



count

Kubernetes Replication controller ,replicaset

```
{..} replicasSet.yml x {..} replication.yml x
src > controller > {..} replicasSet.yml > ... src > controller > {..} replication.yml > ...
1 controllers/frontend.yaml 1 controllers/replication.yaml
2 --- 2 ---
3 apiVersion: apps/v1 3 apiVersion: v1
4 kind: ReplicaSet 4 kind: ReplicationController
5 metadata: 5 metadata:
6   name: frontend 6   name: nginx
7   labels: 7 spec:
8     app: guestbook 8   replicas: 3
9     tier: frontend 9   selector:
10 spec: 10   app: nginx
11 # modify replicas according to your case 11   template:
12 replicas: 3 12     metadata:
13 selector: 13       name: nginx
14   matchLabels: 14       labels:
15     tier: frontend 15         app: nginx
16 template: 16     spec:
17   metadata: 17       containers:
18     labels: 18         - name: nginx
19       tier: frontend 19           image: nginx
20 spec: 20           ports:
21   containers: 21             - containerPort: 80
22     - name: php-redis 22
23     image: gcr.io/google_samples/gb-frontend:v3
24
```

A **Deployment** provides declarative updates for Pods and ReplicaSets.

You describe a *desired state* in a Deployment, and the Deployment Controller changes the actual state to the desired state at a controlled rate. You can define Deployments to *create new ReplicaSets*, or to remove existing Deployments and adopt all their resources with new Deployments.

Kubernetes Deployment

apiVersion: apps/v1
kind: Deployment
metadata:

name: nginx-deployment

spec:

replicas: 3

selector:

matchLabels:

app: nginx

template:

metadata:

labels:

app: nginx

spec:

containers:

- name: nginx

image: nginx:1.12.2

ports:

- containerPort: 80

Make a 3
replicas of
service
nginx using
matchlabels

An abstract way to expose an application running on a set of Pods as a network service.

With Kubernetes you don't need to modify your application to use an unfamiliar service discovery mechanism. Kubernetes gives Pods their own IP addresses and a single DNS name for a set of Pods, and can load-balance across them.

```
kind: Service  
apiVersion: v1
```

```
metadata:  
  name: hostname-service
```

```
spec:  
  type: NodePort  
  selector:  
    app: echo-hostname
```

```
ports:  
- nodePort: 30163  
  port: 8080  
  targetPort: 80
```

Make the service available
to network requests from
external clients

Forward requests to pods
with label of this value

nodePort
access service via this external port number

port
port number exposed internally in cluster

targetPort
port that containers are listening on

	Swarm	Kubernetes
Controller	Manager	Master
Slave	Worker	Nodes
Workload Definition	Service	Deployment
Deployment Unit	Task	Pod
Scale-out Definition	Replicas	Replica Set
Service Discovery	DNS	DNS
Load Balancing	Ingress	Service
Port	PublishedPort	Endpoint
Storage	Volumes	Persistent Volumes / Claims
Network	Overlay	Flat Networking Space



Linux
Professional
Institute

Module 6

Container Infrastructure

Plan

- **Continuous Integration (CI)**
- **What is it?**
- **What are the benefits?**
- **Continuous Build Systems**
- **Jenkins**
- **What is it?**
- **Where does it fit in?**
- **Why should I use it?**
- **What can it do?**
- **How does it work?**
- **Where is it used?**
- **How can I get started?**
- **Putting it all together**
- **Conclusion**
- **References**



“Continuous Integration is a software development practice where members of a team integrate their work frequently, usually each person integrates at least daily - leading to multiple integration per day. Each integration is verified by an automated build (including test) to detect integration errors as quickly as possible”

Martin Fowler

CI- What does it really mean ?



At a regular frequency (ideally at every commit), the system is:

- *Integrated*

All changes up until that point are combined into the project

- *Built*

The code is compiled into an executable or package

- *Tested*

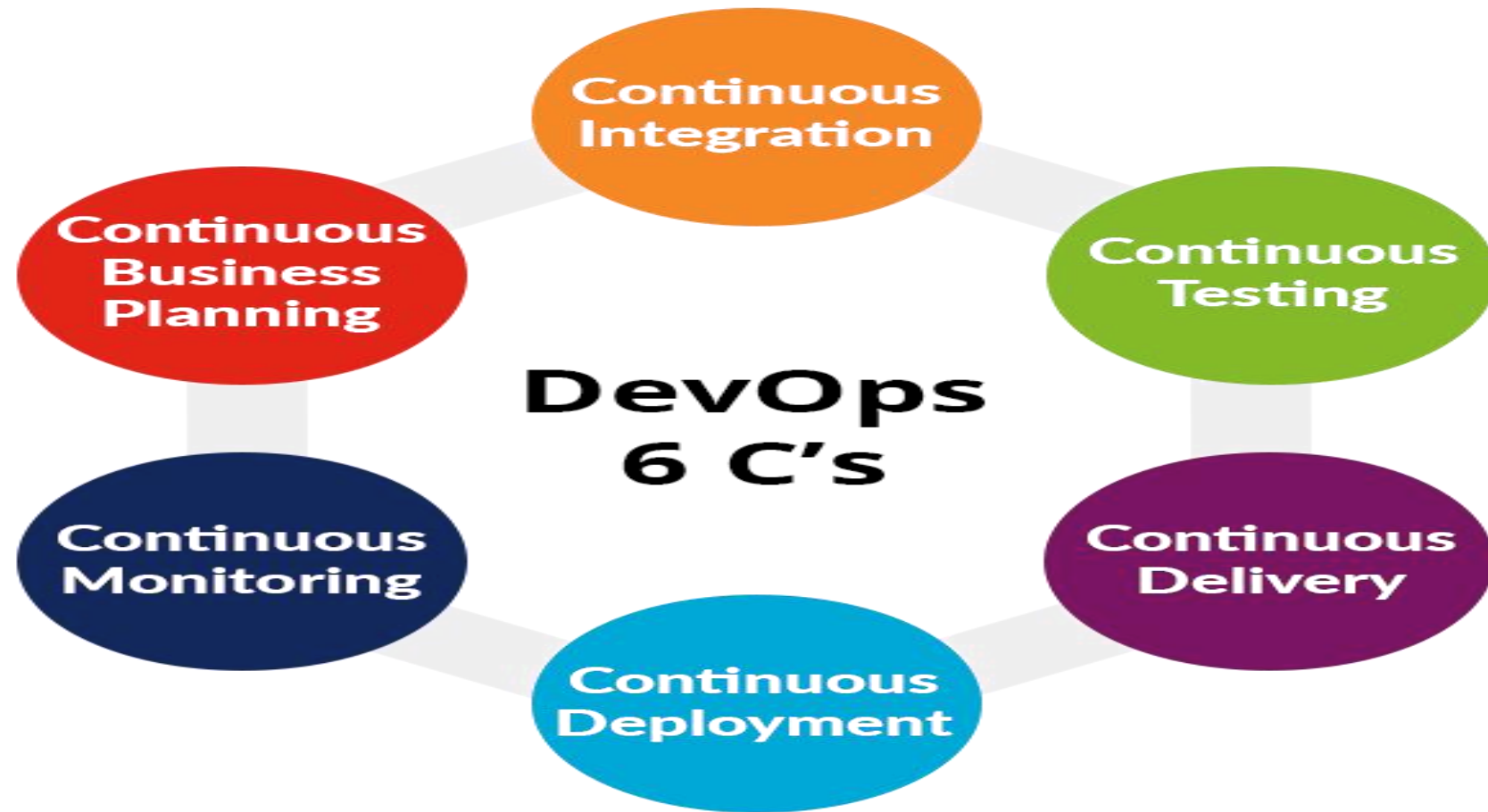
Automated test suites are run

- *Archived*

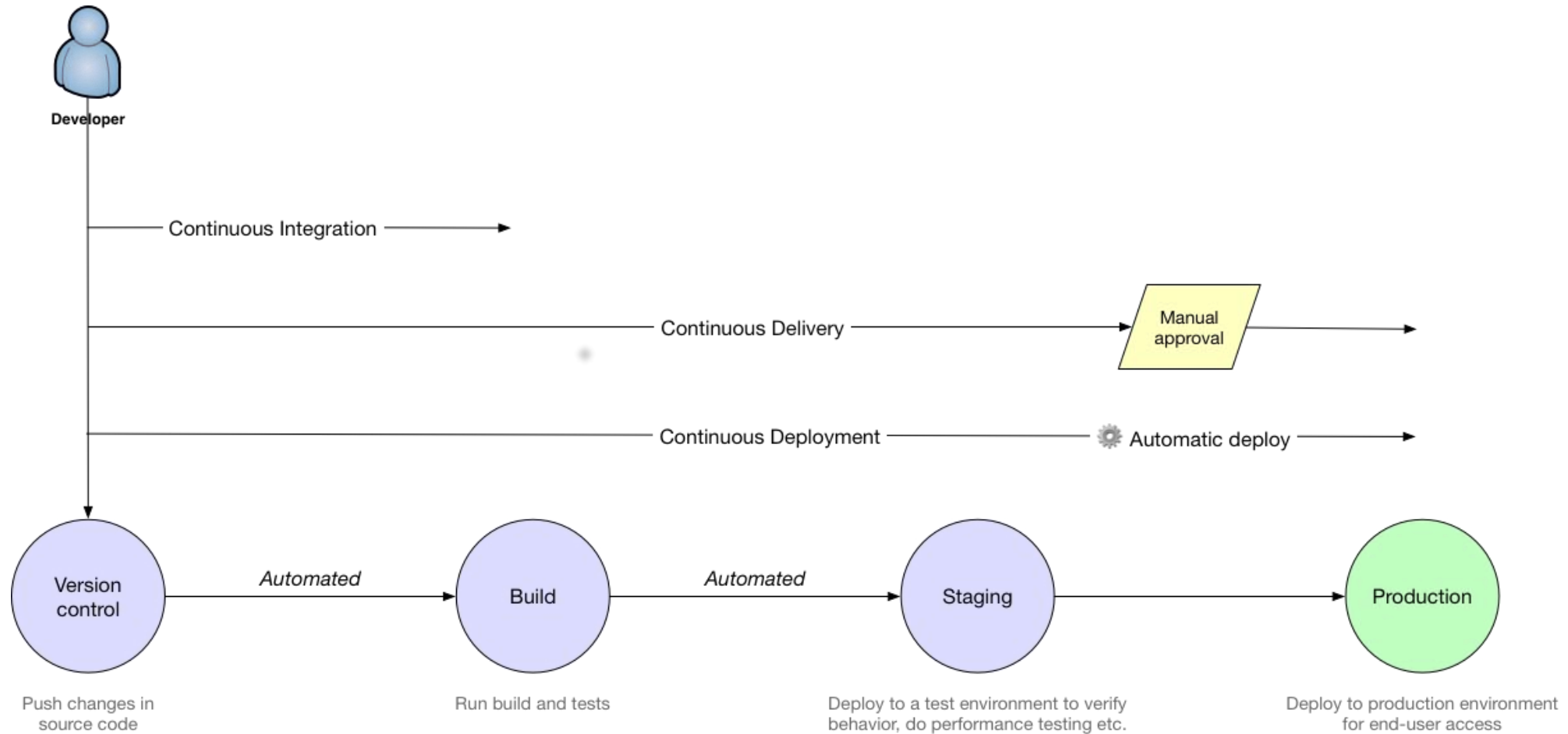
Versioned and stored so it can be distributed as is, if desired

- *Deployed*

Loaded onto a system where the developers can interact with it



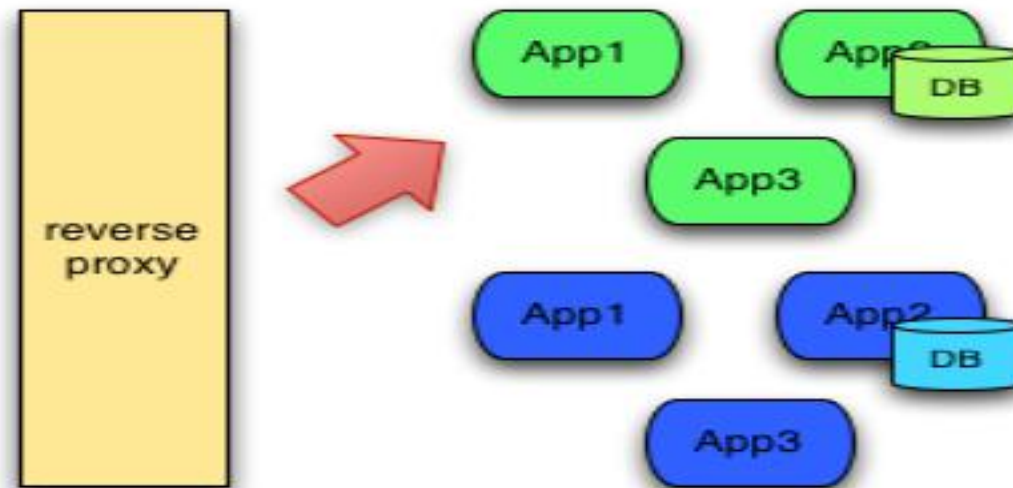
continuous integration , delivery , deployment



Blue/green deployment



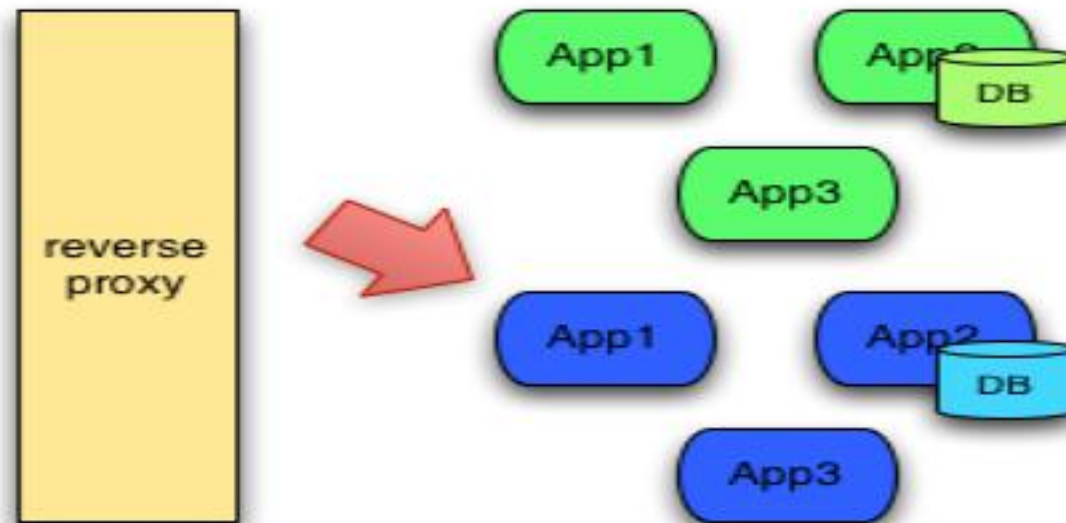
Simply, you have two identical environments (infrastructure) with the “green” environment hosting the current production apps (app1 version1, app2 version1, app3 version1 for example):



Blue/green deployment



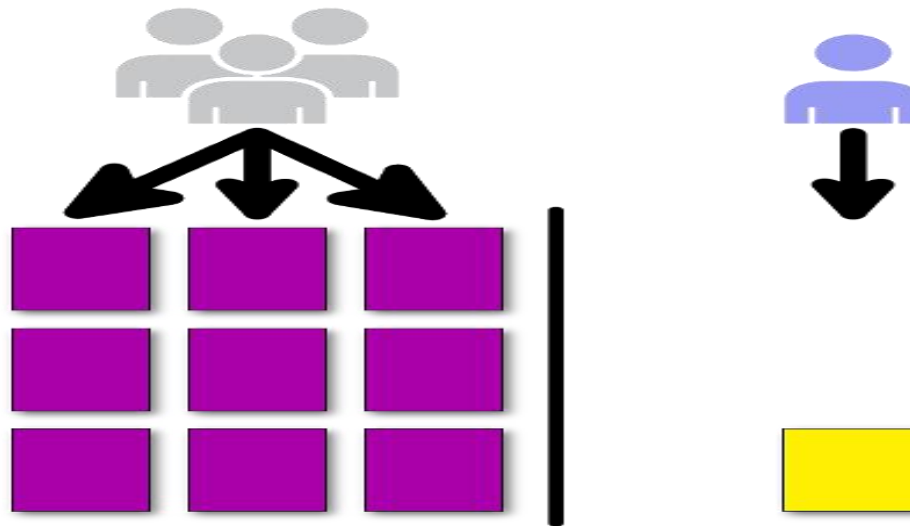
Now, when you're ready to make a change to app2 for example and upgrade it to v2, you'd do so in the "blue environment". In that environment you deploy the new version of the app, run smoke tests, and any other tests (including those to exercise/prime the OS, cache, CPU, etc). When things look good, you change the loadbalancer/reverse proxy/router to point to the blue environment:



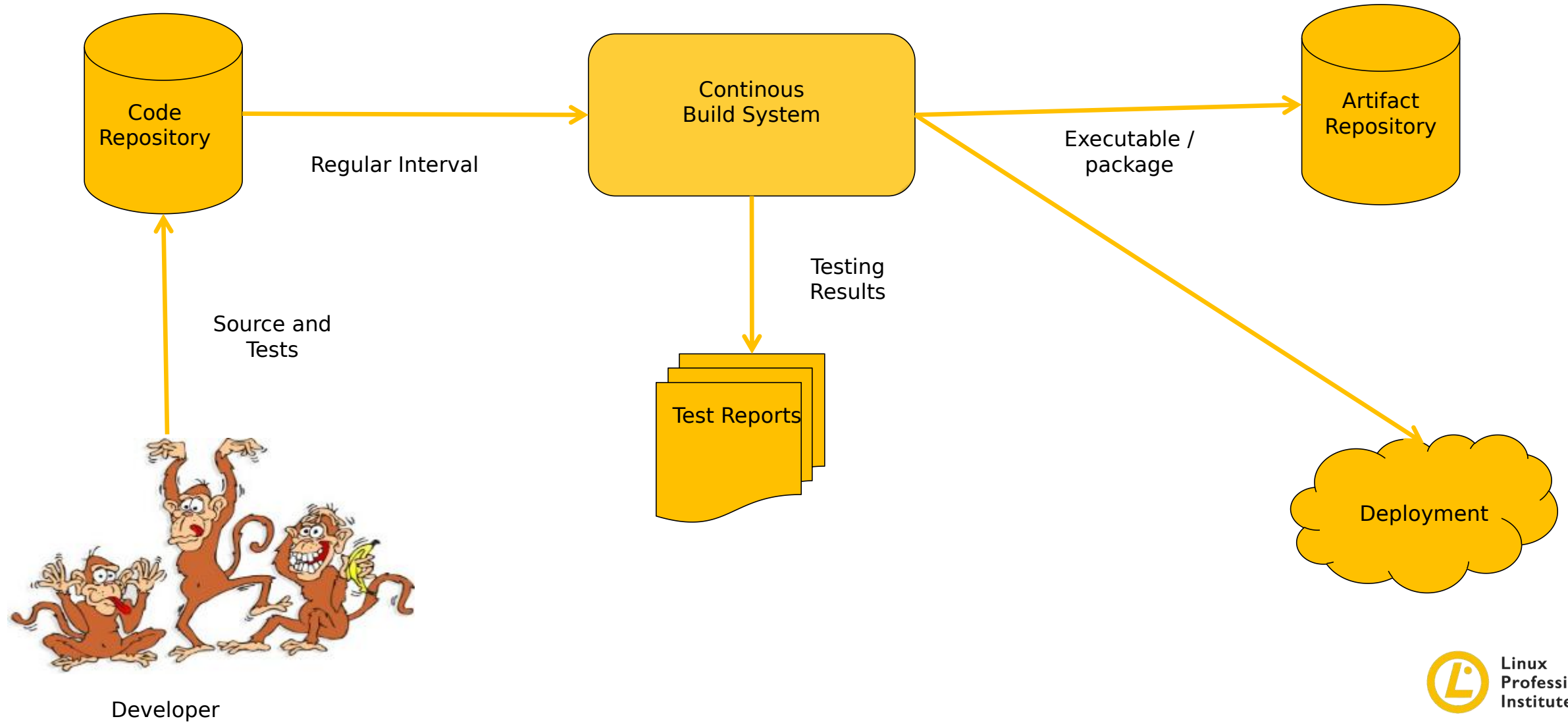
Canary deployment



Canary is about deploying an application *in small, incremental steps*, and *only to a small group of people*. There are a few possible approaches, with the simplest being to serve only some percentage of the traffic to the new application, to a more complicated solutions, such as a *feature toggle*. A feature toggle allows you to gate access to certain features based on specific criteria (e.g., gender, age, country of origin). The most advanced feature toggle I am aware of, *gatekeeper*, is implemented at Facebook.



CI- Workflow



Improving Your Productivity

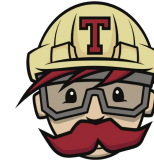


- Continuous integration can help you go faster:
 - Detect build breaks sooner
 - Report failing tests more clearly
 - Make progress more visible

Improving Your Productivity



- *Code Repositories :*
 - SVN , Mercurial , Git
- *Continuous Build Systems:*
 - Jenkins , Bombo , Cercle-CI,...
- *Continuous Test Frameworks:*
 - Junit , CppUnit, PHPUnit ...
- *Artifact Repositories:*
 - Nexus , Artifactory, Archiva



Travis CI



Jenkins for Continuous Integration



- Jenkins – open source continuous integration server
- Jenkins (<http://jenkins-ci.org/>) is
 - Easy to install
 - Easy to use
 - Multi-technology
 - Multi-platform
 - Widely used
 - Extensible
 - Free



- *Easy to install*

- Download one file – jenkins.war
- Run one command – java -jar jenkins.war

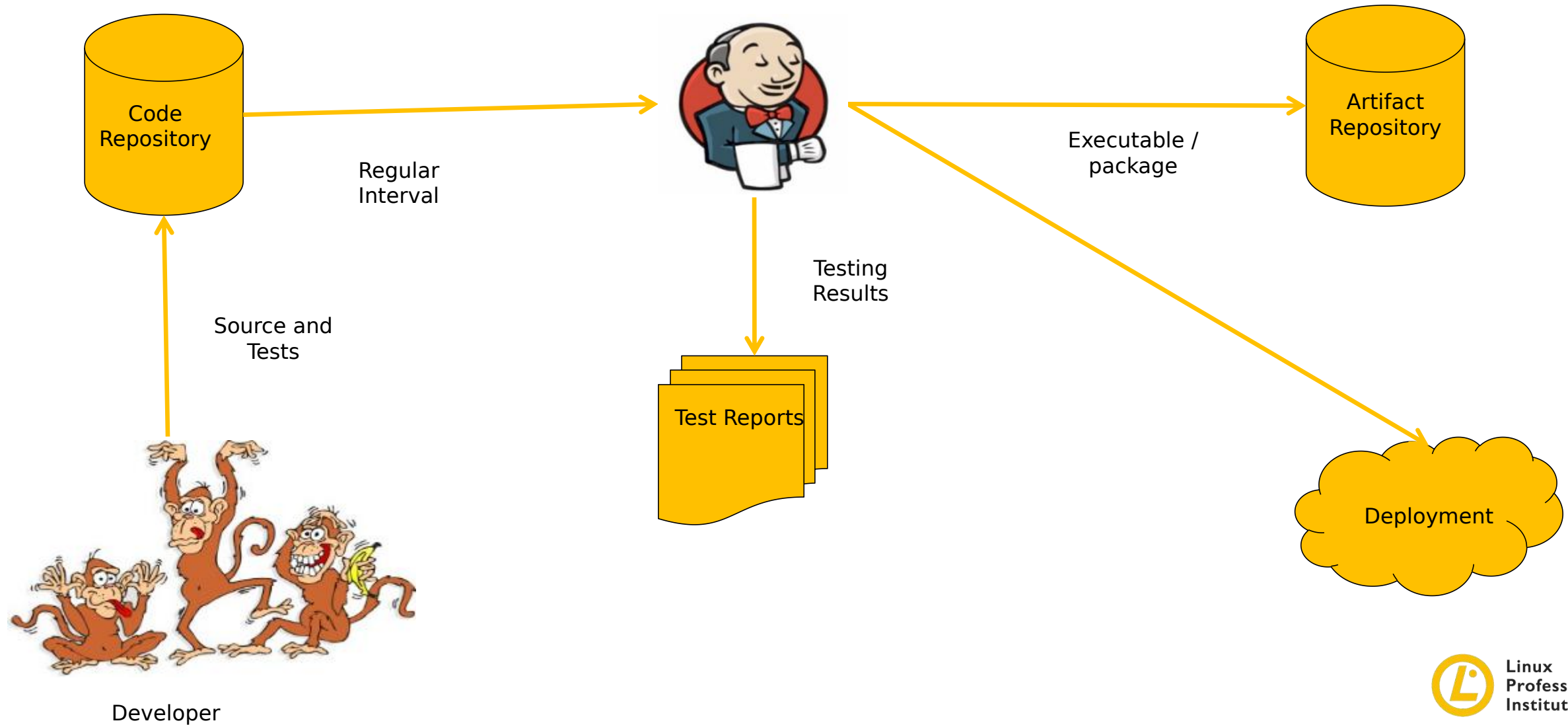
- *Easy to use*

- Create a new job – checkout and build a small project
- Checkin a change – watch it build
- Create a test – watch it build and run
- Fix a test – checkin and watch it pass

- *Multi-technology*

- Build C, Java, C#, Python, Perl, SQL, etc.
- Test with Junit, Nunit, MSTest, etc.

CI- Workflow



Jenkins User Interface



Actions

Nodes

Jobs

The screenshot shows the Jenkins dashboard for 'OBP Automation Team Jenkins'. The left sidebar contains navigation links: New Item, People, Build History, Manage Jenkins, Credentials, My Views, Job Import Plugin, and Dependency Graph. Below these are sections for 'Build Queue' (No builds in the queue) and 'Build Executor Status' (listing 'master' as idle and two offline nodes: 'VIMAL_LAPTOP' and 'VIMAL_VM'). The main area displays a table of jobs with columns for status, weather icon, name, last success, last failure, and last duration. The table lists seven jobs, including 'Automation Stats Failures', 'Automation Stats FW Error', 'Automation Stats Skipped', 'Origination-CS-L1', 'Origination-LNL1', 'Origination-OD-L1', and 'Transaction Reversals'. At the bottom, there is a legend for RSS feeds and a footer indicating the page was generated on Sep 23, 2014.

S	W	Name	Last Success	Last Failure	Last Duration
		Automation Stats Failures	18 days - #43	19 days - #24	7 sec
		Automation Stats FW Error	18 days - #23	N/A	7 sec
		Automation Stats Skipped	18 days - #29	N/A	6.7 sec
		Origination-CS-L1	N/A	13 days - #40	10 sec
		Origination-LNL1	N/A	N/A	N/A
		Origination-OD-L1	N/A	N/A	N/A
		Origination-TD-L1	N/A	N/A	N/A
		Transaction Reversals	1 day 0 hr - #7	N/A	1 hr 3 min

Developer demo goes here...



- Create a new job from a Subversion repository
- Build that code, see build results
- Run its tests, see test results
- Make a change and watch it run through the system
- Languages
 - Java
 - C
 - Python

More Power – Jenkins Plugins



- Jenkins has over 1000 plugins:
 - Software configuration management
 - Builders
 - Test Frameworks
 - Virtual Machine Controllers
 - Notifiers
 - Static Analyzers



- Version Control Systems :

- Accurev
- Bazaar
- BitKeeper
- ClearCase
- Darcs
- Dimensions
- Git
- Harvest
- MKS Integrity
- PVCS
- StarTeam
- Subversion
- Team Foundation Server
- Visual SourceSafe



- Build Tools :

- Ant
- Maven
- MSBuild
- Cmake
- Gradle
- Grails
- Scons
- Groovy

- Test Frameworks:

- Junit
- Nunit
- MStest
- Selenium
- Fitnesse



- Static Analysis ;

- Checkstyle
- CodeScanner
- DRY
- Crap4j
- Findbugs
- PMD
- Fortify
- Sonar
- FXCop

- Code Coverage:

- Emma
- Cobertura
- Clover
- GCC/GCOV



- Notification:

- Twitter
- Campfire
- Google Calendar
- IM
- IRC
- Lava Lamp
- Sounds
- Speak

- Authorization

- Active Directory
- LDAP

- Virtual Machines

- Amazon EC2
- VMWare
- VirtualBox
- Xen
- Libvirt

Jenkins - Integration for You



- Jenkins can help your development be:
 - Faster
 - Safer
 - Easier
 - Smarter

Declarative Pipelines



Pipelines can now be defined with a simpler syntax:

- Declarative “section” blocks for common configuration areas, like...
- Stages
- Tools
- Post-build actions
- Notifications
- Environment
- Build agent or Docker image and more to come!
- All wrapped up in a pipeline `{ ... }` step, with syntactic and semantic validation available.

Declarative Pipelines



This is not a separate thing from Pipeline. It's part of **Pipeline**:

- In fact, it's actually even still Groovy. Sort of. =)
- Configured and run from a **Jenkinsfile**.
- Step syntax is valid within the pipeline block and outside it.
- But this does make some things easier:
- Notifications and postBuild actions are run at the end of your build even if the build has failed.
- Agent provides simpler control over where your build runs.
- You'll see more as we keep going!

Declarative Pipelines



What does this look like?

```
pipeline {  
  agent { docker { image 'golang' } }  
  stages {  
    stage('build') {  
      steps {  
        sh 'go version'  
      }  
    }  
  }  
}
```



So what goes in the pipeline block ?

- What we're calling "sections"
- Name of the section and the value for that section
- Current sections:
- Stages
- Agent
- Environment
- Tools
- Post Build
- Notifications



Stages:

The stages section contains one or more stage blocks:

- Stage blocks look the same as the new block-scoped stage step.
- Think of each stage block as like an individual Build Step in a Freestyle job.
- There must be a stages section present in your pipeline block.

Example:

```
stages {  
  stage("build") {  
    timeout(time: 5, units: 'MINUTES') {  
      sh './run-some-script.sh'  
    }  
  }  
  stage("deploy") {  
    sh './deploy-something.sh'  
  }  
}
```



Agent:

- Agent determines where your build runs.
- Current possible settings:
- Agent label:” - Run on any node
- Agent docker:’ubuntu’ - Run on any node within a Docker container of the “ubuntu” image
- Agent docker:’ubuntu’, label:’foo’ - Run on a node with the label “foo” within a Docker container of the “ubuntu” image
- Agent none - Don’t run on a node at all - manage node blocks yourself within your stages.
- We are planning to make this extensible and composable going forward.
- There must be an agent section in your pipeline block.



Tools:

- The tools section allows you to define tools to autoinstall and add to the PATH.
- Note - this doesn't work with agent docker:'ubuntu'.
- Note - this will be ignored if agent none is specified.
- The tools section takes a block of tool name/tool version pairs, where the tool
- version is what you've configured on this master.

Example:

```
tools {  
  maven "Maven 3.3.9"  
  jdk "Oracle JDK 8u40"  
}
```



Build condition blocks:

- Build Condition is an extension point.
- Implementations provide:
 - A condition name
 - A method to check whether the condition has been satisfied with the current build status.
- Built-in conditions are listed on the right.

Name	Satisfied When
success	the build is successful
failure	the build has failed
unstable	the build is unstable
changed	the build's status is different than the previous build
always	Always true



Notifications and postBuild examples:

```
notifications {  
  success {  
    hipchatSend "Build passed"  
  }  
  failure {  
    hipchatSend "Build failed"  
    mail to:"me@example.com",  
    subject:"Build failed",  
    body:"Fix me please!"  
  }  
}
```

```
-----  
  
postBuild {  
  always {  
    archive "target/**/*"  
    junit 'path/to/*.xml'  
  }  
  failure {  
    sh './cleanup-failure.sh'  
  }  
}
```

Declarative Pipelines



A real-world example with tools, postBuild and notifications

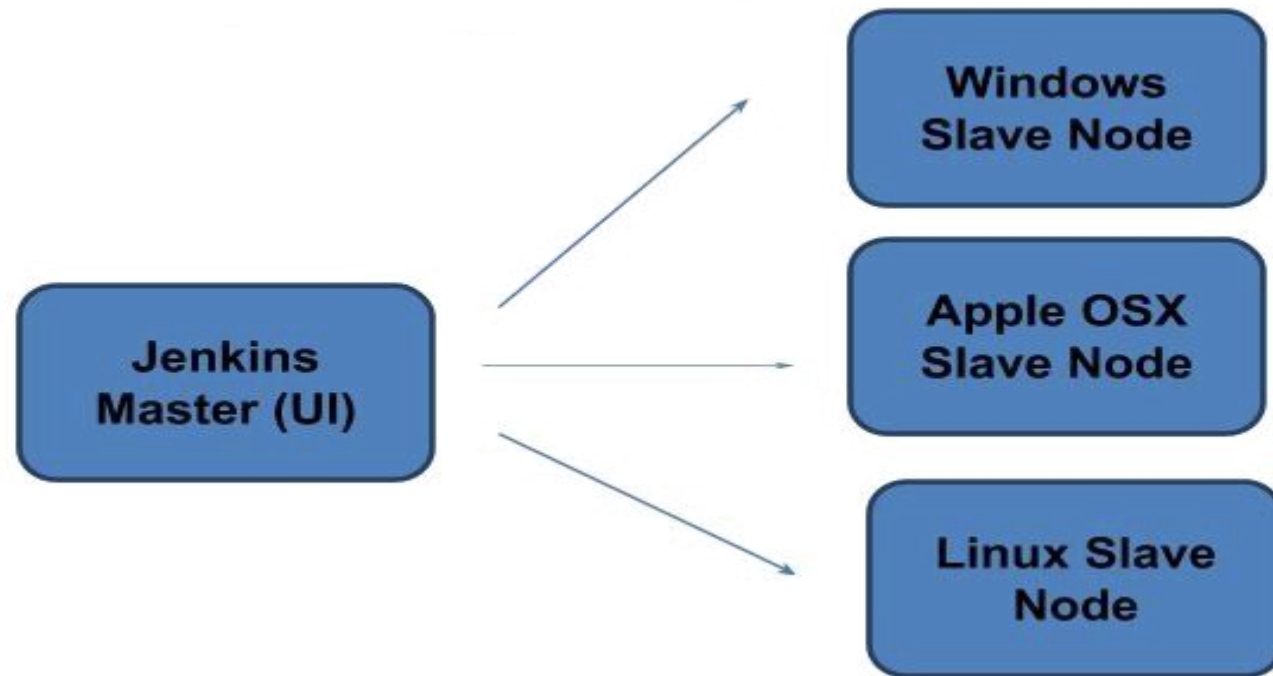
```
pipeline{
  tools {
    maven "Maven 3.3.9"
    jdk "oracle JDK 8u40"
  }
  agent label:"" // run on any excuter
  stages {
    stage("build") {
      sh "mvn clean install -Dmaven.test.failure.ignore=true"
    }
  }
  postBuild {
    always {
      archive "target/**/*"
    }
  }
  junit 'target/surefire-reports/*.xml'
}

notification {
  success { mail(to:"assakra.radhouen@gmail.com", subject:"SUCCESS: ${currentBuild.fullDisplayName}", body:"Huh, we're success.") }
  failure { mail(to:"assakra.radhouen@gmail.com", subject:"FAILURE: ${currentBuild.fullDisplayName}", body:"Huh, we're failure.") }
  unstable{ mail(to:"assakra.radhouen@gmail.com", subject:"UNSTABLE: ${currentBuild.fullDisplayName}", body:"Huh, we're unstable.") }
}
}
```


Declarative Pipelines



Master/slave architecture:





Master/slave architecture:

- **Jenkins Master:**

- Scheduling build jobs.
- Dispatching builds to the slaves for the execution.
- Monitor the slaves.
- Recording and presenting the build results.
- Can also execute build jobs directly.

Jenkins Slave:

- It hears requests from the Jenkins Master instance.
- Slaves can run on a variety of operating systems.
- The job of a Slave is to do as they are told to, which involves executing build jobs dispatched by the Master.
- We can configure a project to always run on a particular Slave machine or a particular type of Slave machine, or simply let Jenkins pick the next available Slave

Declarative Pipelines



Parallel execution on multiple OSes

```
pipeline{
  agent none
  stages {
    stage("distribute") {
      parallel (
"windows":{
          node('windows') {
            bat "print from windows"
          }
        },
"mac":{
          node('osx') {
            sh "print from mac"
          }
        },
"linux":{
          node('linux') {
            sh "print from linux"
          }
        }
      )
    }
  }
}
```



Linux
Professional
Institute

Module 9

Ansible and configuration management tools

Plan

- **Configuration management tools**
- **Ansible**
- **Inventory**
- **Playbook**
- **Variables**
- **Template module (Jinja2)**
- **Roles**
- **ansible-vault**
- **Puppet**
- **Chef**

Configuration management tools



DevOps is evolving and gaining traction as organizations discover how it enables them to produce better applications and reduce their software products' time to market.

DevOps' core values are Culture, Automation, Measurement, and Sharing (CAMS), and an organization's adherence to them influences how successful it is.

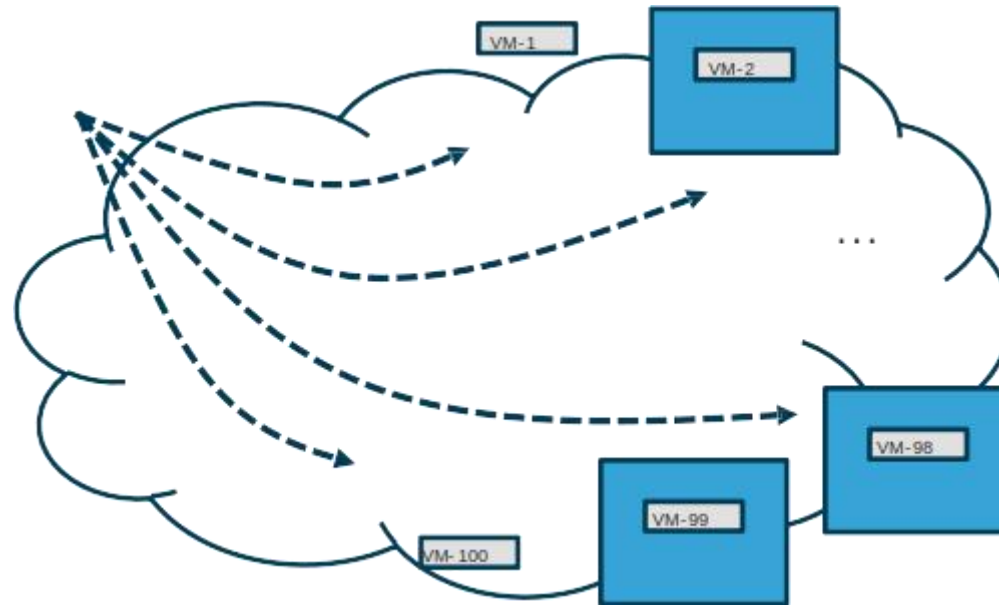
- **Culture** brings people and processes together;
- **Automation** creates a fabric for DevOps;
- **Measurement** permits improvements;
- **Sharing** enables the feedback loop in the CAMS cycle.

Another DevOps concept is the idea that almost everything can be managed in code: servers, databases, networks, log files, application configurations, documentation, automated tests, deployment processes, and more.

Why Ansible:Real-Time Remote Execution of Commands

1. Audit routes on all virtual machines:

```
$ ansible -m shell -a "netstat -rn" datacenter-east
```



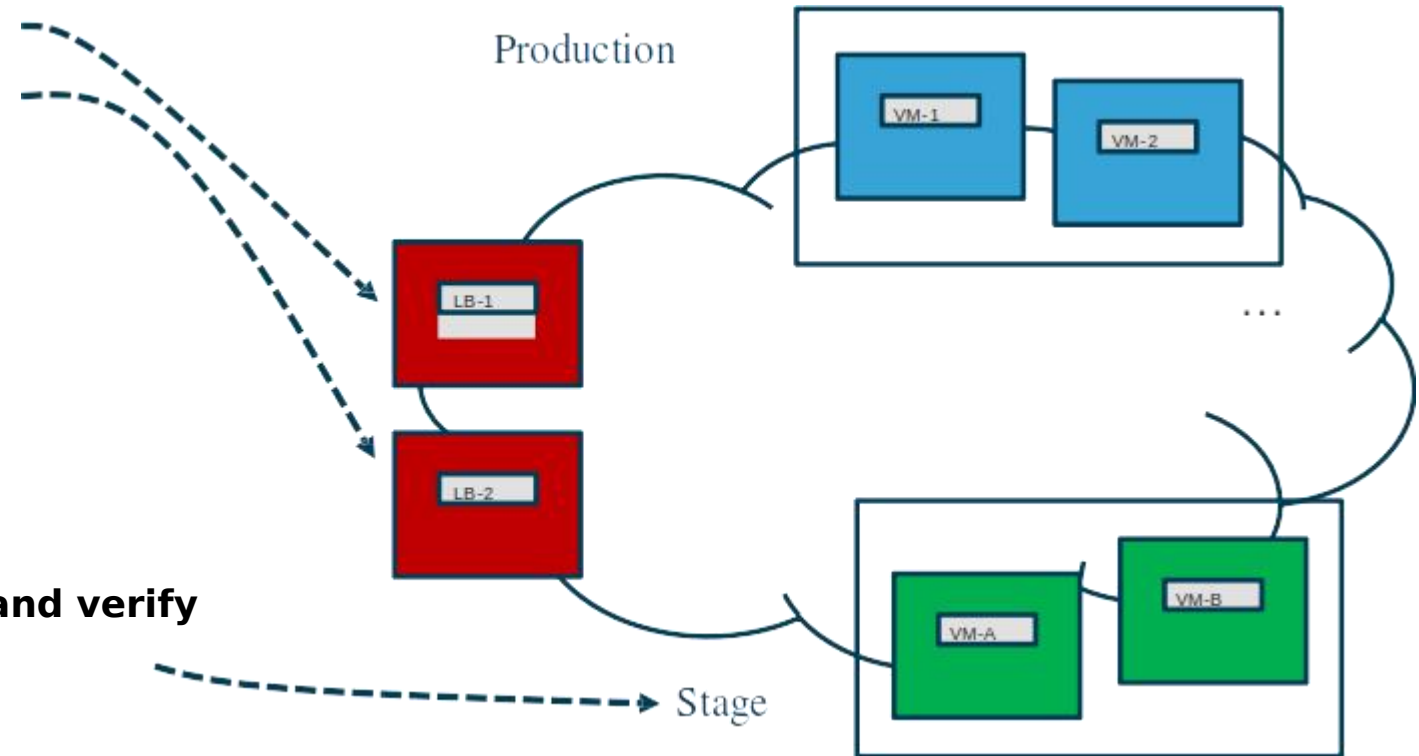
1. Updates routes required for consistency:

```
$ ansible -m shell -a "route add X.X.X.X" datacenter-east
```

1. Update load balancer pools to point to stage

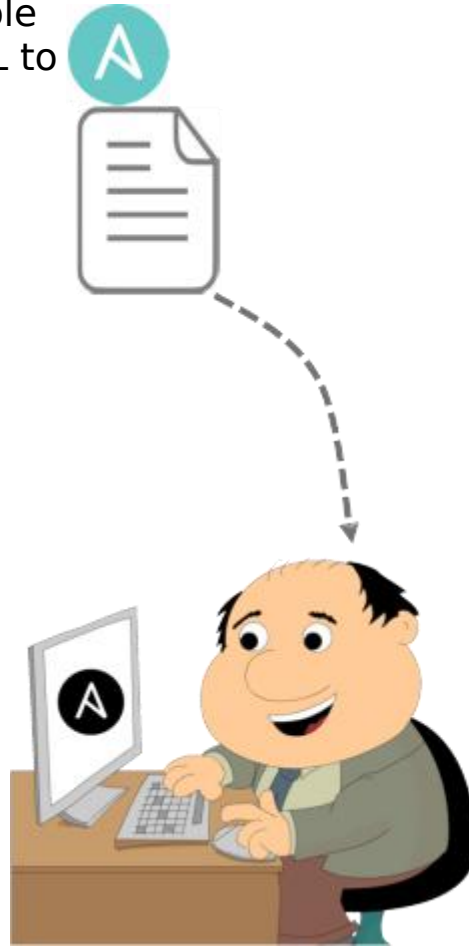


1. Deploy application change to stage and verify

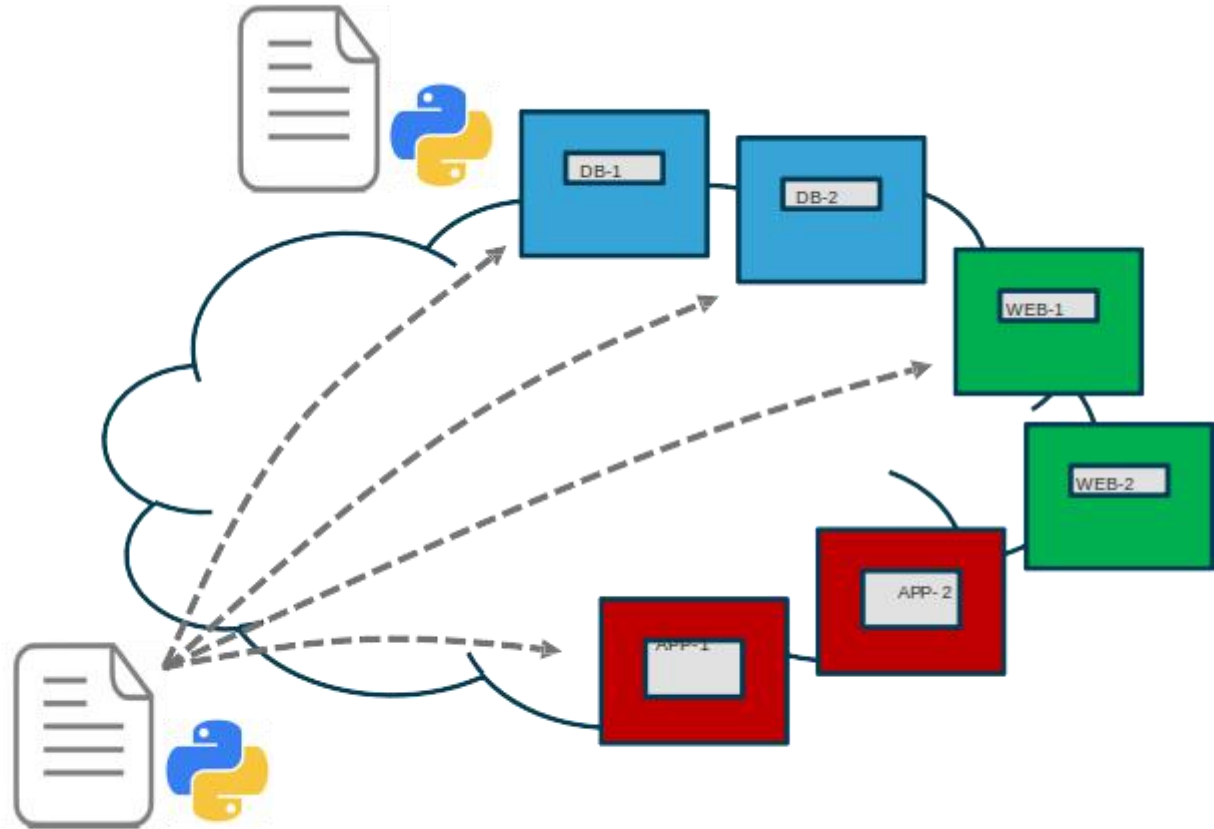


How does Ansible work?

1. Engineers deploy Ansible playbooks written in YAML to a control station



Ansible Control Station



2. Ansible copies modules typically written in Python to remote hosts to execute tasks

- Linux host with a Python and the Ansible installed
- Support transport to remote hosts
 - Typically SSH but could use an API
- Ansible Components
 - Ansible configuration file
 - Inventory files
 - Ansible modules
 - Playbooks



- Control operation of Ansible
- Default configuration
 - /etc/ansible/ansible.cfg
- Override default settings
 - ANSIBLE_CONFIG ENV
 - ansible.cfg in current directory
 - .ansible.cfg in home directory
- See Ansible documentation for all options
- https://docs.ansible.com/ansible/latest/installation_guide/intro_configuration.html

```
DevNet$ cat ansible.cfg

# config file for ansible
# override global certain global settings

[defaults]
# default to inventory file of ./hosts inventory
= ./hosts

# disable host checking to automatically add # hosts to
known_hosts
host_key_checking = False

# set the roles path to the local directory roles_path
= ./
```

- Typically, Ansible uses SSH for authentication and assumes keys are in place
- Setting up and transferring SSH keys allows playbooks to be run automatically
- Using passwords is possible
- Network Devices often use passwords

```
DevNet$ ssh-keygen
```

```
Generating public/private rsa key pair. Enter file in  
which to save the key:  
Enter passphrase (empty for no passphrase): Enter same  
passphrase again:  
Your identification has been saved in ~/.ssh/id_rsa. Your public  
key has been saved in ~/.ssh/id_rsa.pub.
```

```
DevNet$ ssh-copy-id root@10.10.20.20
```

```
Number of key(s) added: 1
```

```
'root@10.10.20.20' "
```

```
DevNet$ ssh root@10.10.20.20
```

```
Last login: Fri Jul 28 13:33:46 2017 from 10.10.20.7 (python2)  
[root@localhost sbx_nxos]#
```

Ansible Inventory File



- Inventory file identifies hosts, and groups of hosts under management
 - Hosts can be IP or FQDN
 - Groups enclosed in []
- Can include host specific parameters as well
- Example: Instructing Ansible to use the active Python Interpreter when using Python Virtual Environments : `$ansible_python_interpreter="/usr/bin/env python"`

Tool	Description
ansible	Executes modules against targeted hosts without creating playbooks.
ansible-playbook	Run playbooks against targeted hosts.
ansible-vault	Encrypt sensitive data into an encrypted YAML file.
ansible-pull	Reverses the normal “push” model and lets clients “pull” from a centralized server for execution.
ansible-docs	Parses the docstrings of Ansible modules to see example syntax and the parameters modules require.
ansible-galaxy	Creates or downloads roles from the Ansible community.

Using Ansible CLI for ad-hoc Commands



- Quickly run a command against a set of hosts
- Specify the module with `-m module`
- Specify the username to use with `-u user`, default is to use local username
- Specify the server or group to target
- Provide module arguments with `-a argument`

```
DevNet$ ansible -m setup -u root servers
10.10.20.20 | SUCCESS => {
  "ansible_facts": {   "ansible_all_ipv4_addresses": [
    "10.10.20.20",
    "172.17.0.1"
  ],
  "ansible_all_ipv6_addresses":
    [ "fe80::250:56ff:febb:3a3f"
  ],
  "ansible_apparmor": {  "status":
    "disabled"
  },
  "ansible_architecture": "x86_64",
  .
  .
}
```



```
---  
- item1  
- item2  
- item3  
- item4
```

YAML
sequences become
Python lists

```
---  
- item1  
- item2  
- item3  
- item4  
---  
- item5  
- item6  
- item7  
- item8
```

Multiple YAML
documents
separates by a

```
---  
key-1: value1  
key-2: value2  
key-3: value3  
key-4: value4
```

YAML mappings
become Python
dictionaries

```
---  
key-1:  
  key-a: value-a  
  key-b: value-b  
  key-c: value-c  
  key-d: value-d  
key-2:  
  key-e: value-e  
  key-f: value-f  
  key-g: value-g  
  key-h: value-h
```

YAML uses spacing to
nest data structures

Tool	Description
module	Code, typically written in Python, that will perform some action on a host. <i>Example: yum - Manages packages with the yum package manager</i>
task	A single action that references a module to run along with any input arguments and actions
play	Matching a set of tasks to a host or group of hosts
playbook	A YAML file that includes one or more play
role	A pre-built set of playbooks designed to perform some standard configuration in a repeatable fashion. A play could leverage a role rather than tasks. <i>Example: A role to configure a web server would install Apache, configure the firewall, and copy application files.</i>

Ansible Playbooks

- Written in YAML
- One or more plays that contain hosts and tasks
- Tasks have a name & module keys.
- Modules have parameters
- Variables referenced with `{{name}}`
 - Ansible gathers “facts”
 - Create your own by register-ing output from another task
- <http://docs.ansible.com/ansible/latest/YAMLSyntax.html>

```
---
- name: Report Hostname and Operating System Details
  hosts: servers

  tasks:
    - name: "Get hostname from server"
      debug:
        msg: "{{ansible_hostname}}"

    - name: "Operating System"
      debug: msg="{{ansible_distribution}}"

- name: Report Network Details of Servers
  hosts: servers

  tasks:
    - name: "Default IPv4 Interface"
      debug: msg="{{ansible_default_ipv4.interface}}"

    - name: "Retrieve network routes"
      command: "netstat -rn"
      register: routes

    - name: "Network routes installed"
      debug: msg="{{routes}}"
```

Ansible Playbooks

```
DevNet$ ansible-playbook -u root example1.yaml

PLAY [Report Hostname and Operating System Details]
*****
TASK [Gathering Facts]
***** ok: [10.10.20.20]

TASK [Get hostname from server]
***** ok: [10.10.20.20] => {
  "msg": "localhost"
}

PLAY [Report Network Details of Servers]
*****
TASK [Network routes installed]
***** ok: [10.10.20.20] => {
  "stdout_lines": [
    "Kernel IP routing table",

    "Destination      Gateway      Genmask      Flags      MSS Window  irtt  Iface",
    "0.0.0.0           10.10.20.254 0.0.0.0      UG         0 0        0     ens160",
    "10.10.20.0        0.0.0.0      255.255.255.0 U          0 0        0     ens160",

    "172.16.30.0       10.10.20.160 255.255.255.0 UG         0 0        0     ens160",
  ]
}

PLAY RECAP
*****
10.10.20.20 : ok=7    changed=1    unreachable=0    failed=0
```

Using Variable Files and Loops with Ansible

- Include external variable files using
- vars_files: filename.yaml
- **Reference variables with {{name}}**
- YAML supports **lists** and **hashes** (ie key/value)
- **Loop to repeat actions with with_items: variable**

example2_vars.yaml

```
---
company_name: "DevNet"
quotes:
  - "DevNet Rocks!"
  - "Programmability is amazing"
  - "Ansible is easy to use"
  - "Lists are fun!"
```

```
---
- name: Illustrate Variables
  hosts: servers
  gather_facts: false
  vars_files:
    - example2_vars.yaml

  tasks:
    - name: "Print Company Name from Variable"
      debug: msg="Hello {{company_name}}"

    - name: "Loop over a List"
      with_items: "{{quotes}}"
      debug: msg="{{item}}"
```

Using Variable Files and Loops with Ansible



```
DevNet$ ansible-playbook -u root example2.yaml

PLAY [Illustrate Variables] *****

TASK [Print Company Name from Variable] ***** ok: [10.10.20.20]
=> {
  "msg": "Hello DevNet"
}

TASK [Loop over a List] ***** ok: [10.10.20.20]
=> (item=DevNet Rocks!) => {
  "item": "DevNet Rocks!", "msg": "DevNet
  Rocks!"
}
ok: [10.10.20.20] => (item=Programmability is amazing) => { "item":
  "Programmability is amazing",
  "msg": "Programmability is amazing"
}
ok: [10.10.20.20] => (item=Ansible is easy to use) => { "item": "Ansible
  is easy to use",
  "msg": "Ansible is easy to use"
}
ok: [10.10.20.20] => (item=Lists are fun!) => { "item": "Lists
  are fun!",
  "msg": "Lists are fun!"
}
```


Using Variable Files and Loops with Ansible

- Include external variable files using
- vars_files: filename.yaml
- **Reference variables with {{name}}**
- YAML supports **lists** and **hashes** (ie key/value)
- **Loop to repeat actions with with_items: variable**

example2_vars.yaml

```
---
company_name: "DevNet"
quotes:
  - "DevNet Rocks!"
  - "Programmability is amazing"
  - "Ansible is easy to use"
  - "Lists are fun!"
```

```
---
- name: Illustrate Variables
  hosts: servers
  gather_facts: false
  vars_files:
    - example2_vars.yaml

  tasks:
    - name: "Print Company Name from Variable"
      debug: msg="Hello {{company_name}}"

    - name: "Loop over a List"
      with_items: "{{quotes}}"
      debug: msg="{{item}}"
```

Using Variable Files and Loops with Ansible



```
DevNet$ ansible-playbook -u root example2.yaml
PLAY [Illustrate Variables] *****

TASK [Print Company Name from Variable] *****
ok: [10.10.20.20] => {
  "msg": "Hello DevNet"
}

TASK [Loop over a List] ***** ok:
[10.10.20.20] => (item=DevNet Rocks!) => {
  "item": "DevNet Rocks!", "msg": "DevNet Rocks!"
}
ok: [10.10.20.20] => (item=Programmability is amazing)
=> { "item": "Programmability is amazing",
  "msg": "Programmability is amazing"
}
ok: [10.10.20.20] => (item=Ansible is easy to use) =>
{ "item": "Ansible is easy to use",
  "msg": "Ansible is easy to use"
}
ok: [10.10.20.20] => (item=Lists are fun!) => { "item":
  "Lists are fun!",
  "msg": "Lists are fun!"
}
```

Using Variable Files and Loops with Ansible



- Not just for Ansible templates
- Powerful templating language
- Loops, conditionals and more supported
- **Leverage template module**
- Attributes
- src: The template file
- dest: Where to save generated template

example3.j2

```
feature {{feature}}  
router bgp {{asn}}  
  router-id {{router_id}}
```

```
---  
- name: Generate Configuration from Template  
  hosts: localhost  
  gather_facts: false  
  vars:  
    feature: bgp  
    asn: 65001  
    router_id: 10.10.10.1  
  
  tasks:  
    - name: "Generate config"  
      template:  
        src: "example3.j2"  
        dest: "./example3.conf"
```


Jinja2 Templating - Variables to the Max!



```
DevNet$ ansible-playbook -u root example3.yaml
PLAY [Generate Configuration from Template]
*****

TASK [Generate config]
*****

changed: [localhost]

PLAY RECAP
*****
***** localhost      : ok=1      changed=1
                        unreachable=0    failed=0

DevNet$ cat example3.conf feature bgp
router bgp 65001
router-id 10.10.10.1
```

- Ansible allows for Group and Host specific variables
 - group_vars/groupname.yaml
 - host_vars/host.yaml
- Variables automatically available

```
├── group_vars
│   ├── all.yaml
│   └── switches.yaml
├── host_vars
│   ├── 172.16.30.101.yaml
│   ├── 172.16.30.102.yaml
│   ├── 172.16.30.103.yaml
│   └── 172.16.30.104.yaml
```

Using Ansible Roles



- roles declares any playbooks defined within a role must be executed.

```
- hosts: dcloud-servers
  roles:
    - { role: geerlingguy.apache }
```

against hosts

Roles promote
playbook reuse



A terminal window showing the command `cd geerlingguy.apache/` followed by `ls`. The output lists the files and directories in the role: `LICENSE`, `README.md`, `defaults`, `handlers`, `meta`, `tasks`, `templates`, `tests`, and `vars`. A blue arrow points from the text 'Roles promote playbook reuse' to the `tasks` directory, and another blue arrow points from the text 'Roles contain playbooks, templates, and variables to complete a workflow (e.g. installing Apache)' to the `tasks` directory.

```
$ cd geerlingguy.apache/
$ ls
LICENSE  README.md  defaults  handlers  meta  tasks  templates  tests  vars
```

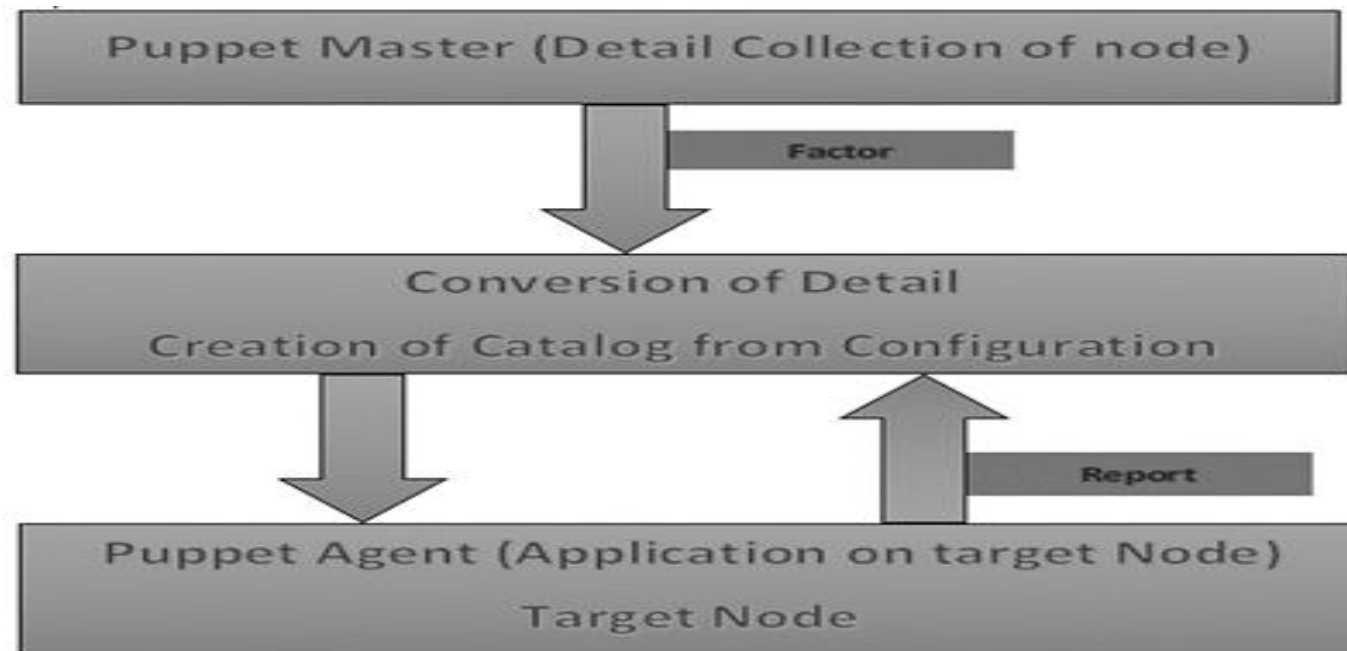
Roles contain playbooks, templates, and variables to complete a workflow (e.g. installing Apache)

- Ansible has an extensive module library capable of operating compute, storage and networking devices
 - http://docs.ansible.com/ansible/modules_by_category.html
- Ansible's domain specific language is powerful
 - Loops
 - Conditionals
 - Many more!
 - <http://docs.ansible.com/ansible/playbooks.html>
- Ansible galaxy contains community supported roles for re-use
 - <https://galaxy.ansible.com/>

What you learn in
this session

- Ansible use cases
- Setting up Ansible infrastructure
- Using the Ansible ad-hoc CLI
- Creating and running Ansible playbooks

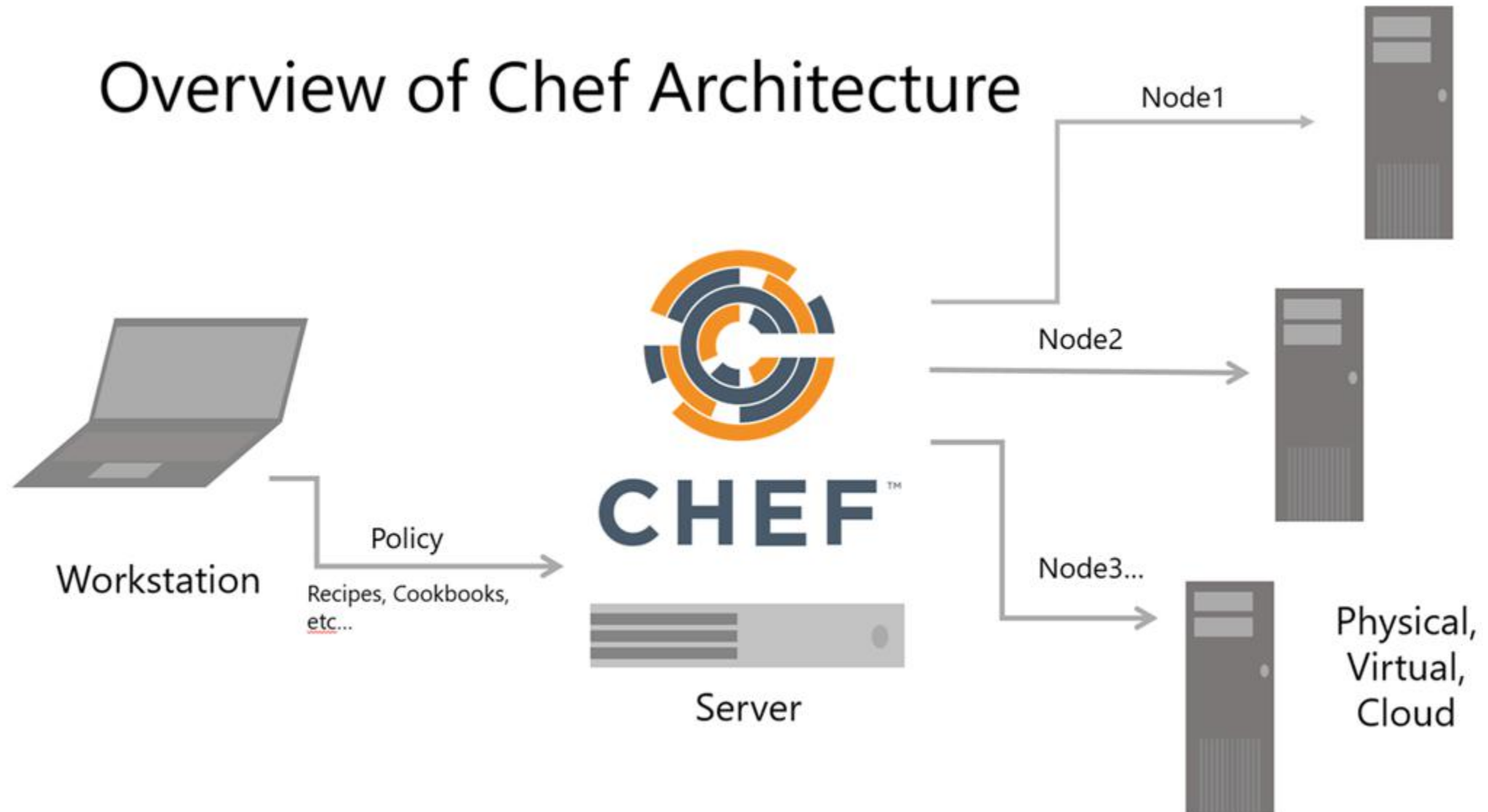
- Puppet is a configuration management technology to manage the infrastructure on physical or virtual machines. It is an open-source software configuration management tool developed using Ruby which helps in managing complex infrastructure on the fly.



- In Puppet, the first thing what the Puppet master does is to collect the details of the target machine. Using the **factor** which is present on all Puppet nodes (similar to **Ohai** in Chef) it gets all the machine level configuration details. These details are collected and sent back to the Puppet master.
- Then the puppet master compares the retrieved **configuration** with defined configuration details, and with the defined configuration it creates a **catalog** and sends it to the targeted Puppet agents.
- The Puppet agent then applies those **configurations** to get the system into a **desired state**.
- Finally, once one has the target node in a desired state, it sends a **report** back to the Puppet master, which helps the Puppet master in understanding where the current state of the system is, as defined in the catalog

- Chef is a configuration management tool for dealing with machine setup on physical servers, virtual machines and in the cloud. Many companies use Chef software to control and manage their infrastructure including Facebook, Etsy, Cheezburger, and Indiegogo.
- As shown in the diagram below, there are three major Chef components:
 - Workstation
 - Server
 - Nodes

Overview of Chef Architecture



Workstation

- The Workstation is the location from which **all of Chef configurations are managed**. This machine holds all the configuration data that can later be pushed to the central Chef Server. These configurations are tested in the workstation before pushing it into the Chef Server. A workstation consists of a command-line tool called **Knife**, that is used to interact with the Chef Server. There can be multiple Workstations that together manage the central Chef Server.
- Workstations are responsible for performing the below functions:
- Writing **Cookbooks** and **Recipes** that will later be pushed to the **central Chef Server**
- Managing **Nodes** on the **central Chef Server**

- Now, let us understand the above mentioned points one by one.

1. Writing Cookbooks and Recipes that will later be pushed to the central Chef Server

- **Recipes: A Recipe is a collection of resources that describes a particular configuration or policy.** It describes everything that is required to configure part of a system. The user writes Recipes that describe how Chef manages applications and utilities (such as Apache HTTP Server, MySQL, or Hadoop) and how they are to be configured.
- These Recipes describe a series of resources that should be in a particular state, i.e. Packages that should be installed, services that should be running, or files that should be written.
- Later in the blog, I will show you how to write a Recipe to install Apache2 package on Chef Nodes by writing a ruby code in Chef Workstation.

#Apache-recipe/apache.rb

#Install & enable Apache

```
package "apache2" do
```

```
  action :install
```

```
end
```

```
service "apache2" do
```

```
  action [:enable, :start]
```

```
end
```

Virtual Host Files

```
node["lamp_stack"]["sites"].each do |sitename, data|
```

```
end
```

Cookbooks: Multiple Recipes can be grouped together to form a Cookbook. A Cookbook defines a scenario and contains everything that is required to support that scenario:

- Recipes, which specifies the resources to use and the order in which they are to be applied
- Attribute values
- File distributions
- Templates
- Extensions to Chef, such as libraries, definitions, and custom resources

2- Managing Nodes on the central Chef Server

The Workstation system will have the required command line utilities, to control and manage every aspect of the central Chef Server. Things like adding a new Node to the central Chef Server, deleting a Node from the central Chef Server, modifying Node configurations etc can all be managed from the Workstation itself.

Workstations have two major components:

- **Knife utility:** This command line tool can be used to communicate with the **central Chef Server from Workstation**. Adding, removing, changing configurations of Nodes in a central Chef Server will be carried out by using this Knife utility. Using the Knife utility, Cookbooks can be uploaded to a central Chef Server and Roles, environments can also be managed. Basically, every aspect of the central Chef Server can be controlled from Workstation using Knife utility.
- **A local Chef repository:** This is the place where every configuration component of central Chef Server is stored. This Chef repository can be synchronized with the central Chef Server (again using the knife utility itself).

- The Chef Server **acts as a hub for configuration data**. The Chef Server **stores Cookbooks**, the policies that are applied to Nodes, and metadata that describes each registered Node that is being managed by the Chef-Client.
- Nodes use the **Chef-Client to ask the Chef Server for configuration details**, such as Recipes, Templates, and file distributions. The Chef-Client then does as much of the configuration work as possible on the Nodes themselves (and not on the Chef Server). Each Node has a Chef Client software installed, which will pull down the configuration from the central Chef Server that are applicable to that Node. This scalable approach distributes the configuration effort throughout the organization.

- Nodes can be a cloud based virtual server or a physical server in your own data center, that is managed using central Chef Server. The main component that needs to be present on the Node is an agent that will establish communication with the central Chef Server. This is called Chef Client.
- Chef Client performs the following functions:
 - It is responsible for interacting with the central Chef Server.
 - It manages the initial registration of the Node to the central Chef Server.
 - It pulls down Cookbooks, and applies them on the Node, to configure it.
 - Periodic polling of the central Chef Server to fetch new configuration items, if any.

- **Chef-Solo** is an open source tool that runs locally and allows to provision guest machines using Chef cookbooks without the complication of any Chef client and server configuration. It helps to execute cookbooks on a self-created server.
- **Knife** is Chef's command-line tool to interact with the Chef server. One uses it for uploading cookbooks and managing other aspects of Chef. It provides an interface between the chefDK (Repo) on the local machine and the Chef server. It helps in managing:
 - Chef nodes, Cookbook , Recipe , Environments , Cloud Resources , Cloud Provisioning and installation on Chef client on Chef nodes.
 - Knife provides a set of commands to manage Chef infrastructure.

https://www.tutorialspoint.com/chef/chef_knife_setup.html

- For more details about chef components visit : https://docs.chef.io/chef_overview.html



Linux
Professional
Institute

Module 10

IT monitoring

Plan

- **Why monitor?**
- **What is Prometheus?**
- **Development History**
- **Prometheus Community**
- **Prometheus Installation**
- **Prometheus Architecture**
- **Features and components**
- **Prometheus configuration file**
- **Metric types**
- **Exporters and integration**
- **Grafana**



Why monitor?



- Know when things go wrong “Something looks wrong on this dashboard”
- To call in a human to prevent a business-level issue, or prevent an issue in advance
- Be able to debug and gain insight
- Trending to see changes over time, and drive technical/business decisions
- To feed into other systems/processes (e.g. QA, security, automation)

What is Prometheus?



- Prometheus is a metrics-based time series database, designed for white box monitoring.
- It supports labels (dimensions/tags).
- Alerting and graphing are unified, using the same language.



- Inspired by Google's Borgmon monitoring system.
- Started in 2012 by ex-Googlers working in Soundcloud as an open source project,
- mainly written in Go. Publically launched in early 2015, 1.0 released in July 2016.
- It continues to be independent of any one company, and is incubating with the
- CNCF.



- Prometheus has a very active community.
- Over 250 people have contributed to official repos.
- There over 100 3rd party integration's
- Over 200 articles, talks and blog posts have been written about it.
- It is estimated that over 500 companies use Prometheus in production.



Using pre-compiled binaries

- We provide precompiled binaries for most official Prometheus components. Check out the download section for a list of all available versions.

From source

- For building Prometheus components from source, see the Makefile targets in the respective repository.

Using Docker

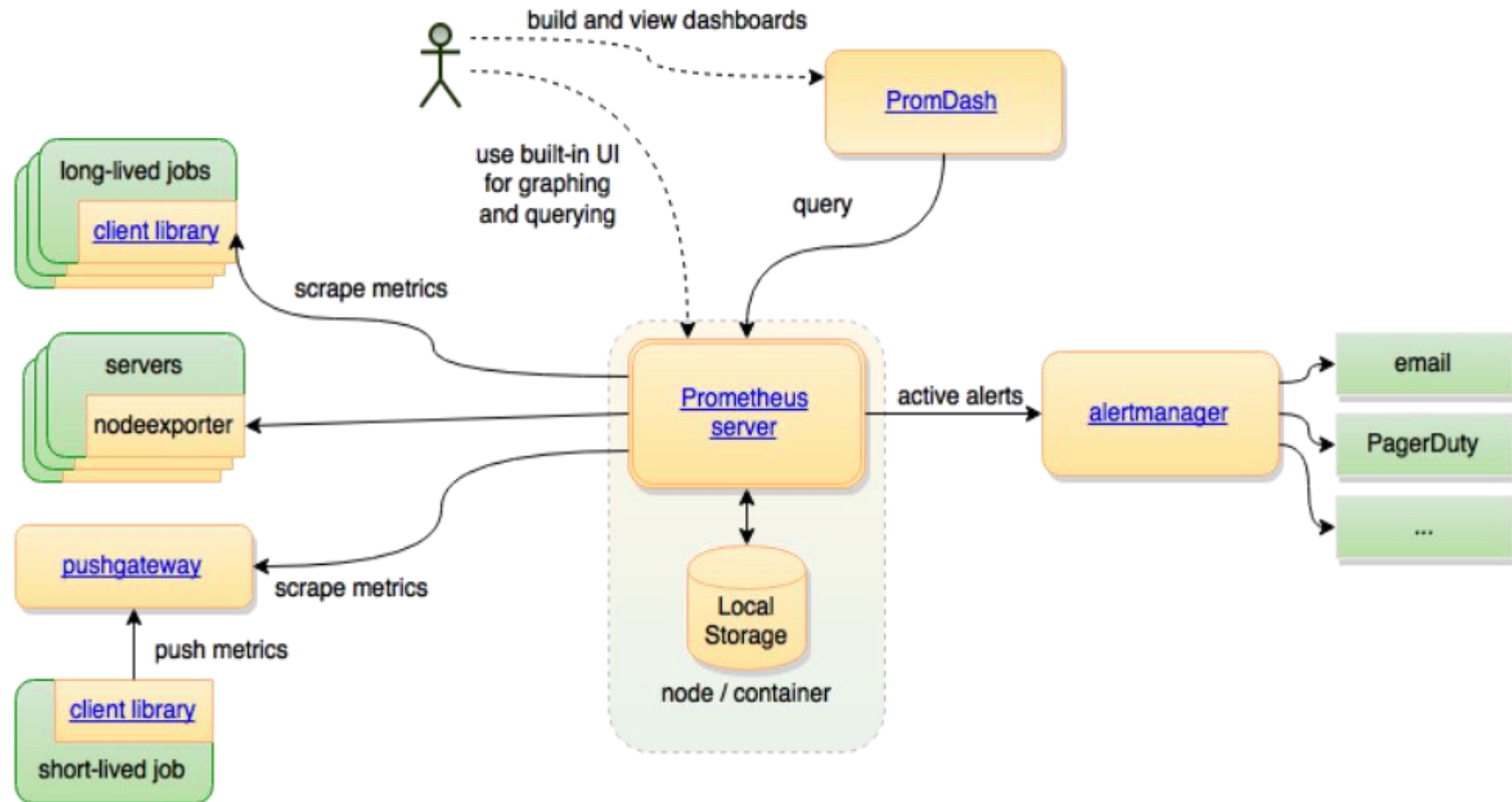
- All Prometheus services are available as Docker images on Quay.io or Docker Hub.
- Running Prometheus on Docker is as simple as `docker run -p 9090:9090 prom/prometheus`. This starts Prometheus with a sample configuration and exposes it on port 9090.



- Has its own time-series database
- Data collection via pull model over HTTP
- Targets are set via static configuration or service discovery
- Metrics have a name, a set of labels, a timestamp and a value

Metric name	Labels	Timestamp	Sample Value
...			
http_requests_total	{status="200",method="GET"}	@1434317560938	94355

Architecture





Prometheus ecosystem consists of multiple components, many of which are optional:

- The main **Prometheus server** which scrapes and stores time series data
- **Client libraries** for instrumenting application code
- A **Push Gateway** for supporting short-lived jobs
- example: Cron jobs, short-lived services , Data that has to be pushed
- special-purpose **Exporters** for services like HAProxy, StatsD, Graphite, etc.
- an **Alertmanager** to handle alerts
- an **node_exporter** :
 - Network, disk, cpu, ram, etc
 - Add your custom metrics (text file)
- various support tools

Most Prometheus components are written in Go, making them easy to build and deploy as static binaries.



Listing 1.11: The default Prometheus configuration file

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

alerting:
  alertmanagers:
    - static_configs:
      - targets:
        # - alertmanager:9093

rule_files:
  # - "first_rules.yml"
  # - "second_rules.yml"

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']
```



Our default configuration file has four YAML blocks defined: **global**, **alerting**, **rule_files**, and **scrape_configs**.

Let's look at each block.

Global : The first block, **global**, contains global settings for controlling the Prometheus server's behavior.

- The first setting, the **scrape_interval** parameter, specifies the interval between scrapes of any application or service—in our case, 15 seconds. This value will be the resolution of your time series, the period in time that each data point in the series covers.
- The **evaluation_interval** tells Prometheus how often to evaluate its rules. Rules come in two major flavors: recording rules and alerting rules:
 - **Recording rules** - Allow you to precompute frequent and expensive expressions and to save their result as derived time series data.
 - **Alerting rules** - Allow you to define alert conditions. With this parameter, Prometheus will (re-)evaluate these rules every 15 seconds. We'll see more about rules in subsequent chapters.



- **Alerting** : alerting is provided by a standalone tool called Alertmanager. Alertmanager is an independent alert management tool that can be clustered.

In our default configuration, the **alerting** block contains the alerting configuration for our server. The **alertmanagers** block lists each Alertmanager used by this Prometheus server. The **static_configs** block indicates we're going to specify any Alertmanagers manually, which we have done in the targets array.

- **Rule files** : The third block, **rule_files**, specifies a list of files that can contain recording or alerting rules.
- **Scrape configuration**: The last block, **scrape_configs**, specifies all of the targets that Prometheus will scrape.



Prometheus's main features are:

- a multi-dimensional data model with time series data identified by metric name and key/value pairs
- **PromQL**, a flexible query language to leverage this dimensionality
- <https://medium.com/@valyala/promql-tutorial-for-beginners-9ab455142085>
- no reliance on distributed storage; single server nodes are autonomous
- time series collection happens via a pull model over HTTP
- pushing time series is supported via an intermediary gateway
- targets are discovered via service discovery or static configuration
- multiple modes of graphing and dashboarding support



The Prometheus client libraries offer four core metric types. These are currently only differentiated in the client libraries (to enable APIs tailored to the usage of the specific types) and in the wire protocol.

- **Counter:** A counter is a cumulative metric that represents a single monotonically increasing counter whose value can only increase or be reset to zero on restart.

Note: “Do not use a counter to expose a value that can decrease”

- **Gauge:** A gauge is a metric that represents a single numerical value that can arbitrarily go up and down.
- **Histogram:** A histogram samples observations (usually things like request durations or response sizes) and counts them in configurable buckets. It also provides a sum of all observed values.
- **Summary:** Similar to a histogram, a summary samples observations (usually things like request durations and response sizes). While it also provides a total count of observations and a sum of all observed values, it calculates configurable quantiles over a sliding time window.



There are a number of libraries and servers which help in exporting existing metrics from third-party systems as Prometheus metrics. This is useful for cases where it is not feasible to instrument a given system with Prometheus metrics directly (for example, HAProxy or Linux system stats).

Third-party exporters:

Some of these exporters are maintained as part of the official [Prometheus GitHub organization](#), those are marked as official, others are externally contributed and maintained.

We encourage the creation of more exporters but cannot vet all of them [for best practices](#). Commonly, those exporters are hosted outside of the Prometheus GitHub organization.

[The exporter default port](#) wiki page has become another catalog of exporters, and may include exporters not listed here due to overlapping functionality or still being in development.

The JMX exporter can export from a wide variety of JVM-based applications, for example Kafka and Cassandra.



Prometheus scrapes metrics from instrumented jobs, either directly or via an intermediary push gateway for short-lived jobs. It stores all scraped samples locally and runs rules over this data to either aggregate and record new time series from existing data or generate alerts. **Grafana** or other API consumers can be used to visualize the collected data.



- Used to query and visualize metrics
- Works with Prometheus, but not only...
- Grafana supports multiple backends
- It is possible to combine data from different sources in the same dashboard
- Fully customizable
- Each panel has a wide variety of styling and formatting options
- Supports templates
- Collection of add-ons and pre-built dashboards



Linux
Professional
Institute

Module 11

Log management and analysis

Plan

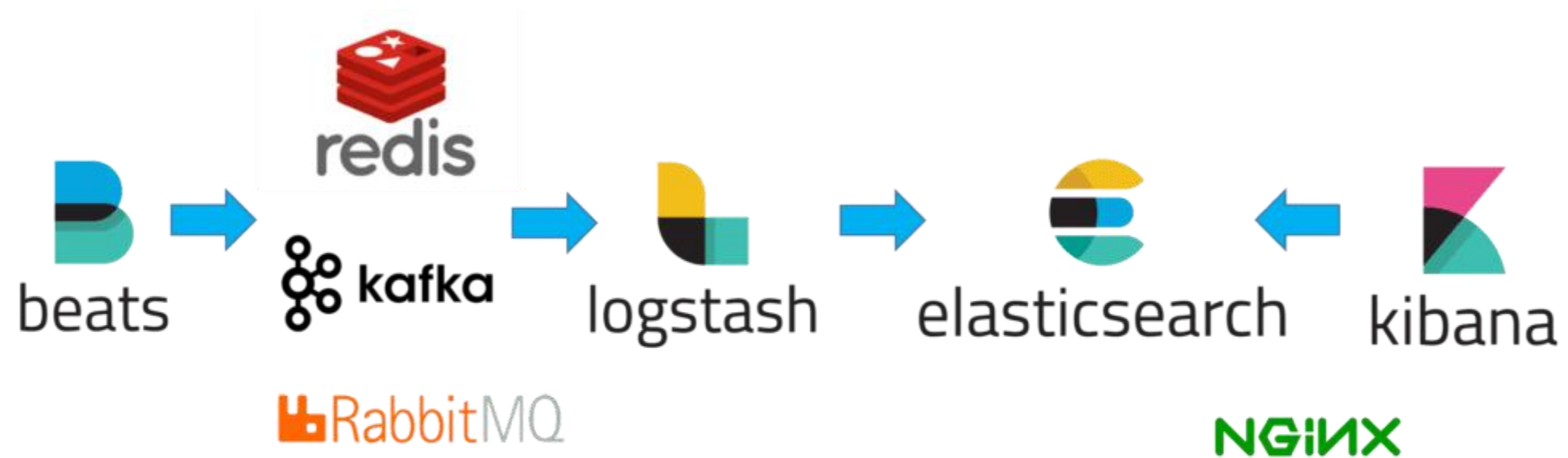
- **Why log analysis**
- **ELK stack**
- **Elasticsearch**
- **Logstash**
- **Kibana**
- **Filebeat**



- Lots of users
- Faculty & staff & students more than 40000 users on campus
- Lots of systems
- Routers, firewalls, servers....
- Lots of logs
- Netflow, syslogs, access logs, service logs, audit logs....
- Nobody cares until something go wrong....

- Log management platform can monitor all above-given issues as well as process operating system logs, NGINX, IIS server log for web traffic analysis, application logs, and logs on cloud.
- Log management helps DevOps engineers, system admin to make better business decisions.
- The performance of virtual machines in the cloud may vary based on the specific loads, environments, and number of active users in the system.
- Therefore, reliability and node failure can become a significant issue

- A collection of three open-source products :
 - E stands for **ElasticSearch**: used for storing logs
 - L stands for **LogStash** : used for both shipping as well as processing and storing logs
 - K stands for **Kibana**: is a visualization tool (a web interface)
- which is hosted through Nginx or Apache Designed to take data from any source, in any format, and to search, analyze, and visualize that data in real time.
- Provides centralized logging that be useful when attempting to identify problems with servers or
- applications.
- It allows user to search all your logs in a single place.



Data
Collection

Buffering

Data
Aggregation
& Processing

Indexing &
storage

Analysis &
visualization

- NoSQL database built with RESTful APIS.
- It offers advanced queries to perform detail analysis and stores all the data centrally.
- Also allows you to store, search and analyze big volume of data.
- Executing a quick search of the documents.
- also offers complex analytics and many advanced features.
- Offers many features and advantages.

Elasticsearch : Features and advantages



Features :

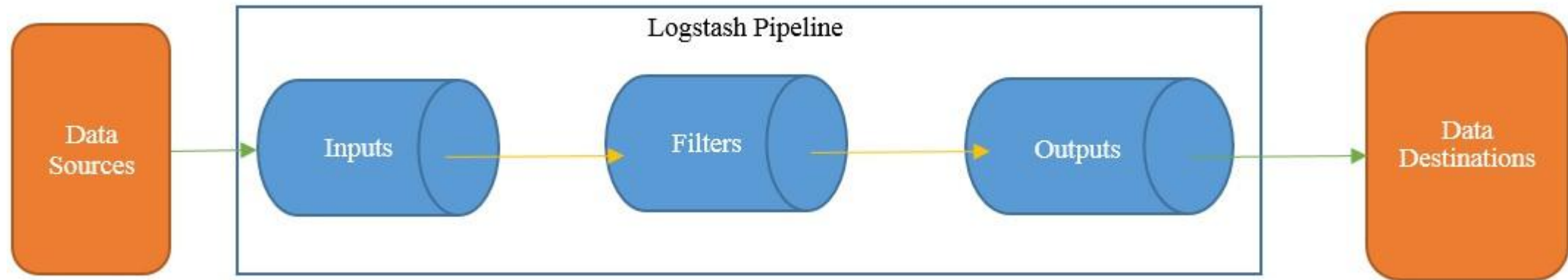
- Open source search server is written using Java
- Used to index any kind of heterogeneous data
- Has REST API web-interface with JSON output
- Full-Text Search
- Shared, replicated searchable, JSON document store
- Multi-language & Geo-location support

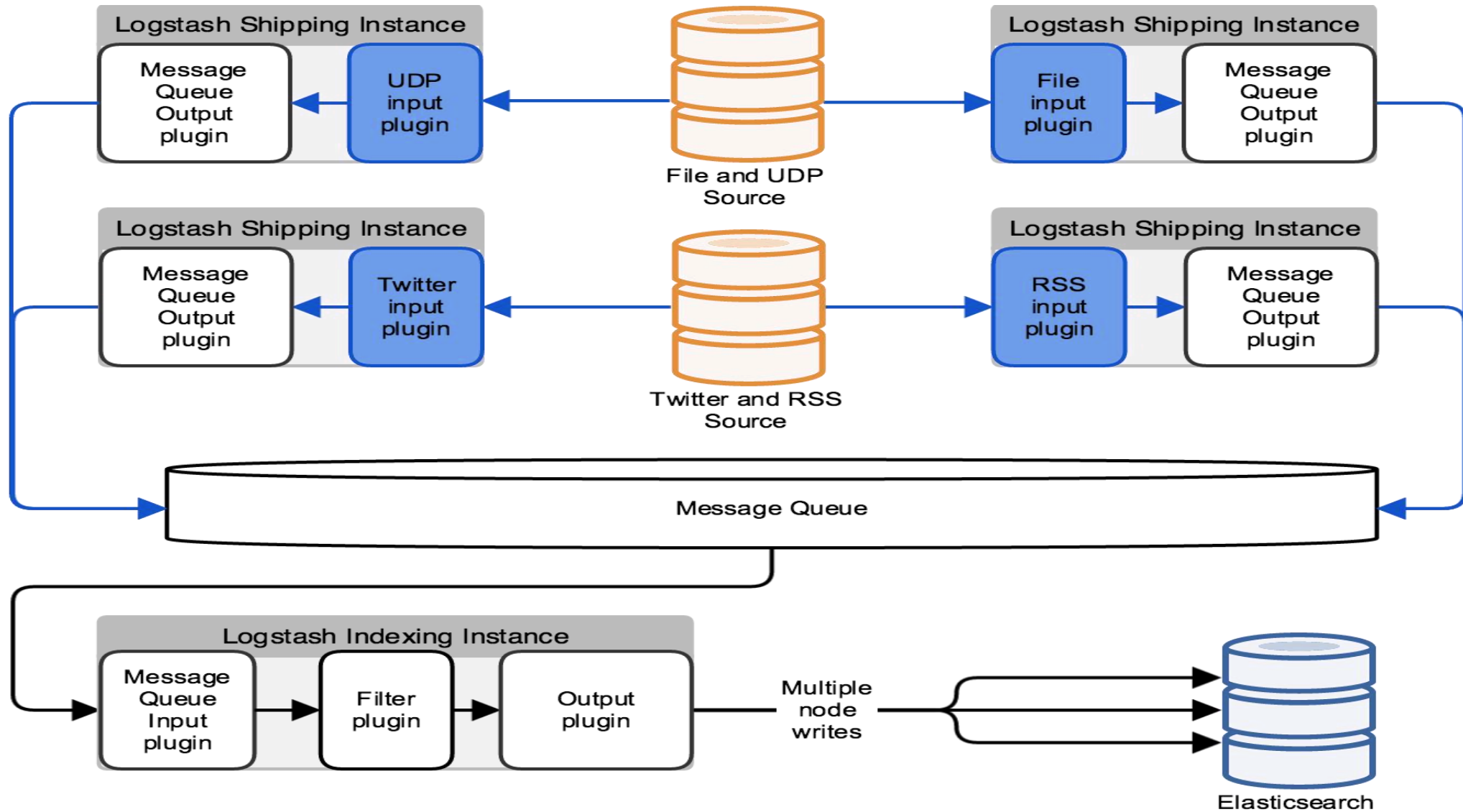
Advantages

- Store schema-less data and also creates a schema for data
- Manipulate data record by record with the help of Multi-document APIs.
- Perform filtering and querying of data for insights
- Based on Apache and provides RESTful API
- Provides horizontal scalability, reliability, and multitenant capability for real time use of indexing to make it faster search

- Cluster : A collection of nodes which together holds data and provides joined indexing and search capabilities.
- Node : An elasticsearch Instance. It is created when an elasticsearch instance begins.
- Index : A collection of documents which has similar characteristics.
 - e.g., customer data, product catalog.
- It is very useful while performing indexing, search, update, and delete operations.
- Document : The basic unit of information which can be indexed. It is expressed in JSON (key: value) pair.
- '{"user": "nullcon"}'. Every single Document is associated with a type and a unique id

- It is the data collection pipeline tool.
- It collects data inputs and feeds into the Elasticsearch.
- It gathers all types of data from the different source and makes it available for further use.
- Logstash can unify data from disparate sources and normalize the data into your desired destinations.
- It consists of three components:
 - **Input** : passing logs to process them into machine understandable format.
 - **Filters** : It is a set of conditions to perform a particular action or event.
 - **Output** : Decision maker for processed event or log





Inputs

You use inputs to get data into Logstash. Some of the more commonly-used inputs are:

- **file:** reads from a file on the filesystem, much like the UNIX command `tail -OF`
- **syslog:** listens on the well-known port 514 for syslog messages and parses according to the RFC3164 format
- **redis:** reads from a redis server, using both redis channels and redis lists. Redis is often used as a "broker" in a centralized Logstash installation, which queues Logstash events from remote Logstash "shippers".
- **beats:** processes events sent by Beats.
- For more information about the available inputs, see [Input Plugins](#).

Logstash Inputs : File , MySql log example



Logstash.conf

```
input {  
  file {  
    path => ["/var/log/mysql/mysql.log", "/var/log/mysql/mysql-slow.log", "/var/log/mysql/mysql_error.log"]  
    type => "mysql"  
  }  
}
```

Logstash.conf

```
input {  
  syslog {  
    port => 12345  
    codec => cef  
    syslog_field => "syslog"  
    grok_pattern => "<{%{POSINT:priority}>{%{SYSLOGTIMESTAMP:timestamp}} CUSTOM GROK HERE"  
  }  
}
```

Logstash Inputs : Redis



Logstash.conf

```
input {  
  redis {  
    id => "my_plugin_id"  
  }  
}
```

- Filters

Filters are intermediary processing devices in the Logstash pipeline. You can combine filters with conditionals to perform an action on an event if it meets certain criteria. Some useful filters include:

- **Grok:** parse and structure arbitrary text. Grok is currently the best way in Logstash to parse unstructured log data into something structured and queryable. With 120 patterns built-in to Logstash, it's more than likely you'll find one that meets your needs!
- **Mutate:** perform general transformations on event fields. You can rename, remove, replace, and modify fields in your events.
- **Drop:** drop an event completely, for example, debug events.
- **Clone:** make a copy of an event, possibly adding or removing fields.
- **Geoip:** add information about geographical location of IP addresses (also displays amazing charts in Kibana!)
- For more information about the available filters, see [Filter Plugins](#).

Logstash.conf

```
filter {  
  grok {  
    match => { "message" => "%{IP:client} %{WORD:method} %{URIPATHPARAM:request}  
%{NUMBER:bytes} %{NUMBER:duration}" }  
  }  
}
```

Why:

A filter within Logstash: used to parse unstructured data to something structured(exp: json)

Library of terms that wrap Regular Expressions that match text patterns and match lines in log files(exp:

INT(?:[+-]?(?:[0-9]+)))

Grok Syntax:

“%{GROK1:semantic1}%{GROK2:semantic2}%{GROK3:semantic3}”

Logstash Filtres :GROK



Test Grok pattern online :

<https://grokdebug.herokuapp.com/>

For more details :

<https://logz.io/blog/logstash-grok/>

[GrokPatterns_title](#)

<https://github.com/elastic/logstash/blob/v1.4.2/patterns/grok-patterns>

Logstash.conf

```
filter {  
  mutate {  
    split => ["hostname", "."]  
    add_field => { "shortHostname" => "%{hostname[0]}" }  
  }  
  mutate {  
    rename => ["shortHostname", "hostname" ]  
  }  
}
```


Outputs are the final phase of the Logstash pipeline. An event can pass through multiple outputs, but once all output processing is complete, the event has finished its execution. Some commonly used outputs include:

Elasticsearch: send event data to Elasticsearch. If you're planning to save your data in an efficient, convenient, and easily queryable format... Elasticsearch is the way to go. Period. Yes, we're biased :)

File: write event data to a file on disk.

Graphite: send event data to graphite, a popular open source tool for storing and graphing metrics.

<http://graphite.readthedocs.io/en/latest/>

Statsd: send event data to statsd, a service that "listens for statistics, like counters and timers, sent over UDP and sends aggregates to one or more pluggable backend services". If you're already using statsd, this could be useful for you!

For more information about the available outputs, see [Output Plugins](#).

Logstash : Outputs file example



Logstash.conf

```
input {  
  file {  
    path => "C:/Program Files/Apache Software Foundation/Tomcat 7.0/logs/*access*"  
    type => "apache"  
  }  
}  
filter {  
  grok {  
    match => { "message" => "%{COMBINEDAPACHELOG}" }  
  }  
  date {  
    match => [ "timestamp" , "dd/MMM/yyyy:HH:mm:ss Z" ]  
  }  
}  
output {  
  file {  
    path => "C:/tpwork/logstash/bin/log/output.log"  
  }  
}
```

Run Logstash: `$ logstash\bin> $ logstash -f Logstash.conf`

Features

- Events are passed through each phase using internal queues
- Allows different inputs for your logs
- Filtering/parsing for your logs

Advantages

- Offers centralize the data processing.
- It analyzes a large variety of structured/unstructured data and events.
- Offers plugins to connect with various types of input sources and platforms

- A data visualization which completes the ELK stack.
- Dashboard offers various interactive diagrams, geospatial data, and graphs to visualize complex queries.
- It can be used for search, view, and interact with data stored in Elasticsearch directories.
- It helps users to perform advanced data analysis and visualize their data in a variety of tables, charts, and maps.
- In Kibana there are different methods for performing searches on data.

Features :

- Visualizing indexed information from the elastic cluster
- Enables real-time search of indexed information
- Users can search, View, and interact with data stored in Elasticsearch.
- Execute queries on data & visualize results in charts, tables, and maps.
- Configurable dashboard to slice and dice logstash logs in elasticsearch.
- Providing historical data in the form of graphs, charts, etc.

Advantages :

- Easy visualizing
- Fully integrated with Elasticsearch
- Real-time analysis, charting, summarization, and debugging capabilities
- Provides instinctive and user-friendly interface
- Sharing of snapshots of the logs searched through
- Permits saving the dashboard and managing multiple dashboards

Filebeat : is a log shipper belonging to the Beats family , a group of lightweight shippers installed on hosts for shipping different kinds of data into the ELK Stack for analysis.

- Each beat is dedicated to shipping different types of information.
- Winlogbeat, for example, ships Windows event logs,
- Metricbeat ships host metrics, and so forth.
- Filebeat, as the name implies, ships log files.
- [Filebeat.yml example](#)
- In an ELK-based logging pipeline, Filebeat plays the role of the logging agent , installed on the machine generating the log files, tailing them, and forwarding the data to either Logstash for more advanced processing or directly into Elasticsearch for indexing.

The Beats Family

All kinds of shippers for all kinds of data.



Filebeat

Log Files



Metricbeat

Metrics



Packetbeat

Network Data



Winlogbeat

Windows Event Logs



Auditbeat

Audit Data



Heartbeat

Uptime Monitoring



Functionbeat

Serverless Shipper

- Centralized logging can be useful when attempting to identify problems with servers or applications
- ELK stack is useful to resolve issues related to centralized logging system
- ELK stack is a collection of three open source tools Elasticsearch, Logstash Kibana
- Elasticsearch is a NoSQL database
- Logstash is the data collection pipeline tool .
- Kibana is a data visualization which completes the ELK stack
- In cloud-based environment infrastructures, performance and isolation is very important
- In ELK stack processing speed is strictly limited whereas Splunk offers accurate and speedy processes
- Netflix, LinkedIn, Tripware, Medium all are using ELK stack for their business
- ELK works best when logs from various Apps of an enterprise converge into a single ELK instance
- Different components In the stack can become difficult to handle when you move on to complex setup